Robotik Mühendisliği Serisi — Cilt 1

Matematik ve Mekanik Temeller

Lineer cebir, dönmeler, kinematik, Jacobian ve rijit cisim dinamiği

Türkçe–İngilizce karma konu anlatımı
teori · sayısal örnekler · çalışan Python ve C++ kodları

Odak: bacaklı ve humanoid robotlar

31 Ekim 2026

İçindekiler

Bu Cilt Hakkında **1**

1 Robotikçi Gözüyle Lineer Cebir **2**

1.1 Örnek sistem: düzlemsel bacak ve Jacobian'ın türetilmesi 2

1.1.1 İleri kinematik 2

1.1.2 Jacobian: zincir kuralıyla türetme 2

1.2 Lineer dönüşüm, rank ve null uzayı 3

1.3 SVD: türetme ve geometrik anlam 3

1.3.1 Simetrik matrisler ve özdeğerler 3

1.3.2 SVD'nin inşası 4

1.3.3 Geometrik anlam: hız elipsi 4

1.4 Ters hız problemi: pseudo-inverse ve null uzayı 5

1.4.1 En küçük normlu çözümün türetilmesi 5

1.4.2 Null uzayına izdüşüm 5

1.4.3 Tekillik ve sönümlü en küçük kareler (DLS) 6

1.5 En küçük kareler ve koşulluluk 7

1.5.1 Normal denklemlerin iki türetmesi 7

1.5.2 Neden `solve`, neden QR, neden asla `inv`? 7

2 Dönmeler: SO(3), Euler Açıları, Kuaterniyonlar **13**

2.1 Dönme matrisi ve SO(3) 13

2.1.1 Dönme matrisi nereden gelir? 13

2.1.2 Neden $\mathbf{R}^T\mathbf{R} = \mathbf{I}$ ve $\det \mathbf{R} = +1$? 13

2.1.3 Çerçeve notasyonu ve bileşke 14

2.2 Euler/RPY açıları ve gimbal lock 14

2.2.1 ZYX matrisinin açılımı 14

2.2.2 Gimbal lock'un türetilmesi 14

2.3 Eksen-açı ve üstel harita 15

2.3.1 Çapraz çarpım matrisi 15

2.3.2 Rodrigues formülünün geometrik türetmesi 15

2.3.3 Aynı formül: matris üsteli 15

2.3.4 Logaritma: $\mathbf{R}$'den eksen ve açıyı çıkarmak . . . 16
2.4 Kuaterniyonlar . . . 16
2.4.1 Tanım ve çarpım . . . 16
2.4.2 Birim kuaterniyon = dönme . . . 16
2.4.3 Slerp: iki yönelim arasında sabit hızla geçiş . . . 17
2.5 Açısal hız ve yönelim integrasyonu . . . 17
2.5.1 $\dot{\mathbf{R}}$ nasıl görünür? . . . 17
2.5.2 Naif Euler neden SO(3)'ten çıkar? . . . 17

3 Rijit Cisim Hareketi ve İleri Kinematik 26
3.1 Homojen dönüşümler ve SE(3) . . . 26
3.1.1 Neden 4×4? . . . 26
3.1.2 Bileşke ve ters . . . 26
3.2 Twist ve vida (screw) ekseni . . . 26
3.2.1 $\dot{\mathbf{T}}\mathbf{T}^{-1}$'in hesaplanması . . . 27
3.2.2 Adjoint: twist'i çerçeve değiştirmek . . . 27
3.2.3 Vida ekseni: bir dönel eklemin twist'i . . . 27
3.2.4 Twist'in üsteli: $e^{[\mathbf{S}]\theta}$ . . . 28
3.3 Üstellerin çarpımı (PoE) formülü . . . 28
3.3.1 Türetme . . . 28

4 Jacobian, Tekillikler ve Statik 33
4.1 Uzay Jacobian'ı . . . 33
4.1.1 PoE'den türetme . . . 33
4.1.2 Ayak ucunun doğrusal hızı . . . 33
4.2 Tekillikler . . . 34
4.3 Statik: $\boldsymbol{\tau} = \mathbf{J}^T\mathbf{F}$ . . . 34
4.3.1 Sanal iş ile türetme . . . 34
4.3.2 Kuvvet elipsi hız elipsinin tersidir . . . 35

5 Ters Kinematik 39
5.1 Analitik IK: 3-DoF bacak . . . 39
5.1.1 Adım 1: roll . . . 39
5.1.2 Adım 2: sagital 2 uzuv, kosinüs teoremi . . . 39
5.2 Sayısal IK . . . 40
5.2.1 Newton yöntemi: Taylor açılımından . . . 40

5.2.2 Sönümlü en küçük kareler (Levenberg–Marquardt) 40

6 Rijit Cisim Dinamiği: Lagrange ve RNEA 46

6.1 Atalet 46

6.1.1 Rijit cismin kinetik enerjisi 46

6.2 Lagrange yöntemi 47

6.2.1 Euler–Lagrange denklemi 47

6.2.2 Genel manipülatör denklemine geçiş 47

6.2.3 2 uzuvlu bacak: tam türetme 47

6.2.4 Yapısal özellik: $\dot{\mathbf{M}} - 2\mathbf{C}$ ters simetrik 48

6.3 RNEA: özyinelemeli Newton–Euler 49

6.3.1 Tek bir uzvun denklemleri 49

6.3.2 İki geçiş 49

6.4 İleri dinamik ve sayısal integrasyon 49

6.4.1 Üç integratör ve harmonik osilatörde ne yaptıkları 49

7 Yüzen Taban, Kütle Merkezi ve Temas Kısıtları 55

7.1 Genelleştirilmiş koordinatlar ve seçim matrisi 55

7.1.1 Koordinatlar 55

7.1.2 Hareket denklemi ve seçim matrisi 55

7.2 Kütle matrisini Jacobian'larla kurmak 56

7.2.1 Türetme 56

7.3 Temas kısıtları ve KKT sistemi 56

7.3.1 Kısıttan ivme denklemine 56

7.3.2 Zemin sadece iter 57

7.3.3 Statik denge: basınç merkezi = kütle merkezi 57

8 Bitirme Projesi P1: SB-1 Bacak Kütüphanesi 63

8.1 Depo yapısı 63

8.2 Neden test? 63

8.3 CMake ile derleme 65

8.4 P1 kontrol listesi 66

Bu Cilt Hakkında

Cilt 1, serinin geri kalanının dilini kuruyor: bir robotun *nerede* olduğunu (kinematik), *nasıl* hareket ettiğini (Jacobian) ve *hangi kuvvetle* hareket ettiğini (dinamik) hesaplamak. Her bölüm aynı robot üzerinde ilerliyor: seri boyunca kullanacağın kendi biped modeli **SB-1**'in bacağı (uyluk ve baldır 0.2 m).

Bölüm	Konu	Birlikte oku (Modern Robotics / diğer)
1	Lineer cebir: SVD, null uzayı, pseudo-inverse, en küçük kareler	MR Ek A; Bölüm 6.2 (DLS)
2	Dönmeler: SO(3), Euler, kuaterniyon, integrasyon	MR 3.1–3.2
3	SE(3), twist, PoE ileri kinematiği	MR 3.3, 4
4	Jacobian, tekillikler, statik	MR 5
5	Ters kinematik (analitik ve sayısal)	MR 6
6	Dinamik: atalet, Lagrange, RNEA, integratörler	MR 8; Featherstone 5
7	Yüzen taban, kütle merkezi, temas kısıtları	ETH Robot Dynamics (floating base)
8	Bitirme projesi P1: test edilmiş kütüphane	–

Kod düzeni. Her bölümde önce Python, sonra aynı işi yapan C++ (Eigen) kodu ve ikisinin gerçek çıktısı var. Bölüm 2'den itibaren iki dilde ortak bir küçük kütüphane kullanılıyor: `robkit.py` ve `robkit.hpp`. Kod arşivinde `code/` (Python), `cpp/` (C++) ve `proje/` (P1 iskeleti, CMake ile) klasörleri var. Python dosyaları `code/` klasöründe `PYTHONPATH =lib python3 c02_rot.py` ile; C++ dosyaları `cpp/` klasöründe `g++ -std =c++17 -O2 -I /usr/include/eigen3 c02_rot.cpp -o c02_rot && ./c02_rot` ile çalışır.

Bu ciltte çalıştırılmayanlar. MuJoCo, Pinocchio ve ROS 2 bu ortamda kurulamadı; onlara atıf yapan alıştırmalar kendi bilgisayarında yapılmak üzere. Diğer her sayı, aşağıdaki kodların gerçek çıktısıdır.

Bölüm 1

Robotikçi Gözüyle Lineer Cebir

Robotikte matrisler bir "sayı tablosu" değil, bir **dönüşümdür**: eklem hızlarını ayak hızına (Jacobian), kuvveti torka ($\mathbf{J}^T$), bir koordinat sistemini diğerine (dönme) götürür. Bu bölümde bütün seri boyunca kullanacağımız dört aracı, bir bacak örneği üzerinden *türeterek* kuruyoruz: **rank**, **SVD**, **null uzayı** ve **en küçük kareler**. Her formülün nereden geldiğini adım adım gösteriyorum; kâğıt kalemle takip edebilmen için her bölümde elle çözülmüş küçük bir örnek var. Modern Robotics karşılığı: Ek A ve Bölüm 5–6; lineer cebir için Gilbert Strang'in *Introduction to Linear Algebra* kitabı ve MIT 18.06 ders videoları (telefonda izlenebilir) en iyi eşlikçi.

1.1 Örnek sistem: düzlemsel bacak ve Jacobian'ın türetilmesi

1.1.1 İleri kinematik

Uyluk $\ell_1 = 0.20$, baldır $\ell_2 = 0.20$, ayak $\ell_3 = 0.08$ m; eklem açıları $\mathbf{q} = (q_1, q_2, q_3)$ (kalça, diz, ayak bileği). Bu bölümde açılar *dikey-aşağı* yönden ölçülüyor ve pozitif açı ayağı $+x$'e (öne) götürüyor. Her uzvun dikeye göre *mutlak* açısı, kendisinden önceki tüm eklem açılarının toplamıdır, çünkü kalça dönünce uyluğun üstüne bağlı her şey de döner:

$$a_1 = q_1, \qquad a_2 = q_1 + q_2, \qquad a_3 = q_1 + q_2 + q_3.$$

Bir uzvun ucu, başlangıcına göre $(\ell_k \sin a_k,\ -\ell_k \cos a_k)$ kadar ötededir (açı dikeyden ölçüldüğü için x bileşeni sin, z bileşeni $-\cos$; $a_k = 0$'da uzuv dümdüz aşağı bakar ve uç $(0, -\ell_k)$'dadır). Uçları uç uca ekleyince ayak ucu:

$$x(\mathbf{q}) = \sum_{k=1}^{3} \ell_k \sin a_k, \qquad z(\mathbf{q}) = -\sum_{k=1}^{3} \ell_k \cos a_k.$$

1.1.2 Jacobian: zincir kuralıyla türetme

Eklemler hareket ederken ayak ne hızla gider? x'in zamana göre türevini zincir kuralıyla alalım. $\frac{d}{dt}\sin a_k = \cos a_k\, \dot{a}_k$ ve $\dot{a}_k = \dot{q}_1 + \cdots + \dot{q}_k$ olduğundan

$$\dot{x} = \sum_{k=1}^{3} \ell_k \cos a_k\, (\dot{q}_1 + \cdots + \dot{q}_k).$$

Şimdi bu toplamı $\dot{q}_i$'lere göre yeniden grupla. $\dot{q}_i$, $k \geq i$ olan her terimde görünür (çünkü a_k, $k \geq i$ ise q_i'yi içerir):

$$\dot{x} = \sum_{i=1}^{3} \Big(\sum_{k \geq i} \ell_k \cos a_k \Big) \dot{q}_i, \qquad \dot{z} = \sum_{i=1}^{3} \Big(\sum_{k \geq i} \ell_k \sin a_k \Big) \dot{q}_i.$$

Bu, matris biçiminde $\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$ demektir; $\mathbf{J}$ 2×3'lük **Jacobian** matrisidir:

$$J_{1i} = \frac{\partial x}{\partial q_i} = \sum_{k \geq i} \ell_k \cos a_k, \qquad J_{2i} = \frac{\partial z}{\partial q_i} = \sum_{k \geq i} \ell_k \sin a_k.$$

Genel tanım: bir $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$ fonksiyonunun Jacobian'ı, (i, j) elemanı $\partial f_i / \partial q_j$ olan $m \times n$ matristir; *j. sütun, sadece j. eklem birim hızla dönerse ayağın hızıdır.* Bu sütun yorumu, Bölüm 4'teki 3-B Jacobian'ın da özüdür.

Elle: 2 uzuvlu bacak, $q_1 = 0$, $q_2 = 90^\circ$

Ayak eklemini atalım ($\ell_3 = 0$), $\ell_1 = \ell_2 = 0.2$. $a_1 = 0$, $a_2 = \pi/2$: $\cos a_1 = 1$, $\sin a_1 = 0$, $\cos a_2 = 0$, $\sin a_2 = 1$. Ayak: $x = 0.2 \cdot 0 + 0.2 \cdot 1 = 0.2$, $z = -0.2 \cdot 1 - 0.2 \cdot 0 = -0.2$ (uyluk dik aşağı, baldır öne yatay).

$$\mathbf{J} = \begin{bmatrix} \ell_1 \cos a_1 + \ell_2 \cos a_2 & \ell_2 \cos a_2 \\ \ell_1 \sin a_1 + \ell_2 \sin a_2 & \ell_2 \sin a_2 \end{bmatrix} = \begin{bmatrix} 0.2 & 0 \\ 0.2 & 0.2 \end{bmatrix}.$$

Sütun kontrolü: sadece diz dönerse ($\dot{\mathbf{q}} = (0, 1)$) ayak hızı $(0,\ 0.2)$, yani dikey yukarı: yatay duran baldırın ucu dizin etrafında dönerken gerçekten dikey hareket eder. Sadece kalça dönerse $(0.2,\ 0.2)$: ayak, kalçadan ayağa giden $(0.2, -0.2)$ vektörüne dik yönde hareket eder (dönmede hız, yarıçapa diktir). Bu iki fizik kontrolünü her yeni Jacobian'da yap.

1.2 Lineer dönüşüm, rank ve null uzayı

$\mathbf{J}$'yi "$\dot{\mathbf{q}}$'yu alıp $\dot{\mathbf{p}}$ veren makine" olarak düşün. İki soru sorulur:

- **Sütun uzayı (görüntü):** hangi ayak hızları elde edilebilir? $\mathbf{J}\dot{\mathbf{q}} = \sum_i \dot{q}_i \, \mathbf{J}_{:,i}$, yani sütunların tüm doğrusal bileşimleri. Bu uzayın boyutu **rank**'tır. Düzlemde rank 2 ise ayak her yöne hareket edebilir; rank 1 ise sadece bir doğru boyunca.
- **Null uzayı (çekirdek):** hangi eklem hızları ayağı hiç oynatmaz? $\mathbf{J}\mathbf{n} = \mathbf{0}$ olan $\mathbf{n}$'ler. Ayağı yerde tutarken gövdeyi veya bileği oynatmak tam olarak budur.

Rank–nullity teoremi: n eklemli bir sistemde $\text{rank} + \dim(\text{null}) = n$. Neden? Eklem uzayındaki her yön ya ayağı hareket ettirir (görüntüye katkı) ya da ettirmez (null); n boyut bu ikisi arasında paylaşılır. 3 eklem, rank 2 $\Rightarrow$ 1 boyutlu null uzayı: **fazlalık** (redundancy).

Determinant ve tekillik (2×2). Kare $\mathbf{J}$ için rank düşmesi $\det \mathbf{J} = 0$ demektir. 2 uzuvlu bacakta açıkça hesaplayalım ($c_i = \cos a_i$, $s_i = \sin a_i$):

$$\begin{aligned} \det \mathbf{J} &= (\ell_1 c_1 + \ell_2 c_2)\ell_2 s_2 - \ell_2 c_2 (\ell_1 s_1 + \ell_2 s_2) \\ &= \ell_1 \ell_2 (c_1 s_2 - s_1 c_2) = \ell_1 \ell_2 \sin(a_2 - a_1) = \ell_1 \ell_2 \sin q_2. \end{aligned}$$

(Son adımda $\sin(\alpha - \beta) = \sin\alpha \cos\beta - \cos\alpha \sin\beta$.) Sonuç: **determinant sadece diz açısına bağlı** ve $q_2 = 0$ (diz düz) veya $q_2 = \pi$ (bacak tamamen katlanmış) iken sıfır. Elle örnekte $\det \mathbf{J} = 0.2 \cdot 0.2 - 0 \cdot 0.2 = 0.04 = 0.2 \cdot 0.2 \cdot \sin 90^\circ$. Kalça açısı determinantı hiç etkilemez: kalça bütün bacağı rijit döndürür, bacağın "şeklini" değiştirmez.

1.3 SVD: türetme ve geometrik anlam

1.3.1 Simetrik matrisler ve özdeğerler

SVD'yi türetmek için tek bir gerçek gerekiyor: **reel simetrik bir matrisin** ($\mathbf{A} = \mathbf{A}^T$) özdeğerleri reeldir ve özvektörleri birbirine dik seçilebilir; yani $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$, $\mathbf{V}^T\mathbf{V} = \mathbf{I}$ (spektral teorem).

Diklik kısmının kısa ispatı: $\mathbf{A}\mathbf{v}_1 = \lambda_1\mathbf{v}_1$, $\mathbf{A}\mathbf{v}_2 = \lambda_2\mathbf{v}_2$, $\lambda_1 \neq \lambda_2$ ise

$$\lambda_1\mathbf{v}_2^T\mathbf{v}_1 = \mathbf{v}_2^T\mathbf{A}\mathbf{v}_1 = (\mathbf{A}^T\mathbf{v}_2)^T\mathbf{v}_1 = (\mathbf{A}\mathbf{v}_2)^T\mathbf{v}_1 = \lambda_2\mathbf{v}_2^T\mathbf{v}_1$$
$$\Rightarrow\ (\lambda_1 - \lambda_2)\mathbf{v}_2^T\mathbf{v}_1 = 0\ \Rightarrow\ \mathbf{v}_2^T\mathbf{v}_1 = 0.$$

1.3.2 SVD'nin inşası

Herhangi bir $m \times n$ matris $\mathbf{J}$ alalım. $\mathbf{J}^T\mathbf{J}$ $(n \times n)$ simetrik ve *pozitif yarı-tanımlı*dır: her $\mathbf{x}$ için $\mathbf{x}^T\mathbf{J}^T\mathbf{J}\mathbf{x} = \|\mathbf{J}\mathbf{x}\|^2 \geq 0$, dolayısıyla özdeğerleri ≥ 0. Onları büyükten küçüğe $\sigma_1^2 \geq \sigma_2^2 \geq \ldots \geq 0$ diye adlandır, birim özvektörleri $\mathbf{v}_i$ olsun. Şimdi

$$\mathbf{u}_i = \frac{\mathbf{J}\mathbf{v}_i}{\sigma_i} \qquad (\sigma_i > 0)$$

tanımla. Bunlar da birbirine dik birim vektörlerdir:

$$\mathbf{u}_i^T\mathbf{u}_j = \frac{\mathbf{v}_i^T\mathbf{J}^T\mathbf{J}\mathbf{v}_j}{\sigma_i\sigma_j} = \frac{\sigma_j^2\,\mathbf{v}_i^T\mathbf{v}_j}{\sigma_i\sigma_j} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}.$$

Tanımı $\mathbf{J}\mathbf{v}_i = \sigma_i\mathbf{u}_i$ diye yazıp tüm i'ler için yan yana dizersek $\mathbf{J}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$, ve $\mathbf{V}$ dik olduğu için $(\mathbf{V}^{-1} = \mathbf{V}^T)$:

$$\boxed{\mathbf{J} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T} \qquad \text{(tekil değer ayrışımı, SVD)}$$

σ_i: **tekil değerler**, $\mathbf{v}_i$: sağ tekil vektörler (eklem uzayında), $\mathbf{u}_i$: sol tekil vektörler (görev uzayında).

1.3.3 Geometrik anlam: hız elipsi

Birim büyüklükte bütün eklem hızlarını düşün: $\|\dot{\mathbf{q}}\| = 1$. Bunları $\mathbf{v}_i$ tabanında yaz: $\dot{\mathbf{q}} = \sum c_i\mathbf{v}_i$, $\sum c_i^2 = 1$. Ayak hızı $\dot{\mathbf{p}} = \mathbf{J}\dot{\mathbf{q}} = \sum c_i\sigma_i\mathbf{u}_i$. Ayak hızının $\mathbf{u}_i$ bileşenleri $y_i = c_i\sigma_i$ ise

$$\sum_i \frac{y_i^2}{\sigma_i^2} = \sum_i c_i^2 = 1:$$

bu, yarı eksenleri σ_i olan ve $\mathbf{u}_i$ yönlerine hizalı bir **elips** denklemidir (manipülabilite elipsi). Yorum: "$\mathbf{v}_i$ yönünde birim eklem hızı, $\mathbf{u}_i$ yönünde σ_i büyüklüğünde ayak hızı üretir."

- **Rank** = sıfır olmayan tekil değer sayısı.
- **Koşul sayısı** $\kappa = \sigma_{\max}/\sigma_{\min}$: elips ne kadar yassı. $\kappa \to \infty \Leftrightarrow$ **tekillik**: bir yönde ayak hareket edemez.
- **Null uzayı** = sıfır tekil değerlere karşılık gelen $\mathbf{v}_i$'ler (ve $n > m$ ise fazladan olan $\mathbf{v}$'ler). 3 eklem, 2 görev boyutu $\Rightarrow$ null uzayı $\mathbf{V}$'nin son sütunu.
- Kare $\mathbf{J}$ için $|\det\mathbf{J}| = \prod\sigma_i$: elipsin alanı (hacmi), birim dairenin alanıyla oranlı. Determinant sıfıra giderken elips bir doğruya ezilir.

Elle: 2 uzuvlu bacağın Jacobian'ının SVD'si

1) $\mathbf{J}^T\mathbf{J} = \begin{bmatrix} 0.2 & 0.2 \\ 0 & 0.2 \end{bmatrix}\begin{bmatrix} 0.2 & 0 \\ 0.2 & 0.2 \end{bmatrix} = \begin{bmatrix} 0.08 & 0.04 \\ 0.04 & 0.04 \end{bmatrix}$.
2) Özdeğerler: $\lambda^2 - \mathrm{iz}\,\lambda + \det = 0$, $\mathrm{iz} = 0.12$, $\det = 0.08 \cdot 0.04 - 0.04^2 = 0.0016$: $\lambda = 0.06 \pm \sqrt{0.0036 - 0.0016} = 0.06 \pm 0.04472 \Rightarrow \lambda_1 = 0.10472,\ \lambda_2 = 0.01528$.

3) Tekil değerler $\sigma_1 = \sqrt{0.10472} = 0.3236$, $\sigma_2 = 0.1236$. Kontrol: $\sigma_1\sigma_2 = 0.04 = \det \mathbf{J}$. Koşul sayısı $\kappa = 2.618$ (ilginç bir şekilde altın oranın karesi).
4) $\mathbf{v}_1$: $(0.08 - 0.10472)v_x + 0.04v_y = 0 \Rightarrow v_y = 0.618v_x$; normalize: $\mathbf{v}_1 = (0.8507,\ 0.5257)$. $\mathbf{v}_2$ ona dik: $(-0.5257,\ 0.8507)$.
5) $\mathbf{u}_1 = \mathbf{J}\mathbf{v}_1/\sigma_1 = (0.1701,\ 0.2753)/0.3236 = (0.5257,\ 0.8507)$.
Yorum: en kolay hareket yönü, kalça ve dizi aynı yönde (kabaca 0.85:0.53 oranında) döndürmek; ayak o zaman yaklaşık 58° yukarı-öne doğru gider. En zor yön ona dik. NumPy aynı sayıları (işaretleri ters olabilir; Bölüm 1 notu) veriyor.

1.4 Ters hız problemi: pseudo-inverse ve null uzayı

1.4.1 En küçük normlu çözümün türetilmesi

İstenen ayak hızı $\mathbf{v}$ için $\mathbf{J}\dot{\mathbf{q}} = \mathbf{v}$ denkleminin ($m < n$ ise) sonsuz çözümü var. Aralarından en "ucuzunu", yani eklem hızlarının kareleri toplamı en küçük olanı seçelim:

$$\min_{\dot{\mathbf{q}}} \ \tfrac{1}{2}\|\dot{\mathbf{q}}\|^2 \quad \text{öyle ki} \quad \mathbf{J}\dot{\mathbf{q}} = \mathbf{v}.$$

Lagrange çarpanları: $\mathcal{L} = \tfrac{1}{2}\dot{\mathbf{q}}^T\dot{\mathbf{q}} + \boldsymbol{\lambda}^T(\mathbf{v} - \mathbf{J}\dot{\mathbf{q}})$. Optimumda $\mathcal{L}$'nin her değişkene göre türevi sıfırdır:

$$\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} = \dot{\mathbf{q}} - \mathbf{J}^T\boldsymbol{\lambda} = \mathbf{0} \ \Rightarrow \ \dot{\mathbf{q}} = \mathbf{J}^T\boldsymbol{\lambda}.$$

Bunu kısıta koy: $\mathbf{J}\mathbf{J}^T\boldsymbol{\lambda} = \mathbf{v}$. $\mathbf{J}$ tam satır ranklıysa $\mathbf{J}\mathbf{J}^T$ $(m \times m)$ tersinirdir: $\boldsymbol{\lambda} = (\mathbf{J}\mathbf{J}^T)^{-1}\mathbf{v}$, dolayısıyla

$$\dot{\mathbf{q}} = \underbrace{\mathbf{J}^T(\mathbf{J}\mathbf{J}^T)^{-1}}_{\mathbf{J}^+}\,\mathbf{v}.$$

Geometrik sezgi: optimum $\dot{\mathbf{q}} = \mathbf{J}^T\boldsymbol{\lambda}$, $\mathbf{J}$'nin *satırlarının* bir bileşimi; yani null uzayına dik. Null uzayında bir bileşen olsaydı, onu çıkarmak ayak hızını değiştirmeden normu küçültürdü.

SVD biçimi: $\mathbf{J} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$ ise $\mathbf{J}^+ = \mathbf{V}\boldsymbol{\Sigma}^+\mathbf{U}^T$, burada $\boldsymbol{\Sigma}^+$ sıfır olmayan σ_i'leri $1/\sigma_i$ ile değiştirir. Tam satır ranklıda iki formül aynıdır (yerine koyarak doğrula: $\mathbf{J}\mathbf{J}^T = \mathbf{U}\boldsymbol{\Sigma}\boldsymbol{\Sigma}^T\mathbf{U}^T$). SVD biçimi tekil durumda da tanımlı ve tekilliğin neden tehlikeli olduğunu açıkça gösterir: $\sigma_{\min} \to 0$ iken $1/\sigma_{\min} \to \infty$.

1.4.2 Null uzayına izdüşüm

$\mathbf{N} = \mathbf{I} - \mathbf{J}^+\mathbf{J}$ matrisinin iki özelliği:

$$\mathbf{J}\mathbf{N} = \mathbf{J} - \mathbf{J}\mathbf{J}^T(\mathbf{J}\mathbf{J}^T)^{-1}\mathbf{J} = \mathbf{J} - \mathbf{J} = \mathbf{0}, \qquad \mathbf{N}^2 = \mathbf{N} \quad \text{(izdüşüm)}.$$

Yani herhangi bir $\mathbf{z}$ için $\mathbf{N}\mathbf{z}$ ayağı oynatmayan bir eklem hızıdır. Genel çözüm:

$$\dot{\mathbf{q}} = \mathbf{J}^+\mathbf{v} + (\mathbf{I} - \mathbf{J}^+\mathbf{J})\mathbf{z}.$$

İkinci terim ayak hızını değiştirmeden ikincil bir amaç (eklem limitinden uzak durmak, gövdeyi dik tutmak) ekler. Bu, Cilt 6'daki WBC'nin "görev önceliği" fikrinin en basit hâlidir.

Elle: tek satırlı Jacobian $\mathbf{J} = [1 \;\; 1]$

İki eklem, ayağın tek bir koordinatı $p = q_1 + q_2$. $\mathbf{J}\mathbf{J}^T = 2$, $\mathbf{J}^+ = \frac{1}{2}\begin{bmatrix}1\\1\end{bmatrix}$. $\dot{p} = 1$ istenirse $\dot{\mathbf{q}} = (0.5,\; 0.5)$: iki eklem işi eşit paylaşır (norm 0.707; $(1,0)$ çözümünün normu 1). $\mathbf{N} = \mathbf{I} - \frac{1}{2}\begin{bmatrix}1 & 1\\1 & 1\end{bmatrix} = \frac{1}{2}\begin{bmatrix}1 & -1\\-1 & 1\end{bmatrix}$; $\mathbf{z} = (1,0)$ için $\mathbf{N}\mathbf{z} = (0.5,\; -0.5)$: biri ileri biri geri, toplam sıfır, p değişmez.

1.4.3 Tekillik ve sönümlü en küçük kareler (DLS)

Kısıtı katı tutmak yerine hatayı cezalandıralım ve eklem hızına da küçük bir ceza ekleyelim:

$$\min_{\dot{\mathbf{q}}} \; \|\mathbf{J}\dot{\mathbf{q}} - \mathbf{v}\|^2 + \lambda^2 \|\dot{\mathbf{q}}\|^2.$$

Gradyanı sıfırla: $2\mathbf{J}^T(\mathbf{J}\dot{\mathbf{q}} - \mathbf{v}) + 2\lambda^2\dot{\mathbf{q}} = 0 \Rightarrow (\mathbf{J}^T\mathbf{J} + \lambda^2\mathbf{I})\dot{\mathbf{q}} = \mathbf{J}^T\mathbf{v}$. $\lambda > 0$ iken bu matris her zaman tersinirdir. Daha küçük bir matrisle yazmak için $\mathbf{J}^T(\mathbf{J}\mathbf{J}^T + \lambda^2\mathbf{I}) = (\mathbf{J}^T\mathbf{J} + \lambda^2\mathbf{I})\mathbf{J}^T$ eşitliğini kullan (iki tarafı aç: ikisi de $\mathbf{J}^T\mathbf{J}\mathbf{J}^T + \lambda^2\mathbf{J}^T$):

$$\dot{\mathbf{q}} = \mathbf{J}^T(\mathbf{J}\mathbf{J}^T + \lambda^2\mathbf{I})^{-1}\mathbf{v}.$$

SVD ile ne yaptığını gör: her tekil yönde kazanç $1/\sigma_i$ yerine

$$g(\sigma) = \frac{\sigma}{\sigma^2 + \lambda^2}.$$

$\sigma \gg \lambda$ iken $g \approx 1/\sigma$ (pinv ile aynı); $\sigma \ll \lambda$ iken $g \approx \sigma/\lambda^2 \to 0$ (o yöne gitmekten vazgeçer). En büyük değeri: $g'(\sigma) = \frac{\lambda^2 - \sigma^2}{(\sigma^2+\lambda^2)^2} = 0 \Rightarrow \sigma = \lambda$, $g_{\max} = \frac{1}{2\lambda}$. Yani DLS eklem hızlarını *hiçbir zaman* $\|\mathbf{v}\|/(2\lambda)$'nın üstüne çıkarmaz. $\lambda = 0.01$ için $\sigma = 0.01$'de pinv kazancı 100 iken DLS 50; $\sigma = 0.001$'de pinv 1000, DLS 9.9.

Sayılar: bacak Jacobian'ı, null uzayı ve tekillik

$\mathbf{q} = (0.3, -0.6, 0.4)$ rad'da $\mathbf{J} = \begin{bmatrix}0.4617 & 0.2707 & 0.0796\\0.0080 & -0.0511 & 0.0080\end{bmatrix}$; tekil değerler 0.5414 ve 0.0493, koşul sayısı $\kappa = 10.99$. Null uzayı $\mathbf{n} = (-0.2335,\; 0.1144,\; 0.9656)$, $\mathbf{J}\mathbf{n} \approx 10^{-18}$. Ayak hızı $\mathbf{v} = (0.1, 0)$ m/s için pinv çözümü $\dot{\mathbf{q}} = (0.1884,\; 0.0359,\; 0.0413)$, $\|\dot{\mathbf{q}}\| = 0.196$; buna $0.5\mathbf{n}$ eklersek $\|\dot{\mathbf{q}}\| = 0.537$ ama ayak hızı *aynı* $(0.1, 0)$: ayak bileği büyük ölçüde dönüyor, ayak ucu yerinde.

Tekillik (2 uzuvlu bacak, $q_1 = 0$, ayak kalçaya doğru 0.1 m/s, yani bacağı kısalt):

q_2 (diz)	$\sigma_{\min}$	$\|\dot{\mathbf{q}}\|$ pinv	$\|\dot{\mathbf{q}}\|$ DLS ($\lambda = 0.01$)	DLS hız hatası
-1.00	0.0852	0.97	0.96	0.001
-0.30	0.0267	3.68	3.23	0.012
-0.10	0.0089	11.17	4.96	0.056
-0.01	0.0009	**111.8**	0.89	0.099
0	0	0 (hata 0.1)	0	0.100

Diz neredeyse düzken ayağı yukarı çekmek için pinv saniyede 112 rad'lık (!) eklem hızı istiyor: gerçek motorda imkânsız, simülasyonda patlama. DLS hızı sınırlı tutuyor ve karşılığında hedef hızın bir kısmından vazgeçiyor. Elle kontrol: $\sigma_{\min} \approx \ell_1\ell_2|\sin q_2|/\sigma_{\max}$ (çünkü $\sigma_1\sigma_2 = |\det \mathbf{J}|$); $q_2 = -0.01$'de $\det = 0.04 \cdot 0.01 = 4 \cdot 10^{-4}$, $\sigma_{\max} \approx 0.45$ ise $\sigma_{\min} \approx 0.0009$ ve pinv hızı

$\approx 0.1/0.0009 \approx 111$. Bipedlerin dizi neden hiç tam düz tutulmaz sorusunun bir cevabı bu tablodur.

1.5 En küçük kareler ve koşulluluk

1.5.1 Normal denklemlerin iki türetmesi

Ölçüm sayısı bilinmeyenden fazlaysa ($\mathbf{A}$ uzun, $m > n$) $\mathbf{Ax} = \mathbf{b}$'nin tam çözümü genelde yoktur; artığın karesini en aza indiririz: $f(\mathbf{x}) = \|\mathbf{Ax} - \mathbf{b}\|^2$.

(1) Türevle: $f = (\mathbf{Ax}-\mathbf{b})^T(\mathbf{Ax}-\mathbf{b}) = \mathbf{x}^T\mathbf{A}^T\mathbf{Ax} - 2\mathbf{b}^T\mathbf{Ax} + \mathbf{b}^T\mathbf{b}$. Kuadratik form türevleri: $\nabla(\mathbf{x}^T\mathbf{Qx}) = 2\mathbf{Qx}$ ($\mathbf{Q}$ simetrik), $\nabla(\mathbf{c}^T\mathbf{x}) = \mathbf{c}$. O halde $\nabla f = 2\mathbf{A}^T\mathbf{Ax} - 2\mathbf{A}^T\mathbf{b} = 0$:

$$\boxed{\mathbf{A}^T\mathbf{A}\,\mathbf{x} = \mathbf{A}^T\mathbf{b}} \qquad \text{(normal denklemler)}$$

(2) Geometriyle: $\mathbf{Ax}$, $\mathbf{A}$'nın sütun uzayında bir nokta. $\mathbf{b}$'ye en yakın nokta, $\mathbf{b}$'nin bu uzaya *dik izdüşümüdür*: artık $\mathbf{r} = \mathbf{b} - \mathbf{Ax}$ her sütuna dik olmalı, yani $\mathbf{A}^T\mathbf{r} = \mathbf{0}$. Bu aynı denklemdir. "Normal" adı da buradan: artık, sütun uzayına normaldir (diktir).

Elle: üç noktaya doğru uydurmak

Noktalar $(0,1)$, $(1,2)$, $(2,4)$; model $y = ax + b$. $\mathbf{A} = \begin{bmatrix}0 & 1\\ 1 & 1\\ 2 & 1\end{bmatrix}$, $\mathbf{b} = (1,2,4)$. $\mathbf{A}^T\mathbf{A} = \begin{bmatrix}\sum x^2 & \sum x\\ \sum x & n\end{bmatrix} = \begin{bmatrix}5 & 3\\ 3 & 3\end{bmatrix}$, $\mathbf{A}^T\mathbf{b} = \begin{bmatrix}\sum xy\\ \sum y\end{bmatrix} = \begin{bmatrix}10\\ 7\end{bmatrix}$. Cramer: $\det = 6$, $a = (10\cdot 3 - 3\cdot 7)/6 = 1.5$, $b = (5\cdot 7 - 3\cdot 10)/6 = 0.8333$. Artıklar $\mathbf{r} = (1-0.833,\ 2-2.333,\ 4-3.833) = (0.167,\ -0.333,\ 0.167)$. Diklik kontrolü: $\sum r_i = 0$ ve $\sum x_i r_i = 0 - 0.333 + 0.333 = 0$. Artık hem "$x$ sütununa" hem "1 sütununa" dik: türetmenin dediği gibi.

1.5.2 Neden `solve`, neden QR, neden asla `inv`?

Koşul sayısı ve hata. $\mathbf{Ax} = \mathbf{b}$'de $\mathbf{b}$ küçük bir $\delta\mathbf{b}$ kadar bozulursa çözüm $\delta\mathbf{x} = \mathbf{A}^{-1}\delta\mathbf{b}$ kadar değişir. Göreli olarak:

$$\frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} \le \|\mathbf{A}^{-1}\|\,\|\mathbf{A}\|\frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|} = \kappa(\mathbf{A})\frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|}$$

(çünkü $\|\delta\mathbf{x}\| \le \|\mathbf{A}^{-1}\|\|\delta\mathbf{b}\|$ ve $\|\mathbf{b}\| \le \|\mathbf{A}\|\|\mathbf{x}\|$; 2-normda $\|\mathbf{A}\| = \sigma_{\max}$, $\|\mathbf{A}^{-1}\| = 1/\sigma_{\min}$). Bilgisayardaki `double` sayıların göreli yuvarlama hatası $\epsilon \approx 1.1\cdot 10^{-16}$: dolayısıyla **çözümde beklenen göreli hata** $\approx \kappa \cdot 10^{-16}$.

Normal denklemler hassasiyet kaybeder: $\mathbf{A} = \mathbf{U\Sigma V}^T$ ise $\mathbf{A}^T\mathbf{A} = \mathbf{V\Sigma}^2\mathbf{V}^T$, yani tekil değerlerin kareleri: $\kappa(\mathbf{A}^T\mathbf{A}) = \kappa(\mathbf{A})^2$. $\kappa(\mathbf{A}) = 10^6$ ise QR ile 10^{-10}, normal denklemlerle 10^{-4} hata. **QR yöntemi:** $\mathbf{A} = \mathbf{QR}$ ($\mathbf{Q}$ dik sütunlu, $\mathbf{R}$ üst üçgen); $\|\mathbf{Ax}-\mathbf{b}\| = \|\mathbf{Rx}-\mathbf{Q}^T\mathbf{b}\|$ (dik matris normu korur) $\Rightarrow \mathbf{Rx} = \mathbf{Q}^T\mathbf{b}$, geriye yerine koyma. $\mathbf{A}^T\mathbf{A}$ hiç oluşturulmaz.

`inv` neden kötü: $\mathbf{A}^{-1}$'i hesaplayıp $\mathbf{b}$ ile çarpmak, LU ayrışımıyla doğrudan çözmekten hem yaklaşık üç kat yavaş hem (aşağıdaki tabloda görüldüğü gibi) daha hatalıdır. Kural: kodda `inv(A) @ b` gördüğün her yeri `solve(A, b)` yap.

Sayılar: servo kalibrasyonu ve Hilbert matrisi

Bir MG996R'a 1000–2000 µs PWM verip açıyı ölçtüğümüzü düşünelim (11 ölçüm, sentetik ama tekrar üretilebilir gürültü: $0.8\sin(7.3i)$ derece; gerçek eğim 0.09 derece/µs, ofset 2 derece). En küçük kareler: eğim 0.08974, ofset 2.0887 derece, RMS artık 0.53 derece. $\kappa(\mathbf{A}) = 316$, $\kappa(\mathbf{A}^T\mathbf{A}) = 10^5 \approx 316^2$: türetmenin söylediği gibi kare. Bu küçük problemde fark etmez, ama büyük tanılama problemlerinde (Cilt 4, sistem tanılama) eder. **Hilbert matrisi** ($H_{ij} = 1/(i+j+1)$, çok kötü koşullu), gerçek çözüm $\mathbf{x} = 1$:

n	κ	numpy `solve`	numpy `inv`	Eigen LU	Eigen `inverse`
6	1.5×10^{7}	1.0×10^{-10}	2.4×10^{-9}	7.6×10^{-10}	1.7×10^{-9}
10	1.6×10^{13}	4.0×10^{-5}	7.7×10^{-3}	1.6×10^{-4}	2.5×10^{-2}
12	$\sim 10^{16}$	0.45	9.4	0.28	56

Kural doğrulandı: hata $\approx \kappa \times 10^{-16}$ ($n = 6$: $1.5 \cdot 10^{7} \cdot 10^{-16} \approx 10^{-9}$). $\kappa \approx 10^{16}$'da sonuç anlamsız; `inv` her durumda `solve`'dan kötü. Python ve C++'ın *tam* aynı hatayı vermemesi normal: farklı algoritma/sıralama, aynı mertebe.

Python

```python
import numpy as np
np.set_printoptions(precision=4, suppress=True)

# Düzlemsel 3 uzuvlu bacak (uyluk, baldır, ayak): eklem açıları q = [kalça, diz, ayak bileği]
# Açılar dikey-aşağı yönden ölçülür: ayak ucu  x = Σ L_k sin(a_k),  z = -Σ L_k cos(a_k),  a_k = q1+...+qk
L = np.array([0.20, 0.20, 0.08])                    # uzuv boyları [m]
def foot_jacobian(q):
    """Ayak ucunun (x, z) hızını eklem hızlarına bağlayan 2x3 Jacobian: v = J q̇"""
    a = np.cumsum(q)                                 # mutlak açılar: q1, q1+q2, q1+q2+q3
    J = np.zeros((2, 3))
    for i in range(3):                               # i. eklem, kendisinden sonraki tüm uzuvları döndürür
        J[0, i] = sum(L[k]*np.cos(a[k]) for k in range(i, 3))     # dx/dq_i
        J[1, i] = sum(L[k]*np.sin(a[k]) for k in range(i, 3))     # dz/dq_i  (açı dikeyden ölçülüyor)
    return J

q = np.array([0.3, -0.6, 0.4])
J = foot_jacobian(q)
print("J =\n", J)
U, s, Vt = np.linalg.svd(J)                         # J = U diag(s) V^T
print("tekil değerler:", s, " rank:", np.sum(s > 1e-10), " koşul sayısı:", round(s[0]/s[-1], 3))
n = Vt[-1]                                           # V'nin son satırı: null uzayı (2x3 matriste 1 boyutlu)
print("null uzayı vektörü n =", n, "  J n =", J@n, "  (sıfır olmalı)")

# Ters hız problemi: ayak ucu v = (0.1, 0) m/s ile hareket etsin
v = np.array([0.1, 0.0])
qd_pinv = np.linalg.pinv(J)@v                        # en küçük normlu çözüm
qd_null = qd_pinv + 0.5*n                            # null uzayından eklenen hareket
print("pinv çözüm  q̇ =", qd_pinv, " |q̇| =", round(np.linalg.norm(qd_pinv), 4), " J q̇ =", J@qd_pinv)
print("+0.5 n      q̇ =", qd_null, " |q̇| =", round(np.linalg.norm(qd_null), 4), " J q̇ =", J@qd_null, "  (aynı ayak hızı, farklı duruş)")

# Tekillik: diz düz (q2 = 0) iken 2 uzuvlu bacak, ayak eksen boyunca hareket edemez
def J2(q1, q2):
    return np.array([[L[0]*np.cos(q1) + L[1]*np.cos(q1 + q2), L[1]*np.cos(q1 + q2)],
```

```python
                    [L[0]*np.sin(q1) + L[1]*np.sin(q1 + q2), L[1]*np.sin(q1 + q2)]])
print("\n2 uzuvlu bacak, diz açısı -> tekil değerler ve pinv ile gereken eklem hızı (ayak bacak boyunca 0.1 m/s kısalsın)")
for q2 in [-1.0, -0.3, -0.1, -0.01, 0.0]:
    Jq = J2(0.0, q2); s2 = np.linalg.svd(Jq, compute_uv=False)
    vdir = np.array([0.0, 0.1])                                          # ayak kalçaya doğru (yukarı) 0.1 m/s: bacağı kısalt
    qd_p = np.linalg.pinv(Jq, rcond=1e-15)@vdir
    lam = 0.01; qd_d = Jq.T@np.linalg.solve(Jq@Jq.T + lam**2*np.eye(2), vdir)      # sönümlü en küçük kareler (DLS)
    print(f"  q2 ={q2:6.2f}: σ = {s2.round(5)}, |q̇| pinv = {np.linalg.norm(qd_p):10.4f} (hız hatası {np.linalg.norm(Jq@qd_p - vdir):.4f}), |q̇| DLS = {np.linalg.norm(qd_d):7.4f}, DLS hız hatası = {np.linalg.norm(Jq@qd_d - vdir):.4f}")

# En küçük kareler: servo kalibrasyonu (PWM darbe genişliği [µs] -> ölçülen açı [derece])
pwm = np.arange(1000.0, 2001.0, 100.0)
noise = 0.8*np.sin(7.3*np.arange(len(pwm)))             # tekrar üretilebilir "gürültü" (C++ ile aynı veri)
angle = 0.09*(pwm - 1500.0) + 2.0 + noise               # gerçek: 0.09 derece/µs, 2 derece ofset
A = np.column_stack([pwm - 1500.0, np.ones_like(pwm)])
x, res, rank, sv = np.linalg.lstsq(A, angle, rcond=None)
print(f"\nservo kalibrasyonu: eğim = {x[0]:.5f} derece/µs, ofset = {x[1]:.4f} derece, RMS artık = {np.sqrt(res[0]/len(pwm)):.4f} derece")
x_ne = np.linalg.solve(A.T@A, A.T@angle)                # normal denklemler (aynı sonuç, daha kötü koşullu)
print(f"normal denklemlerle: {x_ne.round(5)};  cond(A) = {np.linalg.cond(A):.1f}, cond(AᵀA) = {np.linalg.cond(A.T@A):.1f} (karesi)")

# Kötü koşullu sistem: Hilbert matrisi
for n_ in [6, 10, 12]:
    H = np.array([[1.0/(i + j + 1) for j in range(n_)] for i in range(n_)]); xt = np.ones(n_); b = H@xt
    e_solve = np.linalg.norm(np.linalg.solve(H, b) - xt); e_inv = np.linalg.norm(np.linalg.inv(H)@b - xt)
    print(f"Hilbert n ={n_:2d}: cond = {np.linalg.cond(H):.2e}, hata solve = {e_solve:.1e}, hata inv = {e_inv:.1e}")
```

Çıktı:

```
J =
 [[ 0.4617  0.2707  0.0796]
 [ 0.008  -0.0511  0.008 ]]
tekil değerler: [0.5414 0.0493]  rank: 2  koşul sayısı: 10.985
null uzayı vektörü n = [-0.2335  0.1144  0.9656]   J n = [ 0. -0.]   (sıfır olmalı)
pinv çözüm  q̇ = [0.1884 0.0359 0.0413]  |q̇| = 0.1962  J q̇ = [ 0.1 -0. ]
+0.5 n      q̇ = [0.0717 0.0931 0.5241]  |q̇| = 0.5371  J q̇ = [0.1 0. ]  (aynı ayak hızı, farklı duruş)

2 uzuvlu bacak, diz açısı -> tekil değerler ve pinv ile gereken eklem hızı (ayak bacak boyunca 0.1 m/s kısalsın)
  q2 = -1.00: σ = [0.3949 0.0852], |q̇| pinv =     0.9699 (hız hatası 0.0000), |q̇| DLS =  0.9570, DLS hız hatası = 0.0011
  q2 = -0.30: σ = [0.4424 0.0267], |q̇| pinv =     3.6820 (hız hatası 0.0000), |q̇| DLS =  3.2297, DLS hız hatası = 0.0121
  q2 = -0.10: σ = [0.4467 0.0089], |q̇| pinv =    11.1654 (hız hatası 0.0000), |q̇| DLS =  4.9599, DLS hız hatası = 0.0555
  q2 = -0.01: σ = [0.4472 0.0009], |q̇| pinv =   111.8019 (hız hatası 0.0000), |q̇| DLS =  0.8873, DLS hız hatası = 0.0992
  q2 =  0.00: σ = [0.4472 0.    ], |q̇| pinv =     0.0000 (hız hatası 0.1000), |q̇| DLS =  0.0000, DLS hız hatası = 0.1000

servo kalibrasyonu: eğim = 0.08974 derece/µs, ofset = 2.0887 derece, RMS artık = 0.5297 derece
normal denklemlerle: [0.0897 2.0887];  cond(A) = 316.2, cond(AᵀA) = 100000.0 (karesi)
Hilbert n = 6: cond = 1.50e+07, hata solve = 1.0e-10, hata inv = 2.4e-09
Hilbert n =10: cond = 1.60e+13, hata solve = 4.0e-05, hata inv = 7.7e-03
Hilbert n =12: cond = 1.62e+16, hata solve = 4.5e-01, hata inv = 9.4e+00
```

C++ (Eigen)

```cpp
// c01_linalg.cpp — Python sürümüyle aynı hesaplar, Eigen ile.
// Derleme: g++ -std=c++17 -O2 -I /usr/include/eigen3 c01_linalg.cpp -o c01_linalg
#include <Eigen/Dense>
#include <cmath>
#include <cstdio>
#include <iostream>
using Eigen::MatrixXd; using Eigen::VectorXd; using Eigen::Vector2d; using Eigen::Vector3d;

const double L[3] = {0.20, 0.20, 0.08};

Eigen::Matrix<double, 2, 3> foot_jacobian(const Vector3d& q) {
    double a[3] = {q(0), q(0) + q(1), q(0) + q(1) + q(2)};        // mutlak açılar
    Eigen::Matrix<double, 2, 3> J = Eigen::Matrix<double, 2, 3>::Zero();
    for (int i = 0; i < 3; ++i)
        for (int k = i; k < 3; ++k) {                              // i. eklem k>=i uzuvlarını döndürür
            J(0, i) += L[k] * std::cos(a[k]);
            J(1, i) += L[k] * std::sin(a[k]);
        }
    return J;
}

int main() {
    Eigen::IOFormat F(4, 0, ", ", "\n", "[", "]");
    Vector3d q(0.3, -0.6, 0.4);
    auto J = foot_jacobian(q);
    std::cout << "J =\n" << J.format(F) << "\n";
    Eigen::JacobiSVD<MatrixXd> svd(J, Eigen::ComputeFullU | Eigen::ComputeFullV);
    VectorXd s = svd.singularValues();
    std::printf("tekil değerler: [%.4f %.4f]  koşul sayısı: %.3f\n", s(0), s(1), s(0) / s(1));
    VectorXd n = svd.matrixV().col(2);                             // null uzayı (işareti Python'dan farklı olabilir)
    std::printf("null uzayı n = [%.4f %.4f %.4f], |J n| = %.1e\n", n(0), n(1), n(2), (J * n).norm());

    Vector2d v(0.1, 0.0);
    MatrixXd Jp = J.completeOrthogonalDecomposition().pseudoInverse();
    VectorXd qd = Jp * v;
    std::printf("pinv çözüm q̇ = [%.4f %.4f %.4f], |q̇| = %.4f\n", qd(0), qd(1), qd(2), qd.norm());

    std::printf("\n2 uzuvlu bacak: tekillik yakınında pinv ve DLS (ayak yukarı 0.1 m/s)\n");
    for (double q2 : {-1.0, -0.3, -0.1, -0.01}) {
        Eigen::Matrix2d J2;
        J2 << L[0] + L[1] * std::cos(q2), L[1] * std::cos(q2),
              L[1] * std::sin(q2),        L[1] * std::sin(q2);     // q1 = 0
        Vector2d vd(0.0, 0.1);
        Vector2d qp = J2.inverse() * vd;                           // kare ve tekil değil: pinv = ters
        double lam = 0.01;
        Vector2d qdls = J2.transpose() * (J2 * J2.transpose() + lam * lam * Eigen::Matrix2d::Identity()).ldlt().solve(vd);
        std::printf("  q2 =%6.2f: |q̇| pinv = %10.4f, |q̇| DLS = %7.4f, DLS hız hatası = %.4f\n",
                    q2, qp.norm(), qdls.norm(), (J2 * qdls - vd).norm());
    }

    // En küçük kareler: servo kalibrasyonu
    const int N = 11; MatrixXd A(N, 2); VectorXd y(N);
    for (int i = 0; i < N; ++i) {
        double pwm = 1000.0 + 100.0 * i;
        A(i, 0) = pwm - 1500.0; A(i, 1) = 1.0;
        y(i) = 0.09 * (pwm - 1500.0) + 2.0 + 0.8 * std::sin(7.3 * i);
```

```
    }
    VectorXd x = A.colPivHouseholderQr().solve(y);                    // QR: lstsq'ya denk
    double rms = std::sqrt((A * x - y).squaredNorm() / N);
    std::printf("\nservo kalibrasyonu: eğim = %.5f derece/µs, ofset = %.4f derece, RMS artık = %.4f derece\n", x(0), x(1), rms);

    for (int n_ : {6, 10, 12}) {
        MatrixXd H(n_, n_);
        for (int i = 0; i < n_; ++i) for (int j = 0; j < n_; ++j) H(i, j) = 1.0 / (i + j + 1);
        VectorXd xt = VectorXd::Ones(n_), b = H * xt;
        double e_lu = (H.partialPivLu().solve(b) - xt).norm();
        double e_inv = (H.inverse() * b - xt).norm();
        Eigen::JacobiSVD<MatrixXd> sv(H); double c = sv.singularValues()(0) / sv.singularValues()(n_ - 1);
        std::printf("Hilbert n =%2d: cond = %.2e, hata LU = %.1e, hata inverse = %.1e\n", n_, c, e_lu, e_inv);
    }
    return 0;
}
```

Çıktı:

```
J =
[  0.4617,   0.2707,   0.0796]
[0.007987, -0.05112, 0.007987]
tekil değerler: [0.5414 0.0493]  koşul sayısı: 10.985
null uzayı n = [-0.2335 0.1144 0.9656], |J n| = 1.7e-18
pinv çözüm q̇ = [0.1884 0.0359 0.0413], |q̇| = 0.1962

2 uzuvlu bacak: tekillik yakınında pinv ve DLS (ayak yukarı 0.1 m/s)
  q2 = -1.00: |q̇| pinv =     0.9699, |q̇| DLS =  0.9570, DLS hız hatası = 0.0011
  q2 = -0.30: |q̇| pinv =     3.6820, |q̇| DLS =  3.2297, DLS hız hatası = 0.0121
  q2 = -0.10: |q̇| pinv =    11.1654, |q̇| DLS =  4.9599, DLS hız hatası = 0.0555
  q2 = -0.01: |q̇| pinv =   111.8019, |q̇| DLS =  0.8873, DLS hız hatası = 0.0992

servo kalibrasyonu: eğim = 0.08974 derece/µs, ofset = 2.0887 derece, RMS artık = 0.5297 derece
Hilbert n = 6: cond = 1.50e+07, hata LU = 7.6e-10, hata inverse = 1.7e-09
Hilbert n =10: cond = 1.60e+13, hata LU = 1.6e-04, hata inverse = 2.5e-02
Hilbert n =12: cond = 1.74e+16, hata LU = 2.8e-01, hata inverse = 5.6e+01
```

Python ve C++ notu: numpy ile Eigen

- **Boyut:** Eigen'de sabit boyut (`Eigen::Matrix<double,2,3>`, `Vector3d`) derleme anında bilinir, yığında (stack) durur, hızlıdır; dinamik boyut (`MatrixXd`) numpy dizisine benzer. Küçük robot matrislerinde (3×3, 6×6) sabit boyut kullan.
- **İndeks:** numpy `J[0, i]`, Eigen `J(0, i)`. İkisi de 0'dan başlar. Eigen varsayılan olarak *sütun öncelikli* (column-major), numpy satır öncelikli: döngü sırası performansı etkiler, sonucu etkilemez.
- **SVD'de işaret:** tekil vektörlerin işareti tanımsızdır ($\mathbf{v}$ ve $-\mathbf{v}$ ikisi de doğru; türetmede $\mathbf{v}_i$'yi özvektör olarak seçtik, $-\mathbf{v}_i$ de özvektördür). Testlerde işarete bağımlı karşılaştırma yapma.
- **`auto` tuzağı (Eigen):** `auto y = A*x;` bir *ifade şablonu* (henüz hesaplanmamış) saklar; `A` veya `x` sonradan değişirse `y` de değişir ya da geçersiz bellek okursun. Eigen ifadelerinde tipi açıkça yaz: `Eigen::VectorXd y = A*x;`.
- **Çözücüler:** numpy `solve` ↔ Eigen `partialPivLu().solve`; `lstsq` ↔ `colPivHouseholderQr().solve`; simetrik pozitif tanımlı ise `ldlt()` veya `llt()` (en

hızlı).

- **`printf` biçimleri:** `%.4f` (4 ondalık), `%.1e` (bilimsel), `%10.4f` (10 karakter genişlik): Python'un f-string'deki `{x:10.4f}` ile aynı dil.

Alıştırma

(Kâğıt) (a) 2 uzuvlu bacakta $q_1 = 0.3$, $q_2 = 1.0$ için $\mathbf{J}$'yi yaz, $\det \mathbf{J} = \ell_1 \ell_2 \sin q_2$'yi doğrula. **(b)** $\mathbf{J} = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$'nin tekil değerlerini elle bul ($\mathbf{J}^T\mathbf{J}$'nin özdeğerleri; cevap $\sqrt{45}$ ve $\sqrt{5}$). **(c)** $\mathbf{J} = [1 \;\; 2]$ için $\mathbf{J}^+$ ve $\mathbf{N}$'yi hesapla; $\mathbf{N}^2 = \mathbf{N}$'yi doğrula. **(d)** $g(\sigma) = \sigma/(\sigma^2 + \lambda^2)$'yi $\lambda = 0.1$ için kabaca çiz. **(e)** $(1,2), (2,3), (3,5), (4,6)$ noktalarına doğru uydur ve artıkların x'e dik olduğunu kontrol et. **(Bilgisayar) (f)** 3 uzuvlu bacakta q_3'ü tara; κ'nın en kötü olduğu açıyı bul ve geometrik yorumla. **(g)** Null uzayı hareketini "ayak bileği açısını 0'a yaklaştır" amacıyla kullan: $\mathbf{z} = -k(\mathbf{q} - \mathbf{q}_{ref})$; 2 s integre et, ayak yörüngesi değişiyor mu? **(h)** DLS'de λ'yı $0.001,\ 0.01,\ 0.05$ yap; tablo nasıl değişiyor? **(i)** Elindeki MG996R ile gerçek kalibrasyon: telefonla açıyı fotoğraflayıp ölç; eğimi ve RMS artığı yukarıdakilerle karşılaştır.

Bölüm 2

Dönmeler: SO(3), Euler Açıları, Kuaterniyonlar

Bir bacaklı robotun gövde yönelimi, IMU'nun ölçtüğü şey, MuJoCo'nun `qpos[3:7]`'si, ayak tabanının yere göre açısı: hepsi *dönme*dir. Dönmelerle çalışırken yapılan hatalar (yanlış sıra, yanlış kuaterniyon konvansiyonu, gimbal lock, integrasyonda kayma) robotikteki en yaygın ve en sinsi hatalardır, çünkü kod çalışır ama yanlış sonuç verir. Bu bölümde her gösterimi sıfırdan türetiyoruz. Modern Robotics karşılığı: Bölüm 3.1–3.2.

2.1 Dönme matrisi ve SO(3)

2.1.1 Dönme matrisi nereden gelir?

Bir dönme, her vektörü başka bir vektöre götüren *doğrusal* bir işlemdir (iki vektörün toplamını döndürmek, ayrı ayrı döndürüp toplamakla aynıdır). Doğrusal her işlem bir matrisle yazılır ve **matrisin sütunları, taban vektörlerinin görüntüleridir**: $\mathbf{R}\mathbf{e}_1$ ilk sütundur. Örnek: z ekseni etrafında θ dönmesi $\mathbf{e}_1 = (1,0,0)$'ı $(\cos\theta, \sin\theta, 0)$'a, $\mathbf{e}_2 = (0,1,0)$'ı $(-\sin\theta, \cos\theta, 0)$'a götürür, $\mathbf{e}_3$'e dokunmaz:

$$\mathbf{R}_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}, \quad \mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}.$$

($\mathbf{R}_y$'deki işaretlerin farklı görünmesinin sebebi: sağ el kuralında y etrafında dönme z'yi x'e götürür, yani $z \to x \to -z$ sırası.) Robotik yorum: $\mathbf{R}_{sb}$'nin sütunları, gövde çerçevesi $\{b\}$'nin x, y, z eksenlerinin dünya çerçevesi $\{s\}$'deki ifadeleridir.

2.1.2 Neden $\mathbf{R}^T\mathbf{R} = \mathbf{I}$ ve $\det\mathbf{R} = +1$?

Dönme uzunlukları ve açıları korur, yani her $\mathbf{x}, \mathbf{y}$ için iç çarpımı korur:

$$(\mathbf{R}\mathbf{x})^T(\mathbf{R}\mathbf{y}) = \mathbf{x}^T\mathbf{R}^T\mathbf{R}\mathbf{y} = \mathbf{x}^T\mathbf{y} \quad \forall \mathbf{x}, \mathbf{y} \implies \mathbf{R}^T\mathbf{R} = \mathbf{I}.$$

($\mathbf{x} = \mathbf{e}_i$, $\mathbf{y} = \mathbf{e}_j$ seçince $\mathbf{R}^T\mathbf{R}$'nin (i,j) elemanının δ_{ij} olduğu çıkar.) Yani sütunlar birbirine dik birim vektörlerdir ve $\mathbf{R}^{-1} = \mathbf{R}^T$: tersini almak bedava. Buradan $\det(\mathbf{R}^T\mathbf{R}) = (\det\mathbf{R})^2 = 1$, yani $\det\mathbf{R} = \pm 1$. -1 olanlar ayna yansımalarıdır (sağ eli sol ele çevirir); fiziksel dönmeler sağ elli çerçeveyi korur: $\det\mathbf{R} = +1$. Bu iki koşulu sağlayan matrislerin kümesi **SO(3)** (Special Orthogonal group).

SO(3) bir gruptur ama vektör uzayı değildir: iki dönmenin çarpımı dönmedir ($(\mathbf{R}_1\mathbf{R}_2)^T\mathbf{R}_1\mathbf{R}_2 = \mathbf{R}_2^T\mathbf{I}\mathbf{R}_2 = \mathbf{I}$), ama toplamı değildir ($\mathbf{I} + \mathbf{I} = 2\mathbf{I}$, determinantı 8). Bu son cümle, aşağıdaki integrasyon hatasının sebebi.

2.1.3 Çerçeve notasyonu ve bileşke

$\mathbf{R}_{sb}$ = "$\{b\}$'nin $\{s\}$'deki yönelimi". Gövdede ifade edilmiş bir vektör $\mathbf{p}_b$, dünyada $\mathbf{p}_s = \mathbf{R}_{sb}\mathbf{p}_b$ (sütunların doğrusal bileşimi: $p_{b,x}$ kere gövde x ekseni + …). Zincir: $\mathbf{R}_{sc} = \mathbf{R}_{sb}\mathbf{R}_{bc}$ (iç indisler "sadeleşir"); ters: $\mathbf{R}_{bs} = \mathbf{R}_{sb}^T$. Bu alışkanlık, seri boyunca çerçeve karışıklığını önler.

Sıra önemli: $\mathbf{R}_{sb}\mathbf{R}_{bc}$ ile $\mathbf{R}_{bc}\mathbf{R}_{sb}$ genelde farklıdır (dönmeler değişmeli değil). Telefonu elinde x etrafında 90° sonra z etrafında 90°, sonra ters sırayla çevir: farklı yerlere varırsın. Kural: *sağdan çarpım* gövde (yeni, hareketli) eksenlerine göre, *soldan çarpım* sabit dünya eksenlerine göre dönmedir. $\mathbf{R}\mathbf{R}_x(\alpha)$: "gövdenin kendi x ekseni etrafında" döndür; $\mathbf{R}_x(\alpha)\mathbf{R}$: "dünyanın x ekseni etrafında" döndür.

2.2 Euler/RPY açıları ve gimbal lock

2.2.1 ZYX matrisinin açılımı

Bu seride (ve ROS'ta yaygın olarak) **ZYX = yaw–pitch–roll**: $\mathbf{R} = \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi)$ (gövde eksenlerine göre sırayla yaw, pitch, roll). Çarpımı açalım ($c_\theta = \cos\theta$ vb.):

$$\mathbf{R} = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix}.$$

Açıları geri çıkarmak: 3. satır ve 1. sütun en sade elemanları içeriyor:

$$\theta = -\arcsin R_{31}, \qquad \phi = \operatorname{atan2}(R_{32},\ R_{33}), \qquad \psi = \operatorname{atan2}(R_{21},\ R_{11}).$$

(Çünkü $R_{32}/R_{33} = \tan\phi$ ve $R_{21}/R_{11} = \tan\psi$, ikisinde de pay ve payda c_θ ile çarpılmış.)

2.2.2 Gimbal lock'un türetilmesi

$c_\theta = 0$ olduğunda yukarıdaki iki `atan2`'nin argümanları $(0, 0)$ olur: tanımsız. $\theta = +90°$ ($s_\theta = 1$, $c_\theta = 0$) koyup matrisi trigonometrik toplam formülleriyle sadeleştirelim:

$$\mathbf{R} = \begin{bmatrix} 0 & \sin(\phi - \psi) & \cos(\phi - \psi) \\ 0 & \cos(\phi - \psi) & -\sin(\phi - \psi) \\ -1 & 0 & 0 \end{bmatrix}.$$

(Örnek: $R_{12} = c_\psi s_\phi - s_\psi c_\phi = \sin(\phi - \psi)$.) Matris **sadece** $\phi - \psi$ **farkına** bağlı: $(\phi, \psi) = (0.3, 0.5)$ ile $(0.1, 0.3)$ aynı dönmeyi verir. Bir serbestlik kayboldu: pitch 90° iken yaw ve roll eksenleri aynı fiziksel eksene düşer. Bu **gösterimin** tekilliğidir, fiziksel bir sorun değildir: robot o yönelimde rahatça durur, sadece onu üç Euler açısıyla tek türlü yazamazsın. Kural: Euler açılarını *insanlara göstermek* için kullan; hesapta dönme matrisi veya kuaterniyon.

2.3 Eksen-açı ve üstel harita

2.3.1 Çapraz çarpım matrisi

$\boldsymbol{\omega} \times \mathbf{p}$ doğrusal bir işlemdir, matrisle yazılabilir. Bileşenleri açınca:

$$\boldsymbol{\omega} \times \mathbf{p} = \begin{bmatrix} \omega_2 p_3 - \omega_3 p_2 \\ \omega_3 p_1 - \omega_1 p_3 \\ \omega_1 p_2 - \omega_2 p_1 \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}}_{[\boldsymbol{\omega}]} \mathbf{p}.$$

$[\boldsymbol{\omega}]^T = -[\boldsymbol{\omega}]$ (ters simetrik, "skew"). İki kullanışlı özdeşlik: (i) birim $\hat{\boldsymbol{\omega}}$ için $[\hat{\boldsymbol{\omega}}]^2 = \hat{\boldsymbol{\omega}}\hat{\boldsymbol{\omega}}^T - \mathbf{I}$ ("bac-cab" kuralından: $\hat{\boldsymbol{\omega}} \times (\hat{\boldsymbol{\omega}} \times \mathbf{p}) = \hat{\boldsymbol{\omega}}(\hat{\boldsymbol{\omega}} \cdot \mathbf{p}) - \mathbf{p}$), dolayısıyla (ii) $[\hat{\boldsymbol{\omega}}]^3 = -[\hat{\boldsymbol{\omega}}]$.

2.3.2 Rodrigues formülünün geometrik türetmesi

Birim eksen $\hat{\boldsymbol{\omega}}$ etrafında θ kadar dönme. Bir $\mathbf{p}$ vektörünü eksene paralel ve dik iki parçaya ayır:

$$\mathbf{p}_{\parallel} = \hat{\boldsymbol{\omega}}(\hat{\boldsymbol{\omega}} \cdot \mathbf{p}), \qquad \mathbf{p}_{\perp} = \mathbf{p} - \mathbf{p}_{\parallel} = -[\hat{\boldsymbol{\omega}}]^2 \mathbf{p}.$$

Dönme paralel parçaya dokunmaz. Dik parça, eksene dik düzlemde θ kadar döner. O düzlemde iki dik yön var: $\mathbf{p}_{\perp}$ ve onu 90° çevirmiş olan $\hat{\boldsymbol{\omega}} \times \mathbf{p}_{\perp} = \hat{\boldsymbol{\omega}} \times \mathbf{p} = [\hat{\boldsymbol{\omega}}]\mathbf{p}$ (ikisi aynı uzunlukta). Düzlemde dönme: $\mathbf{p}_{\perp} \to \cos\theta\, \mathbf{p}_{\perp} + \sin\theta\, [\hat{\boldsymbol{\omega}}]\mathbf{p}$. Toplarsak:

$$\mathbf{R}\mathbf{p} = \mathbf{p}_{\parallel} + \cos\theta\, \mathbf{p}_{\perp} + \sin\theta [\hat{\boldsymbol{\omega}}]\mathbf{p} = \mathbf{p} + \mathbf{p}_{\perp}(\cos\theta - 1) + \sin\theta [\hat{\boldsymbol{\omega}}]\mathbf{p}$$

ve $\mathbf{p}_{\perp} = -[\hat{\boldsymbol{\omega}}]^2 \mathbf{p}$ koyunca

$$\boxed{\mathbf{R} = \mathbf{I} + \sin\theta\, [\hat{\boldsymbol{\omega}}] + (1 - \cos\theta)[\hat{\boldsymbol{\omega}}]^2} \qquad \text{(Rodrigues)}.$$

2.3.3 Aynı formül: matris üsteli

$\boldsymbol{\omega} = \theta\hat{\boldsymbol{\omega}}$ ile $\exp([\boldsymbol{\omega}]) = \sum_k \frac{[\boldsymbol{\omega}]^k}{k!}$. $[\hat{\boldsymbol{\omega}}]^3 = -[\hat{\boldsymbol{\omega}}]$ olduğundan kuvvetler iki türe ayrılır: tek kuvvetler $\pm[\hat{\boldsymbol{\omega}}]$, çift kuvvetler $\pm[\hat{\boldsymbol{\omega}}]^2$:

$$\exp(\theta[\hat{\boldsymbol{\omega}}]) = \mathbf{I} + \Big(\theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \ldots\Big)[\hat{\boldsymbol{\omega}}] + \Big(\frac{\theta^2}{2!} - \frac{\theta^4}{4!} + \ldots\Big)[\hat{\boldsymbol{\omega}}]^2 = \mathbf{I} + \sin\theta[\hat{\boldsymbol{\omega}}] + (1 - \cos\theta)[\hat{\boldsymbol{\omega}}]^2.$$

İki türetme aynı sonucu veriyor; üstel biçimin anlamı Bölüm 2.5'te ("sabit açısal hızla θ süre dönmek") netleşecek.

Elle: Rodrigues ile z etrafında 90°

$\hat{\boldsymbol{\omega}} = (0,0,1)$, $\theta = 90^\circ$, $\mathbf{p} = (1,0,0)$. $[\hat{\boldsymbol{\omega}}]\mathbf{p} = \hat{\boldsymbol{\omega}} \times \mathbf{p} = (0,1,0)$; $[\hat{\boldsymbol{\omega}}]^2\mathbf{p} = \hat{\boldsymbol{\omega}} \times (0,1,0) = (-1,0,0)$. $\mathbf{R}\mathbf{p} = (1,0,0) + 1 \cdot (0,1,0) + (1-0)(-1,0,0) = (0,1,0)$. Doğru: x ekseni y'ye döndü.

2.3.4 Logaritma: R'den eksen ve açıyı çıkarmak

Rodrigues formülünün izini (köşegen toplamı) al: $\mathrm{iz}(\mathbf{I}) = 3$, $\mathrm{iz}([\hat{\boldsymbol{\omega}}]) = 0$, $\mathrm{iz}([\hat{\boldsymbol{\omega}}]^2) = \mathrm{iz}(\hat{\boldsymbol{\omega}}\hat{\boldsymbol{\omega}}^T) - 3 = 1 - 3 = -2$:

$$\mathrm{iz}\,\mathbf{R} = 3 + 0 - 2(1 - \cos\theta) = 1 + 2\cos\theta \;\Rightarrow\; \theta = \arccos\frac{\mathrm{iz}\,\mathbf{R} - 1}{2}.$$

Ters simetrik kısım: $[\hat{\boldsymbol{\omega}}]^2$ simetrik olduğundan $\mathbf{R} - \mathbf{R}^T = 2\sin\theta[\hat{\boldsymbol{\omega}}]$, yani $[\hat{\boldsymbol{\omega}}] = \frac{\mathbf{R}-\mathbf{R}^T}{2\sin\theta}$. İki özel durum: $\theta \approx 0$ (pay ve payda sıfır; eksen önemsiz, $\boldsymbol{\omega} \approx 0$) ve $\theta \approx \pi$ ($\sin\theta \approx 0$ ama eksen önemli; o zaman $\mathbf{R} = \mathbf{I} + 2[\hat{\boldsymbol{\omega}}]^2 = 2\hat{\boldsymbol{\omega}}\hat{\boldsymbol{\omega}}^T - \mathbf{I}$'den $\hat{\boldsymbol{\omega}}\hat{\boldsymbol{\omega}}^T = (\mathbf{R} + \mathbf{I})/2$ okunur). Kodda bu yüzden iki özel dal var.

Üstel harita, iki yönelim arasındaki "hatayı" tek bir 3-vektör olarak verir: $\mathbf{e}_R = \log(\mathbf{R}_d^T\mathbf{R})$. Gövde yönelim kontrolünde (Cilt 6) PD'nin oransal terimi budur.

2.4 Kuaterniyonlar

2.4.1 Tanım ve çarpım

Kuaterniyon $q = w + xi + yj + zk$, Hamilton kurallarıyla: $i^2 = j^2 = k^2 = ijk = -1$, buradan $ij = k$, $ji = -k$, $jk = i$, $kj = -i$, $ki = j$, $ik = -j$. Skaler–vektör biçiminde $q = (w, \mathbf{v})$ yazınca iki kuaterniyonun çarpımı (terim terim açıp yukarıdaki kuralları uygulayınca; i, j, k çarpımları tam olarak çapraz çarpım tablosudur):

$$q_1 \otimes q_2 = (w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2,\quad w_1\mathbf{v}_2 + w_2\mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2).$$

Çapraz çarpım terimi yüzünden çarpım değişmeli değildir: dönmeler gibi.

2.4.2 Birim kuaterniyon = dönme

Eksen $\hat{\boldsymbol{\omega}}$, açı θ için $q = (\cos\frac{\theta}{2},\ \hat{\boldsymbol{\omega}}\sin\frac{\theta}{2})$, $|q| = 1$. Bir vektör $\mathbf{p}$'yi $p = (0, \mathbf{p})$ saf kuaterniyonu yapıp $q \otimes p \otimes q^*$ ile döndürürüz ($q^* = (w, -\mathbf{v})$ eşlenik, birim için ters). Bu işlemin açılımı Rodrigues'e eşittir. Türetmenin kısa yolu: dönme matrisini doğrudan kuaterniyon bileşenleriyle yazmak. Rodrigues'te yarım açı özdeşliklerini kullan: $\sin\theta = 2\sin\frac{\theta}{2}\cos\frac{\theta}{2}$, $1 - \cos\theta = 2\sin^2\frac{\theta}{2}$. $\mathbf{v} = \hat{\boldsymbol{\omega}}\sin\frac{\theta}{2}$ olduğundan $\sin\theta[\hat{\boldsymbol{\omega}}] = 2w[\mathbf{v}]$ ve $(1 - \cos\theta)[\hat{\boldsymbol{\omega}}]^2 = 2[\mathbf{v}]^2$:

$$\boxed{\mathbf{R}(q) = \mathbf{I} + 2w[\mathbf{v}] + 2[\mathbf{v}]^2}$$

Bu formül tam olarak kodundaki `quat_to_R`'dir (elemanlar açılınca $1 - 2(y^2 + z^2)$, $2(xy - wz)$ gibi tanıdık terimler çıkar).

Neden yarım açı? $q \otimes p \otimes q^*$'de q iki kez kullanılıyor; her biri $\theta/2$ katkı yapar. Bunun bir sonucu **çift örtü**: θ yerine $\theta + 2\pi$ koyunca $q \to -q$ ama dönme aynı. Yani q ve $-q$ *aynı* dönmedir. İki kuaterniyonu karşılaştırırken veya ortalarken işareti hizala ($q_a \cdot q_b < 0$ ise birini çevir).

Elle: kuaterniyonla z etrafında $90°$

$q = (\cos 45°,\ 0, 0, \sin 45°) = (0.7071,\ 0, 0,\ 0.7071)$, yani $w = 0.7071$, $\mathbf{v} = (0, 0, 0.7071)$. $\mathbf{R} = \mathbf{I} + 2w[\mathbf{v}] + 2[\mathbf{v}]^2$: $2w[\mathbf{v}]$'nin $(2,1)$ elemanı $2 \cdot 0.7071 \cdot 0.7071 = 1$, $(1,2)$'si -1; $2[\mathbf{v}]^2 = 2 \cdot 0.5 \cdot$

diag$(-1,-1,0)$. Toplam: $\begin{bmatrix}0 & -1 & 0\\ 1 & 0 & 0\\ 0 & 0 & 1\end{bmatrix} = \mathbf{R}_z(90^\circ)$.

Sıra konvansiyonu: MuJoCo ve bu seri $[w,x,y,z]$; `scipy.spatial.transform.Rotation` ve ROS mesajları $[x,y,z,w]$; Eigen'in `Quaterniond(w,x,y,z)` kurucusu w önce alır ama `coeffs()` (x,y,z,w) döndürür. Bir projede bu karışıklık en az bir kez başına gelecek; her arayüzde sırayı yorum olarak yaz.

2.4.3 Slerp: iki yönelim arasında sabit hızla geçiş

q_0'dan q_1'e, aralarındaki açı Ω ($\cos\Omega = q_0 \cdot q_1$, 4-B iç çarpım):

$$\mathrm{slerp}(t) = \frac{\sin((1-t)\Omega)}{\sin\Omega} q_0 + \frac{\sin(t\Omega)}{\sin\Omega} q_1.$$

Türetme: birim kuaterniyonlar 4-B birim küre üzerinde; q_0 ve q_1'in gerdiği düzlemde, büyük çember üzerinde sabit açısal hızla gitmek istiyoruz. O düzlemde q_0'a dik birim vektör $q_\perp = (q_1 - \cos\Omega\, q_0)/\sin\Omega$; çember noktası $\cos(t\Omega)q_0 + \sin(t\Omega)q_\perp$. $q_\perp$'yi yerine koyup $\sin(\Omega - t\Omega) = \sin\Omega\cos t\Omega - \cos\Omega\sin t\Omega$ ile düzenleyince formül çıkar. "Lineer interpolasyon + normalize" aynı yolu izler ama sabit hızla değil (aşağıdaki sayılar).

2.5 Açısal hız ve yönelim integrasyonu

2.5.1 $\dot{\mathbf{R}}$ nasıl görünür?

$\mathbf{R}(t)^T\mathbf{R}(t) = \mathbf{I}$'nin zamana göre türevi: $\dot{\mathbf{R}}^T\mathbf{R} + \mathbf{R}^T\dot{\mathbf{R}} = \mathbf{0}$. Yani $\mathbf{R}^T\dot{\mathbf{R}}$ matrisi kendi transpozunun eksisidir: **ters simetriktir**, dolayısıyla bir vektörün çapraz çarpım matrisi olarak yazılabilir:

$$\mathbf{R}^T\dot{\mathbf{R}} = [\boldsymbol{\omega}_b] \;\Rightarrow\; \dot{\mathbf{R}} = \mathbf{R}[\boldsymbol{\omega}_b], \qquad \text{aynı şekilde } \dot{\mathbf{R}}\mathbf{R}^T = [\boldsymbol{\omega}_s] \;\Rightarrow\; \dot{\mathbf{R}} = [\boldsymbol{\omega}_s]\mathbf{R}.$$

$\boldsymbol{\omega}_b$: gövde çerçevesinde ifade edilmiş açısal hız (IMU jiroskobunun ölçtüğü şey), $\boldsymbol{\omega}_s = \mathbf{R}\boldsymbol{\omega}_b$: dünyada ifade edilmiş. ($\mathbf{R}[\boldsymbol{\omega}_b]\mathbf{R}^T = [\mathbf{R}\boldsymbol{\omega}_b]$ özdeşliğinden.)

Üstel haritanın anlamı: $\boldsymbol{\omega}_b$ sabitse $\dot{\mathbf{R}} = \mathbf{R}[\boldsymbol{\omega}_b]$ doğrusal sabit katsayılı bir diferansiyel denklemdir; çözümü $\mathbf{R}(t) = \mathbf{R}(0)\exp([\boldsymbol{\omega}_b]t)$. Yani $\exp([\boldsymbol{\omega}])$ "$\boldsymbol{\omega}$ açısal hızıyla 1 saniye dönmektir."

2.5.2 Naif Euler neden SO(3)'ten çıkar?

Naif adım $\mathbf{R}_{k+1} = \mathbf{R}_k(\mathbf{I} + [\boldsymbol{\omega}]\Delta t)$. Ortogonalliği kontrol edelim:

$$(\mathbf{I} + [\boldsymbol{\omega}]\Delta t)^T(\mathbf{I} + [\boldsymbol{\omega}]\Delta t) = \mathbf{I} + \underbrace{([\boldsymbol{\omega}]^T + [\boldsymbol{\omega}])}_{=0}\Delta t + [\boldsymbol{\omega}]^T[\boldsymbol{\omega}]\Delta t^2$$
$$= \mathbf{I} + \Delta t^2(\|\boldsymbol{\omega}\|^2\mathbf{I} - \boldsymbol{\omega}\boldsymbol{\omega}^T).$$

Her adımda $\mathbf{R}^T\mathbf{R}$, $\boldsymbol{\omega}$'ya dik yönlerde $1 + \|\boldsymbol{\omega}\|^2\Delta t^2$ çarpanıyla büyür. N adım sonra bu yönlerdeki tekil değer $(1+\|\boldsymbol{\omega}\|^2\Delta t^2)^{N/2}$. Aşağıdaki örnekte $\|\boldsymbol{\omega}\|^2 = 0.5^2+0.3^2+1^2 = 1.34$, $\Delta t = 0.01$, $N = 1000$: tekil değer $\approx (1.000134)^{500} = 1.069$ ve $\mathbf{R}^T\mathbf{R}-\mathbf{I}$ hatası $\approx N\|\boldsymbol{\omega}\|^2\Delta t^2 = 0.134$. Simülasyonda ölçülen değerlerle birebir: **hatanın nereden geldiğini türetince sayıyı önceden tahmin edebilirsin.**

Doğru yol: grup üzerinde integrasyon. $\mathbf{R}_{k+1} = \mathbf{R}_k \exp([\boldsymbol{\omega}]\Delta t)$: her adım tam bir dönmeyle çarpılır, sonuç SO(3)'te kalır. $\boldsymbol{\omega}$ bir adım boyunca sabitse bu kesindir. Kuaterniyonla aynı şey $\dot{q} = \frac{1}{2} q \otimes (0, \boldsymbol{\omega}_b)$ (türetme: $q(t+\Delta t) = q(t) \otimes (\cos\frac{\|\boldsymbol{\omega}\|\Delta t}{2},\ \hat{\boldsymbol{\omega}} \sin\frac{\|\boldsymbol{\omega}\|\Delta t}{2})$'yi Δt'ye göre türev al, $\Delta t \to 0$'da $\frac{1}{2} q \otimes (0, \boldsymbol{\omega})$); Euler adımından sonra her seferinde normalize edilmelidir.

Sayılar: gösterimler ve integrasyon

RPY $= (0.1, 0.2, 0.3)$ rad: kuaterniyon [0.98335, 0.03427, 0.10602, 0.14357], eksen-açı 0.3655 rad; tüm gidiş-dönüş dönüşümleri $\sim 10^{-16}$ hata. **Gimbal lock:** $\mathbf{R}(0.3, 90°, 0.5)$ ile $\mathbf{R}(0.1, 90°, 0.3)$ makine hassasiyetinde aynı (türetmedeki gibi, $\phi - \psi$ ikisinde de -0.2). $\mathbf{R} \to$RPY dönüşümü (0.3, 1.5708, 0.5) veriyor ama matrise 10^{-10} büyüklüğünde bir hata eklenince roll $0.3 \to 2.356$'ya sıçrıyor: tekil noktada çıktı girdiye "sonsuz" duyarlı (`atan2`(0,0) civarında). **İntegrasyon** ($\boldsymbol{\omega}_b = (0.5, -0.3, 1.0)$ rad/s, 10 s, $\Delta t = 0.01$):

Yöntem	açı hatası [rad]	$\max \lvert\mathbf{R}^T\mathbf{R} - \mathbf{I}\rvert$
Naif Euler ($\mathbf{R} \mathrel{+}= \mathbf{R}[\boldsymbol{\omega}]\Delta t$)	anlamsız (matris dönme değil)	0.134 (tekil değerler 1.069)
Grup üzerinde ($\mathbf{R}\exp([\boldsymbol{\omega}]\Delta t)$)	2×10^{-15}	9×10^{-14}
Kuaterniyon Euler + normalize	1.3×10^{-4}	–

Sabit $\boldsymbol{\omega}$'da grup üzerinde integrasyon *kesin*dir. Kuaterniyonun birinci dereceden Euler adımı $\mathcal{O}(\Delta t)$ hata bırakıyor. **Slerp:** yaw $0 \to 2.5$ rad, 10 eşit adımda slerp her adımda tam 0.25 rad; "lineer + normalize" 0.203 ile 0.287 rad arasında değişen adımlar veriyor (açısal hız sabit değil).

Paylaşılan kütüphane (Python): `robkit.py`

```python
"""robkit.py — Cilt 1 boyunca büyüyen küçük kinematik kütüphanesi (C++ karşılığı: robkit.hpp).
Konvansiyonlar: kuaterniyon q = [w, x, y, z] (Hamilton, MuJoCo ile aynı); RPY = ZYX: R = Rz(yaw) Ry(pitch) Rx(roll);
twist/vida ekseni S = [ω; v] (önce açısal, sonra doğrusal; Modern Robotics ile aynı)."""
import numpy as np

# ---------------- SO(3) ----------------
def skew(w):
    """w x (.) çapraz çarpım matrisi: skew(w) @ p == np.cross(w, p)"""
    return np.array([[0, -w[2], w[1]], [w[2], 0, -w[0]], [-w[1], w[0], 0]])

def unskew(S):
    return np.array([S[2, 1], S[0, 2], S[1, 0]])

def exp_so3(w):
    """Rodrigues: R = exp([w]) ; |w| = dönme açısı, w/|w| = eksen"""
    th = np.linalg.norm(w)
    if th < 1e-12:
        return np.eye(3) + skew(w)                      # birinci dereceden yaklaşım
    K = skew(w/th)
    return np.eye(3) + np.sin(th)*K + (1 - np.cos(th))*K@K

def log_so3(R):
    """R -> w (eksen*açı), açı [0, pi]"""
    c = np.clip((np.trace(R) - 1)/2, -1.0, 1.0); th = np.arccos(c)
    if th < 1e-9:
        return unskew(R - R.T)/2
    if np.pi - th < 1e-6:                               # 180 derece civarı: sin ~ 0, ayrı formül
        k = np.argmax(np.diag(R)); a = np.sqrt(max((R[k, k] + 1)/2, 0.0)); w = R[:, k]/(2*a); w[k] = a
        return th*w/np.linalg.norm(w)
```

```python
    return th/(2*np.sin(th))*unskew(R - R.T)

def rot_x(a): c, s = np.cos(a), np.sin(a); return np.array([[1, 0, 0], [0, c, -s], [0, s, c]])
def rot_y(a): c, s = np.cos(a), np.sin(a); return np.array([[c, 0, s], [0, 1, 0], [-s, 0, c]])
def rot_z(a): c, s = np.cos(a), np.sin(a); return np.array([[c, -s, 0], [s, c, 0], [0, 0, 1]])

def rpy_to_R(roll, pitch, yaw):
    return rot_z(yaw)@rot_y(pitch)@rot_x(roll)

def R_to_rpy(R):
    pitch = np.arcsin(np.clip(-R[2, 0], -1.0, 1.0))
    return np.array([np.arctan2(R[2, 1], R[2, 2]), pitch, np.arctan2(R[1, 0], R[0, 0])])

# ---------------- Kuaterniyon [w, x, y, z] ----------------
def quat_mul(a, b):
    w1, v1 = a[0], a[1:]; w2, v2 = b[0], b[1:]
    return np.concatenate([[w1*w2 - v1@v2], w1*v2 + w2*v1 + np.cross(v1, v2)])

def quat_to_R(q):
    w, x, y, z = q/np.linalg.norm(q)
    return np.array([[1 - 2*(y*y + z*z), 2*(x*y - w*z), 2*(x*z + w*y)],
                     [2*(x*y + w*z), 1 - 2*(x*x + z*z), 2*(y*z - w*x)],
                     [2*(x*z - w*y), 2*(y*z + w*x), 1 - 2*(x*x + y*y)]])

def R_to_quat(R):
    """Shepperd yöntemi: en büyük bileşenden başla (sayısal kararlılık); w >= 0 seçilir"""
    t = np.trace(R)
    if t > 0:
        s = 2*np.sqrt(t + 1); q = np.array([s/4, (R[2, 1] - R[1, 2])/s, (R[0, 2] - R[2, 0])/s, (R[1, 0]
        - R[0, 1])/s])
    else:
        i = np.argmax(np.diag(R)); j, k = (i + 1) % 3, (i + 2) % 3
        s = 2*np.sqrt(1 + R[i, i] - R[j, j] - R[k, k]); q = np.zeros(4)
        q[0] = (R[k, j] - R[j, k])/s; q[1 + i] = s/4; q[1 + j] = (R[j, i] + R[i, j])/s; q[1 + k] = (R[k,
        i] + R[i, k])/s
    return q if q[0] >= 0 else -q

def quat_exp(w):
    """exp(w/2) kuaterniyonu: |w| açısı kadar w ekseni etrafında dönme"""
    th = np.linalg.norm(w)
    if th < 1e-12:
        return np.array([1.0, *(w/2)])
    return np.concatenate([[np.cos(th/2)], np.sin(th/2)*w/th])

# ---------------- SE(3) ----------------
def homog(R, p):
    T = np.eye(4); T[:3, :3] = R; T[:3, 3] = p; return T

def inv_T(T):
    R, p = T[:3, :3], T[:3, 3]; return homog(R.T, -R.T@p)

def exp_se3(S, th):
    """Vida ekseni S =[w; v] etrafında th kadar hareket: e^{[S] th} (Modern Robotics 3.88)"""
    w, v = S[:3], S[3:]
    if np.linalg.norm(w) < 1e-12:                        # saf öteleme (prizmatik)
        return homog(np.eye(3), v*th)
    K = skew(w); R = exp_so3(w*th)
    G = np.eye(3)*th + (1 - np.cos(th))*K + (th - np.sin(th))*K@K
    return homog(R, G@v)

```

```
def adjoint(T):
    """6x6 Ad_T: bir twist'i {b}'den {s}'ye taşır: V_s = Ad_T V_b"""
    R, p = T[:3, :3], T[:3, 3]; A = np.zeros((6, 6))
    A[:3, :3] = R; A[3:, 3:] = R; A[3:, :3] = skew(p)@R
    return A

# ---------------- Seri zincir (çarpımların üsteli, PoE) ----------------
def fk_poe(Slist, M, q):
    """T(q) = e^{[S1]q1} ... e^{[Sn]qn} M   (uzay çerçevesinde vida eksenleri)"""
    T = np.eye(4)
    for S, qi in zip(Slist, q):
        T = T@exp_se3(S, qi)
    return T@M

def space_jacobian(Slist, q):
    """J_s sütun i = Ad_{e^{[S1]q1}...e^{[S_{i-1}]q_{i-1}}} S_i  (6 x n, satırlar [ω; v])"""
    n = len(q); J = np.zeros((6, n)); T = np.eye(4)
    for i in range(n):
        J[:, i] = adjoint(T)@Slist[i]
        T = T@exp_se3(Slist[i], q[i])
    return J
```

Python

```
import numpy as np
from robkit import (skew, exp_so3, log_so3, rpy_to_R, R_to_rpy, quat_mul, quat_to_R, R_to_quat,
quat_exp)
np.set_printoptions(precision=5, suppress=True)

# (1) Gösterimler arası dönüşüm ve tutarlılık
rpy = np.array([0.1, 0.2, 0.3])                                  # roll, pitch, yaw [rad]
R = rpy_to_R(*rpy)
print("R =\n", R)
print("RᵀR - I  max:", np.abs(R.T@R - np.eye(3)).max(), "  det R =", np.linalg.det(R))
q = R_to_quat(R); w = log_so3(R)
print("kuaterniyon [w x y z] =", q, " |q| =", np.linalg.norm(q))
print("eksen-açı: açı =", np.linalg.norm(w), "rad, eksen =", w/np.linalg.norm(w))
print("geri dönüşüm hataları: quat->R", np.abs(quat_to_R(q) - R).max(), " exp(log R)",
np.abs(exp_so3(w) - R).max(), " R->rpy", np.abs(R_to_rpy(R) - rpy).max())
print("-q aynı dönmeyi verir mi?", np.allclose(quat_to_R(-q), R), "(çift örtü: q ve -q)")

# (2) Gimbal lock: pitch = 90 derecede roll ve yaw ayırt edilemez
Ra = rpy_to_R(0.3, np.pi/2, 0.5); Rb = rpy_to_R(0.1, np.pi/2, 0.3); Rc = rpy_to_R(0.0, np.pi/2, 0.8)
print(f"\ngimbal lock: |R(0.3,90°,0.5) - R(0.1,90°,0.3)| = {np.abs(Ra-Rb).max():.1e}  (aynı!),
|R(0.3,90°,0.5) - R(0,90°,0.8)| = {np.abs(Ra-Rc).max():.3f}")
E = 1e-10*np.array([[0, 0, 0], [0, 0, 0], [0, 1, -1]])              # 1e-10 mertebesinde
ölçüm/yuvarlama hatası
print("  pitch =90°'de R sadece (yaw - roll)'a bağlı. R->rpy:", R_to_rpy(Ra), " ama R'ye 1e-10 gürültü
eklenince:", R_to_rpy(Ra + E))

# (3) Sabit gövde açısal hızıyla yönelim integrasyonu: Ṙ = R [ω]
wb = np.array([0.5, -0.3, 1.0]); dt, T = 0.01, 10.0; N = int(T/dt)
R_true = exp_so3(wb*T)                                           # sabit ω için kesin çözüm
R_eu = np.eye(3); R_ex = np.eye(3); q_n = np.array([1.0, 0, 0, 0]); q_u = q_n.copy()
for k in range(N):
    R_eu = R_eu + R_eu@skew(wb)*dt                               # naif Euler: SO(3)'ten çıkar
    R_ex = R_ex@exp_so3(wb*dt)                                  # grup üzerinde (geometrik) integrasyon
    q_u = q_u + 0.5*quat_mul(q_u, np.array([0, *wb]))*dt         # kuaterniyon Euler, normalize ETMEDEN
    q_n = q_n + 0.5*quat_mul(q_n, np.array([0, *wb]))*dt; q_n /= np.linalg.norm(q_n)   # ... normalize
    EDEREK
```

```python
ang_err = lambda Ra: np.linalg.norm(log_so3(Ra.T@R_true))
orth = lambda Ra: np.abs(Ra.T@Ra - np.eye(3)).max()
print(f"\n10 s, dt =0.01, ω ={wb}:")
print(f"  naif Euler R:          açı hatası {ang_err(R_eu):.2e} rad*, ortogonallik hatası {orth(R_eu):.2e}")
print(f"  exp ile (geometrik):   açı hatası {ang_err(R_ex):.2e} rad,  ortogonallik hatası {orth(R_ex):.2e}")
print(f"  kuaterniyon, normsuz:  |q| = {np.linalg.norm(q_u):.5f}, açı hatası {ang_err(quat_to_R(q_u)):.2e} rad (quat_to_R normalize ediyor)")
print(f"  kuaterniyon, normlu:   |q| = {np.linalg.norm(q_n):.5f}, açı hatası {ang_err(quat_to_R(q_n)):.2e} rad")
U, s, Vt = np.linalg.svd(R_eu); print(f"  * naif Euler matrisi artık bir dönme değil: tekil değerleri {s.round(4)} (hepsi 1 olmalı); log'u anlamsız")

# (4) İki yönelim arasında ara değer: slerp vs normalize edilmiş lineer
qa = R_to_quat(rpy_to_R(0, 0, 0)); qb = R_to_quat(rpy_to_R(0.0, 0.0, 2.5))
def slerp(a, b, t):
    c = np.clip(a@b, -1, 1); th = np.arccos(c)
    return (np.sin((1-t)*th)*a + np.sin(t*th)*b)/np.sin(th)
ts = np.linspace(0, 1, 11)
ang = lambda qq: 2*np.arccos(np.clip(abs(qq[0]), -1, 1))
d_sl = np.diff([ang(slerp(qa, qb, t)) for t in ts]); d_ln = np.diff([ang(((1-t)*qa + t*qb)/np.linalg.norm((1-t)*qa + t*qb)) for t in ts])
print(f"\n0'dan 2.5 rad'a yaw geçişi, 10 eşit adım: slerp adımları {d_sl.min():.4f}..{d_sl.max():.4f} rad (sabit), normalize-lineer adımları {d_ln.min():.4f}..{d_ln.max():.4f} rad (değişken)")
```

Çıktı:

```
R =
 [[ 0.93629 -0.2751   0.21835]
 [ 0.28963  0.95643 -0.03696]
 [-0.19867  0.09784  0.97517]]
RᵀR - I  max: 2.220446049250313e-16   det R = 0.9999999999999999
kuaterniyon [w x y z] = [0.98335 0.03427 0.10602 0.14357]  |q| = 1.0
eksen-açı: açı = 0.36550218635669873 rad, eksen = [0.18858 0.58338 0.79001]
geri dönüşüm hataları: quat->R 1.1102230246251565e-16  exp(log R) 2.7755575615628914e-17  R->rpy 2.7755575615628914e-17
-q aynı dönmeyi verir mi? True (çift örtü: q ve -q)

gimbal lock: |R(0.3,90°,0.5) - R(0.1,90°,0.3)| = 2.8e-17  (aynı!),  |R(0.3,90°,0.5) - R(0,90°,0.8)| = 0.519
  pitch =90°'de R sadece (yaw - roll)'a bağlı. R->rpy: [0.3    1.5708 0.5   ]  ama R'ye 1e-10 gürültü eklenince:
  [2.35619 1.5708  0.5    ]

10 s, dt =0.01, ω =[ 0.5 -0.3  1. ]:
  naif Euler R:          açı hatası 5.53e-04 rad*, ortogonallik hatası 1.34e-01
  exp ile (geometrik):   açı hatası 1.87e-15 rad,  ortogonallik hatası 8.74e-14
  kuaterniyon, normsuz:  |q| = 1.01689, açı hatası 1.29e-04 rad (quat_to_R normalize ediyor)
  kuaterniyon, normlu:   |q| = 1.00000, açı hatası 1.29e-04 rad
  * naif Euler matrisi artık bir dönme değil: tekil değerleri [1.0693 1.0693 1.    ] (hepsi 1 olmalı); log'u anlamsız

0'dan 2.5 rad'a yaw geçişi, 10 eşit adım: slerp adımları 0.2500..0.2500 rad (sabit), normalize-lineer adımları 0.2030..0.2866 rad (değişken)
```

Paylaşılan kütüphane (C++): `robkit.hpp`

```cpp
// robkit.hpp — robkit.py'nin C++/Eigen karşılığı (sadece başlık dosyası; #include "robkit.hpp" yeterli).
// Konvansiyonlar Python ile aynı: kuaterniyon [w,x,y,z], RPY = ZYX, twist S = [ω; v].
#pragma once
#include <Eigen/Dense>
#include <cmath>
#include <vector>
```

```cpp

namespace rk {
using Mat3 = Eigen::Matrix3d;  using Vec3 = Eigen::Vector3d;
using Mat4 = Eigen::Matrix4d;  using Vec4 = Eigen::Vector4d;
using Mat6 = Eigen::Matrix<double, 6, 6>;  using Vec6 = Eigen::Matrix<double, 6, 1>;
using Jac  = Eigen::Matrix<double, 6, Eigen::Dynamic>;

inline Mat3 skew(const Vec3& w) {
    Mat3 S; S << 0, -w(2), w(1),  w(2), 0, -w(0),  -w(1), w(0), 0; return S;
}
inline Vec3 unskew(const Mat3& S) { return Vec3(S(2, 1), S(0, 2), S(1, 0)); }

inline Mat3 exp_so3(const Vec3& w) {                       // Rodrigues
    double th = w.norm();
    if (th < 1e-12) return Mat3::Identity() + skew(w);
    Mat3 K = skew(w / th);
    return Mat3::Identity() + std::sin(th) * K + (1 - std::cos(th)) * K * K;
}

inline Vec3 log_so3(const Mat3& R) {
    double c = std::clamp((R.trace() - 1) / 2, -1.0, 1.0), th = std::acos(c);
    if (th < 1e-9) return unskew(R - R.transpose()) / 2;
    if (M_PI - th < 1e-6) {                                  // 180 derece civarı
        int k; R.diagonal().maxCoeff(&k);
        double a = std::sqrt(std::max((R(k, k) + 1) / 2, 0.0));
        Vec3 w = R.col(k) / (2 * a); w(k) = a;
        return th * w.normalized();
    }
    return th / (2 * std::sin(th)) * unskew(R - R.transpose());
}

inline Mat3 rot_x(double a) { Mat3 R; R << 1, 0, 0, 0, std::cos(a), -std::sin(a), 0, std::sin(a),
std::cos(a); return R; }
inline Mat3 rot_y(double a) { Mat3 R; R << std::cos(a), 0, std::sin(a), 0, 1, 0, -std::sin(a), 0,
std::cos(a); return R; }
inline Mat3 rot_z(double a) { Mat3 R; R << std::cos(a), -std::sin(a), 0, std::sin(a), std::cos(a), 0,
0, 0, 1; return R; }
inline Mat3 rpy_to_R(double r, double p, double y) { return rot_z(y) * rot_y(p) * rot_x(r); }
inline Vec3 R_to_rpy(const Mat3& R) {
    return Vec3(std::atan2(R(2, 1), R(2, 2)), std::asin(std::clamp(-R(2, 0), -1.0, 1.0)),
std::atan2(R(1, 0), R(0, 0)));
}

// ---- kuaterniyon [w,x,y,z] (Eigen::Quaterniond yerine açık formül: Python ile birebir) ----
inline Vec4 quat_mul(const Vec4& a, const Vec4& b) {
    Vec3 v1 = a.tail<3>(), v2 = b.tail<3>();
    Vec4 r; r(0) = a(0) * b(0) - v1.dot(v2);
    r.tail<3>() = a(0) * v2 + b(0) * v1 + v1.cross(v2);
    return r;
}
inline Mat3 quat_to_R(Vec4 q) {
    q.normalize(); double w = q(0), x = q(1), y = q(2), z = q(3);
    Mat3 R;
    R << 1 - 2 * (y * y + z * z), 2 * (x * y - w * z), 2 * (x * z + w * y),
         2 * (x * y + w * z), 1 - 2 * (x * x + z * z), 2 * (y * z - w * x),
         2 * (x * z - w * y), 2 * (y * z + w * x), 1 - 2 * (x * x + y * y);
    return R;
}
inline Vec4 R_to_quat(const Mat3& R) {                     // Shepperd
    Vec4 q; double t = R.trace();
```

```
    if (t > 0) {
        double s = 2 * std::sqrt(t + 1);
        q << s / 4, (R(2, 1) - R(1, 2)) / s, (R(0, 2) - R(2, 0)) / s, (R(1, 0) - R(0, 1)) / s;
    } else {
        int i; R.diagonal().maxCoeff(&i); int j = (i + 1) % 3, k = (i + 2) % 3;
        double s = 2 * std::sqrt(1 + R(i, i) - R(j, j) - R(k, k));
        q(0) = (R(k, j) - R(j, k)) / s; q(1 + i) = s / 4;
        q(1 + j) = (R(j, i) + R(i, j)) / s; q(1 + k) = (R(k, i) + R(i, k)) / s;
    }
    return q(0) >= 0 ? q : Vec4(-q);
}
inline Vec4 quat_exp(const Vec3& w) {
    double th = w.norm(); Vec4 q;
    if (th < 1e-12) { q << 1, w / 2; return q; }
    q << std::cos(th / 2), std::sin(th / 2) * w / th; return q;
}

// ---- SE(3) ----
inline Mat4 homog(const Mat3& R, const Vec3& p) {
    Mat4 T = Mat4::Identity(); T.topLeftCorner<3, 3>() = R; T.topRightCorner<3, 1>() = p; return T;
}
inline Mat4 inv_T(const Mat4& T) {
    Mat3 R = T.topLeftCorner<3, 3>(); Vec3 p = T.topRightCorner<3, 1>();
    return homog(R.transpose(), -R.transpose() * p);
}
inline Mat4 exp_se3(const Vec6& S, double th) {
    Vec3 w = S.head<3>(), v = S.tail<3>();
    if (w.norm() < 1e-12) return homog(Mat3::Identity(), v * th);
    Mat3 K = skew(w);
    Mat3 G = Mat3::Identity() * th + (1 - std::cos(th)) * K + (th - std::sin(th)) * K * K;
    return homog(exp_so3(w * th), G * v);
}
inline Mat6 adjoint(const Mat4& T) {
    Mat3 R = T.topLeftCorner<3, 3>(); Vec3 p = T.topRightCorner<3, 1>();
    Mat6 A = Mat6::Zero();
    A.topLeftCorner<3, 3>() = R; A.bottomRightCorner<3, 3>() = R; A.bottomLeftCorner<3, 3>() = skew(p)
* R;
    return A;
}
inline Mat4 fk_poe(const std::vector<Vec6>& S, const Mat4& M, const Eigen::VectorXd& q) {
    Mat4 T = Mat4::Identity();
    for (size_t i = 0; i < S.size(); ++i) T = T * exp_se3(S[i], q(i));
    return T * M;
}
inline Jac space_jacobian(const std::vector<Vec6>& S, const Eigen::VectorXd& q) {
    Jac J(6, q.size()); Mat4 T = Mat4::Identity();
    for (int i = 0; i < q.size(); ++i) { J.col(i) = adjoint(T) * S[i]; T = T * exp_se3(S[i], q(i)); }
    return J;
}
}  // namespace rk
```

C++

```
// c02_rot.cpp — dönme gösterimleri ve yönelim integrasyonu (Python c02_rot.py ile aynı sayılar)
#include "robkit.hpp"
#include <cstdio>
#include <iostream>
using namespace rk;

```

```cpp
double ang_err(const Mat3& Ra, const Mat3& Rt) { return log_so3(Ra.transpose() * Rt).norm(); }
double orth_err(const Mat3& Ra) { return (Ra.transpose() * Ra - Mat3::Identity()).cwiseAbs().maxCoeff(); }

int main() {
    Eigen::IOFormat F(5, 0, " ", "\n", "[", "]");
    Mat3 R = rpy_to_R(0.1, 0.2, 0.3);
    std::cout << "R =\n" << R.format(F) << "\n";
    Vec4 q = R_to_quat(R); Vec3 w = log_so3(R);
    std::printf("kuaterniyon [w x y z] = [%.5f %.5f %.5f %.5f]\n", q(0), q(1), q(2), q(3));
    std::printf("eksen-açı: açı = %.5f rad\n", w.norm());
    std::printf("geri dönüşüm hataları: quat->R %.1e, exp(log R) %.1e\n",
                (quat_to_R(q) - R).cwiseAbs().maxCoeff(), (exp_so3(w) - R).cwiseAbs().maxCoeff());

    // Eigen'in kendi tipleriyle çapraz kontrol: Quaterniond (w,x,y,z) kurucusu, ama coeffs() sırası (x,y,z,w)!
    Eigen::Quaterniond qe(R);
    std::printf("Eigen::Quaterniond: w =%.5f x =%.5f y =%.5f z =%.5f  |  coeffs() = [%.5f %.5f %.5f %.5f] (x,y,z,w sırası)\n",
                qe.w(), qe.x(), qe.y(), qe.z(), qe.coeffs()(0), qe.coeffs()(1), qe.coeffs()(2), qe.coeffs()(3));

    // Gimbal lock
    Mat3 Ra = rpy_to_R(0.3, M_PI / 2, 0.5), Rb = rpy_to_R(0.1, M_PI / 2, 0.3);
    Mat3 E = Mat3::Zero(); E(2, 1) = 1e-10; E(2, 2) = -1e-10;
    Vec3 r1 = R_to_rpy(Ra), r2 = R_to_rpy(Ra + E);
    std::printf("\ngimbal lock: |Ra - Rb| = %.1e; R->rpy = (%.5f %.5f %.5f), 1e-10 gürültüyle (%.5f %.5f %.5f)\n",
                (Ra - Rb).cwiseAbs().maxCoeff(), r1(0), r1(1), r1(2), r2(0), r2(1), r2(2));

    // Yönelim integrasyonu
    Vec3 wb(0.5, -0.3, 1.0); double dt = 0.01; int N = 1000;
    Mat3 Rt = exp_so3(wb * 10.0), Reu = Mat3::Identity(), Rex = Mat3::Identity();
    Vec4 qn(1, 0, 0, 0), wq; wq << 0, wb;
    for (int k = 0; k < N; ++k) {
        Reu = Reu + Reu * skew(wb) * dt;
        Rex = Rex * exp_so3(wb * dt);
        qn = qn + 0.5 * quat_mul(qn, wq) * dt; qn.normalize();
    }
    std::printf("\n10 s, dt =0.01:\n  naif Euler: ortogonallik hatası %.2e\n  exp ile: açı hatası %.2e, ortogonallik %.2e\n  kuaterniyon (normlu): açı hatası %.2e\n",
                orth_err(Reu), ang_err(Rex, Rt), orth_err(Rex), ang_err(quat_to_R(qn), Rt));
    return 0;
}
```

Çıktı:

```
R =
[  0.93629   -0.2751   0.21835]
[  0.28963   0.95643 -0.036957]
[ -0.19867  0.097843   0.97517]
kuaterniyon [w x y z] = [0.98335 0.03427 0.10602 0.14357]
eksen-açı: açı = 0.36550 rad
geri dönüşüm hataları: quat->R 1.1e-16, exp(log R) 2.8e-17
Eigen::Quaterniond: w =0.98335 x =0.03427 y =0.10602 z =0.14357  |  coeffs() = [0.03427 0.10602 0.14357 0.98335] (x,y,z,w sırası)

gimbal lock: |Ra - Rb| = 5.6e-17; R->rpy = (0.30000 1.57080 0.50000), 1e-10 gürültüyle (2.35619 1.57080 0.50000)

10 s, dt =0.01:
  naif Euler: ortogonallik hatası 1.34e-01
```

```
exp ile: açı hatası 2.09e-15, ortogonallik 8.77e-14
kuaterniyon (normlu): açı hatası 1.29e-04
```

Python ve C++ notu: kütüphane düzeni

- **Bir kez yaz, her yerde kullan:** `robkit.py` / `robkit.hpp` bu ciltte büyüyecek. Python'da `from robkit import exp_so3`; C++'ta `#include "robkit.hpp"` ve `rk::exp_so3`. `namespace rk` isim çakışmalarını önler.
- **Başlık dosyası (`.hpp`) ve `inline`:** fonksiyon gövdesi başlıkta ise `inline` şart; yoksa iki `.cpp` dosyası aynı başlığı içerdiğinde bağlayıcı "multiple definition" hatası verir. `#pragma once` başlığın bir derlemede iki kez açılmasını önler.
- **`const Vec3&`:** büyük nesneleri kopyalamadan, değiştirmeyeceğine söz vererek geçirir (C'deki `const double*`'ın güvenli hali).
- **Sayısal özel durumlar:** `exp_so3` ve `log_so3`'teki `if th < 1e-12` dalları süs değil: $\theta \to 0$'da $\sin\theta/\theta$ formülü $0/0$ olur (logaritma türetmesindeki iki özel durum). Her trigonometrik bölmede "payda sıfır olabilir mi?" diye sor.
- **`std::clamp`** (C++17) ve `np.clip`: arccos'a $[-1, 1]$ dışından gelen 1.0000000002 gibi değerler NaN üretir.

Alıştırma

(Kâğıt) (a) $\mathbf{R}_x(90^\circ)\mathbf{R}_z(90^\circ)$ ve $\mathbf{R}_z(90^\circ)\mathbf{R}_x(90^\circ)$'yi çarp; farklı olduklarını göster ve $\mathbf{e}_1$'in nereye gittiğini telefonu çevirerek doğrula. **(b)** ZYX matrisinin $(2,3)$ elemanını elle çarparak bul. **(c)** $\theta = -90^\circ$ için gimbal lock matrisini türet; bu sefer hangi kombinasyona ($\phi+\psi$ mi $\phi-\psi$ mi) bağlı? **(d)** $\hat{\boldsymbol{\omega}} = (1,0,0)$, $\theta = 90^\circ$ ile $\mathbf{p} = (0,1,0)$'ı Rodrigues ile döndür. **(e)** $\mathbf{R}_z(60^\circ)$'nin izinden açıyı geri bul. **(f)** x ekseni etrafında 180°'nin kuaterniyonunu yaz; q ve $-q$'nun aynı $\mathbf{R}$'yi verdiğini $\mathbf{R} = \mathbf{I} + 2w[\mathbf{v}] + 2[\mathbf{v}]^2$ ile göster. **(g)** $\|\boldsymbol{\omega}\| = 2$ rad/s, $\Delta t = 5$ ms, 20 s: naif Euler'de $\mathbf{R}^T\mathbf{R} - \mathbf{I}$ hatasını önceden tahmin et. **(Bilgisayar) (h)** IMU simülasyonu: $\boldsymbol{\omega}_b(t) = (0.5\sin t,\ 0.3,\ \cos 2t)$ ile 20 s; kesin çözümü $\Delta t = 10^{-5}$'lik grup integrasyonuyla üret, $\Delta t = 0.005$'te üç yöntemi karşılaştır. **(i)** Jiroskopa sabit 0.01 rad/s bias ekle: 60 s sonra yaw hatası? (Cilt 5'teki sensör füzyonunun motivasyonu.) **(j)** `R_to_quat`'ı $\theta = 179.9^\circ$ ve 180° dönmelerde test et.

Bölüm 3

Rijit Cisim Hareketi ve İleri Kinematik

İleri kinematik (FK): eklem açıları $\mathbf{q}$ verildiğinde ayak (veya el) çerçevesinin konumu ve yönelimi $\mathbf{T}(\mathbf{q}) \in SE(3)$. Bacaklı robotta FK her kontrol döngüsünde onlarca kez çağrılır: ayakların gövdeye göre nerede olduğu, kütle merkezi, temas noktaları. Bu bölümde homojen dönüşümleri, twist'i, vida eksenini ve üstellerin çarpımı (PoE) formülünü sıfırdan türetiyoruz. Modern Robotics karşılığı: Bölüm 3.3 ve 4.

3.1 Homojen dönüşümler ve SE(3)

3.1.1 Neden 4×4?

$\{b\}$ çerçevesinin $\{s\}$'deki yönelimi $\mathbf{R}_{sb}$, orijini $\mathbf{p}_{sb}$ ise, gövdede verilen bir nokta $\mathbf{r}_b$ dünyada

$$\mathbf{r}_s = \mathbf{R}_{sb}\mathbf{r}_b + \mathbf{p}_{sb}$$

olur: önce döndür, sonra ötele. Bu bir *afin* dönüşümdür, doğrusal değil (orijini sıfıra götürmüyor), dolayısıyla 3×3 bir matrisle yazılamaz. Hile: noktaya dördüncü koordinat olarak 1 ekle (**homojen koordinatlar**):

$$\begin{bmatrix}\mathbf{r}_s\\ 1\end{bmatrix} = \underbrace{\begin{bmatrix}\mathbf{R}_{sb} & \mathbf{p}_{sb}\\ \mathbf{0}^T & 1\end{bmatrix}}_{\mathbf{T}_{sb}} \begin{bmatrix}\mathbf{r}_b\\ 1\end{bmatrix}.$$

Matrisi çarpınca üst satır tam olarak $\mathbf{R}\mathbf{r} + \mathbf{p}$, alt satır 1. Artık dönme ve öteleme tek bir matris çarpımı. (Bir *yön* vektörü ötelenmemelidir; onun dördüncü koordinatı 0 alınır.)

3.1.2 Bileşke ve ters

İki dönüşümü ardışık uygula: $\mathbf{r}_s = \mathbf{R}_{sb}(\mathbf{R}_{bc}\mathbf{r}_c + \mathbf{p}_{bc}) + \mathbf{p}_{sb} = (\mathbf{R}_{sb}\mathbf{R}_{bc})\mathbf{r}_c + (\mathbf{R}_{sb}\mathbf{p}_{bc} + \mathbf{p}_{sb})$. Blok çarpımla aynı sonuç: $\mathbf{T}_{sc} = \mathbf{T}_{sb}\mathbf{T}_{bc}$.

Ters: $\mathbf{r}_s = \mathbf{R}\mathbf{r}_b + \mathbf{p}$ denklemini $\mathbf{r}_b$ için çöz: $\mathbf{r}_b = \mathbf{R}^T(\mathbf{r}_s - \mathbf{p}) = \mathbf{R}^T\mathbf{r}_s - \mathbf{R}^T\mathbf{p}$:

$$\mathbf{T}^{-1} = \begin{bmatrix}\mathbf{R}^T & -\mathbf{R}^T\mathbf{p}\\ \mathbf{0}^T & 1\end{bmatrix}.$$

Ters dönüşümü `np.linalg.inv` ile değil bu formülle al (`inv_T`): hem hızlı hem tam olarak SE(3)'te kalır.

3.2 Twist ve vida (screw) ekseni

3.2.1 $\dot{\mathbf{T}}\mathbf{T}^{-1}$'in hesaplanması

Hareket eden bir cisim için $\mathbf{T}(t)$'nin türevini alıp ters ile çarpalım:

$$\dot{\mathbf{T}}\mathbf{T}^{-1} = \begin{bmatrix} \dot{\mathbf{R}} & \dot{\mathbf{p}} \\ \mathbf{0}^T & 0 \end{bmatrix} \begin{bmatrix} \mathbf{R}^T & -\mathbf{R}^T\mathbf{p} \\ \mathbf{0}^T & 1 \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{R}}\mathbf{R}^T & \dot{\mathbf{p}} - \dot{\mathbf{R}}\mathbf{R}^T\mathbf{p} \\ \mathbf{0}^T & 0 \end{bmatrix}.$$

Bölüm 2'den $\dot{\mathbf{R}}\mathbf{R}^T = [\boldsymbol{\omega}_s]$. Sağ üst blokta $\mathbf{v}_s \equiv \dot{\mathbf{p}} - \boldsymbol{\omega}_s \times \mathbf{p}$ tanımlayalım. O halde

$$[\mathbf{V}_s] \equiv \dot{\mathbf{T}}\mathbf{T}^{-1} = \begin{bmatrix} [\boldsymbol{\omega}_s] & \mathbf{v}_s \\ \mathbf{0}^T & 0 \end{bmatrix}, \qquad \mathbf{V}_s = \begin{bmatrix} \boldsymbol{\omega}_s \\ \mathbf{v}_s \end{bmatrix} \in \mathbb{R}^6 \text{ (\textbf{uzay twist'i})}.$$

$\mathbf{v}_s$ fiziksel olarak nedir? Cisme rijit bağlı, o an *dünya orijininde* bulunan (hayali) noktanın hızıdır. İspat: cismin bir noktası $\mathbf{r}(t)$ ise $\dot{\mathbf{r}} = \boldsymbol{\omega}_s \times (\mathbf{r} - \mathbf{p}) + \dot{\mathbf{p}}$ (cisim $\mathbf{p}$ etrafında döner ve $\mathbf{p}$ ile birlikte ötelenir). $\mathbf{r} = \mathbf{0}$ koyunca $\dot{\mathbf{r}} = \dot{\mathbf{p}} - \boldsymbol{\omega}_s \times \mathbf{p} = \mathbf{v}_s$. Bu yüzden $\mathbf{v}_s$, ayak ucunun hızı *değildir*; ayak ucunun hızı

$$\dot{\mathbf{p}} = \mathbf{v}_s + \boldsymbol{\omega}_s \times \mathbf{p}.$$

Benzer şekilde $[\mathbf{V}_b] = \mathbf{T}^{-1}\dot{\mathbf{T}} = \begin{bmatrix} \mathbf{R}^T\dot{\mathbf{R}} & \mathbf{R}^T\dot{\mathbf{p}} \\ \mathbf{0}^T & 0 \end{bmatrix}$: **gövde twist'i**; $\boldsymbol{\omega}_b = \mathbf{R}^T\boldsymbol{\omega}_s$ ve $\mathbf{v}_b = \mathbf{R}^T\dot{\mathbf{p}}$ (gövde orijininin hızı, gövde eksenlerinde).

3.2.2 Adjoint: twist'i çerçeve değiştirmek

$[\mathbf{V}_s] = \dot{\mathbf{T}}\mathbf{T}^{-1} = \mathbf{T}(\mathbf{T}^{-1}\dot{\mathbf{T}})\mathbf{T}^{-1} = \mathbf{T}[\mathbf{V}_b]\mathbf{T}^{-1}$. Blokları çarp:

$$\mathbf{T}[\mathbf{V}_b]\mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R}[\boldsymbol{\omega}_b]\mathbf{R}^T & -\mathbf{R}[\boldsymbol{\omega}_b]\mathbf{R}^T\mathbf{p} + \mathbf{R}\mathbf{v}_b \\ \mathbf{0}^T & 0 \end{bmatrix}.$$

$\mathbf{R}[\boldsymbol{\omega}_b]\mathbf{R}^T = [\mathbf{R}\boldsymbol{\omega}_b]$ (dönmüş vektörün çapraz çarpım matrisi) ve $-[\mathbf{R}\boldsymbol{\omega}_b]\mathbf{p} = \mathbf{p} \times \mathbf{R}\boldsymbol{\omega}_b = [\mathbf{p}]\mathbf{R}\boldsymbol{\omega}_b$. Dolayısıyla $\boldsymbol{\omega}_s = \mathbf{R}\boldsymbol{\omega}_b$, $\mathbf{v}_s = [\mathbf{p}]\mathbf{R}\boldsymbol{\omega}_b + \mathbf{R}\mathbf{v}_b$; 6-B matrisle:

$$\mathbf{V}_s = \mathrm{Ad}_{\mathbf{T}}\mathbf{V}_b, \qquad \mathrm{Ad}_{\mathbf{T}} = \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ [\mathbf{p}]\mathbf{R} & \mathbf{R} \end{bmatrix}.$$

Adjoint, Jacobian'ı bir çerçeveden ötekine taşımanın aracıdır (Bölüm 4).

3.2.3 Vida ekseni: bir dönel eklemin twist'i

Eklem ekseni, birim yön $\hat{\boldsymbol{\omega}}$ ve eksen üzerindeki bir nokta $\mathbf{q}_a$ ile tanımlı; eklem $\dot{\theta} = 1$ rad/s ile dönüyor. Eksene bağlı her nokta $\mathbf{r}$'nin hızı $\hat{\boldsymbol{\omega}} \times (\mathbf{r} - \mathbf{q}_a)$. Dünya orijinindeki noktanın hızı ($\mathbf{r} = \mathbf{0}$): $\mathbf{v} = -\hat{\boldsymbol{\omega}} \times \mathbf{q}_a$. O halde birim hızdaki twist

$$\mathbf{S} = \begin{bmatrix} \hat{\boldsymbol{\omega}} \\ -\hat{\boldsymbol{\omega}} \times \mathbf{q}_a \end{bmatrix}$$

ve buna **vida ekseni** denir. Kayar (prizmatik) eklem için $\mathbf{S} = (\mathbf{0}, \hat{\mathbf{v}})$: dönme yok, her nokta $\hat{\mathbf{v}}$ yönünde gider. $\mathbf{q}_a$'nın eksen üzerinde nerede seçildiği önemli değil: $\mathbf{q}_a + t\hat{\boldsymbol{\omega}}$ seçince $\hat{\boldsymbol{\omega}} \times t\hat{\boldsymbol{\omega}} = \mathbf{0}$ ek terimi kaybolur.

3.2.4 Twist'in üsteli: $e^{[\mathbf{S}]\theta}$

Eklem θ kadar dönünce cisim ne kadar hareket eder? $\dot{\mathbf{T}} = [\mathbf{S}]\mathbf{T}$ diferansiyel denkleminin çözümü $\mathbf{T}(\theta) = e^{[\mathbf{S}]\theta}\mathbf{T}(0)$. Matris üstelini hesaplamak için $[\mathbf{S}]$'nin kuvvetlerine bak:

$$[\mathbf{S}] = \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ \mathbf{0}^T & 0 \end{bmatrix}, \qquad [\mathbf{S}]^k = \begin{bmatrix} [\boldsymbol{\omega}]^k & [\boldsymbol{\omega}]^{k-1}\mathbf{v} \\ \mathbf{0}^T & 0 \end{bmatrix} \ (k \geq 1).$$

Seriyi toplarsak sol üst blok $e^{[\boldsymbol{\omega}]\theta}$ (Rodrigues), sağ üst blok

$$\Big(\sum_{k\geq 1} \frac{\theta^k}{k!}[\boldsymbol{\omega}]^{k-1}\Big)\mathbf{v} = \Big(\mathbf{I}\theta + \frac{\theta^2}{2!}[\boldsymbol{\omega}] + \frac{\theta^3}{3!}[\boldsymbol{\omega}]^2 + \ldots\Big)\mathbf{v}.$$

$[\boldsymbol{\omega}]^3 = -[\boldsymbol{\omega}]$ ile tek ve çift kuvvetleri grupla (Bölüm 2'deki gibi):

$$\boxed{e^{[\mathbf{S}]\theta} = \begin{bmatrix} e^{[\boldsymbol{\omega}]\theta} & \mathbf{G}(\theta)\mathbf{v} \\ \mathbf{0}^T & 1 \end{bmatrix}, \quad \mathbf{G}(\theta) = \mathbf{I}\theta + (1-\cos\theta)[\boldsymbol{\omega}] + (\theta - \sin\theta)[\boldsymbol{\omega}]^2}$$

(kodda `exp_se3`; Modern Robotics denk. 3.88.) **Sağlama:** dönel eklem için $\mathbf{v} = -[\boldsymbol{\omega}]\mathbf{q}_a$ koy: $\mathbf{G}\mathbf{v} = -(\theta[\boldsymbol{\omega}] + (1-\cos\theta)[\boldsymbol{\omega}]^2 + (\theta - \sin\theta)[\boldsymbol{\omega}]^3)\mathbf{q}_a = -(\sin\theta[\boldsymbol{\omega}] + (1-\cos\theta)[\boldsymbol{\omega}]^2)\mathbf{q}_a = (\mathbf{I} - \mathbf{R})\mathbf{q}_a$. Yani nokta $\mathbf{r} \to \mathbf{R}\mathbf{r} + (\mathbf{I}-\mathbf{R})\mathbf{q}_a = \mathbf{q}_a + \mathbf{R}(\mathbf{r} - \mathbf{q}_a)$: $\mathbf{q}_a$ **etrafında dönme**. Tam beklediğimiz şey.

3.3 Üstellerin çarpımı (PoE) formülü

3.3.1 Türetme

Robot "ev" konumunda ($\mathbf{q} = \mathbf{0}$) ayak çerçevesi $\mathbf{M}$, eklemlerin ev konumundaki uzay vida eksenleri $\mathbf{S}_1, \ldots, \mathbf{S}_n$. Eklemleri **uçtan tabana doğru** sırayla döndürelim:

1. Sadece n. eklemi q_n kadar döndür. Diğer eklemler ev konumunda olduğu için n. eksen hâlâ $\mathbf{S}_n$'de; sadece son uzuv hareket eder: $\mathbf{T} = e^{[\mathbf{S}_n]q_n}\mathbf{M}$.
2. Şimdi $(n-1)$. eklemi döndür. Bu eklemin ekseni, kendisinden *sonraki* eklemlerin hareketinden etkilenmez (onlar ona bağlı değil), dolayısıyla hâlâ $\mathbf{S}_{n-1}$'de. Dönme, ondan sonraki her şeyi (zaten dönmüş son uzuv dahil) rijit olarak taşır: $\mathbf{T} = e^{[\mathbf{S}_{n-1}]q_{n-1}}e^{[\mathbf{S}_n]q_n}\mathbf{M}$.
3. Tabana kadar devam:

$$\boxed{\mathbf{T}(\mathbf{q}) = e^{[\mathbf{S}_1]q_1}e^{[\mathbf{S}_2]q_2}\ldots e^{[\mathbf{S}_n]q_n}\,\mathbf{M}}$$

Son konum eklemlerin hangi sırayla döndürüldüğüne bağlı değildir, bu yüzden uçtan başlama hilesi genel sonucu verir. Sadece ev konumunu ve eksenleri bilmen yeter; DH parametrelerindeki çerçeve yerleştirme kuralları gerekmez. (DH hâlâ birçok endüstriyel kolun dokümantasyonunda kullanılır; okuyabilmen yeterli. URDF/MJCF dosyaları her eklem için "ebeveyne göre sabit dönüşüm + eksen" verir; bu, elle kurduğumuz zincire karşılık gelir.)

Elle: sagital 2 uzuvlu bacak, PoE ile

Kalça pitch (y ekseni, kalçada), diz pitch (y ekseni, $(0,0,-\ell_1)$'de), ev konumunda bacak dümdüz aşağı: ayak $(0,0,-\ell_1-\ell_2)$.
Eksenler: $\mathbf{S}_1 = (0,1,0,\ 0,0,0)$ ($\mathbf{q}_a = \mathbf{0}$). $\mathbf{S}_2$: $\mathbf{v} = -(0,1,0)\times(0,0,-\ell_1) = -(-\ell_1,0,0) = (\ell_1,0,0)$.

Sağdan sola uygula. $e^{[\mathbf{S}_2]q_2}$ diz noktası etrafında döndürür: ayak $\to$ $(0,0,-\ell_1) + \mathbf{R}_y(q_2)(0,0,-\ell_2) = (-\ell_2 \sin q_2,\ 0,\ -\ell_1 - \ell_2 \cos q_2)$ ($\mathbf{R}_y(\alpha)(0,0,-L) = (-L\sin\alpha, 0, -L\cos\alpha)$). Sonra $e^{[\mathbf{S}_1]q_1}$ orijin etrafında $\mathbf{R}_y(q_1)$:

$$x = -\ell_1 \sin q_1 - \ell_2 \sin(q_1+q_2), \qquad z = -\ell_1 \cos q_1 - \ell_2 \cos(q_1+q_2).$$

($\cos q_1 \sin q_2 + \sin q_1 \cos q_2 = \sin(q_1+q_2)$ toplam formülüyle.) Bölüm 1'deki formülün aynısı, tek fark işaret: burada y etrafında *pozitif* dönme (sağ el kuralı) aşağı bakan bacağı $-x$'e götürüyor, Bölüm 1'de pozitif açıyı $+x$ yönü seçmiştik. Hangi kuralı seçtiğin önemli değil; ama *her dosyada aynı* olmalı ve yazılmalı.

Sayılar: 3-DoF bacak

Kalça roll (x ekseni), kalça pitch (y), diz pitch (y); uyluk = baldır = 0.2 m; ev konumunda bacak dümdüz aşağıda: $\mathbf{M} = (0,0,-0.4)$'e öteleme. Vida eksenleri: $\mathbf{S}_1 = (1,0,0,\ 0,0,0)$, $\mathbf{S}_2 = (0,1,0,\ 0,0,0)$, $\mathbf{S}_3 = (0,1,0,\ 0.2,0,0)$. $\mathbf{q} = (0.2, -0.5, 0.9)$ rad: ayak $(0.0180,\ 0.0715,\ -0.3526)$ m. **Elle sağlama:** önce sagital düzlemde (yukarıdaki formül, $q_1 \to -0.5$, $q_2 \to 0.9$): $x = -0.2\sin(-0.5) - 0.2\sin(0.4) = 0.0959 - 0.0779 = 0.0180$, $z_s = -0.2\cos 0.5 - 0.2\cos 0.4 = -0.3597$. Sonra roll $\mathbf{R}_x(0.2)$ bu noktayı y–z düzleminde döndürür: $y = -\sin(0.2) z_s = 0.0715$, $z = \cos(0.2) z_s = -0.3526$. Tamam. Aynı sonuç "$R_x(q_1)R_y(q_2)\,T_z(-0.2)\,R_y(q_3)\,T_z(-0.2)$" zinciriyle 3×10^{-17} farkla. **İşaret kontrolü:** sadece kalça pitch $= +0.3$ ise ayak $(-0.118,\ 0,\ -0.382)$: $-0.4\sin 0.3 = -0.118$, $-0.4\cos 0.3 = -0.382$. Bir URDF'te bu işaret yanlışsa robotun bacağı ters yöne gider; ilk iş her eklemi tek tek $\pm$ küçük açıyla test etmektir. Gövde $(1, 0.5, 0.45)$'te, RPY $(0.05, -0.1, 0.3)$ ve kalça gövdede $(0.1, -0.08, -0.05)$'te ise dünyada ayak $(1.1471,\ 0.5577,\ 0.0613)$ m ($\mathbf{T}_{wf} = \mathbf{T}_{wb}\mathbf{T}_{bh}\mathbf{T}_{hf}$, iç indisler sadeleşiyor). **Twist kontrolü:** $\dot{\mathbf{q}} = (0.4, -1, 1.5)$ için sonlu farkla $\dot{\mathbf{T}}$'den $\mathbf{V}_s = (0.400,\ 0.490,\ 0.099,\ 0.263,\ -0.029,\ 0.141)$; $\mathrm{Ad}_{\mathbf{T}}\mathbf{V}_b$ ile fark 1.6×10^{-11} (sonlu fark hatası); ayak ucu hızı $(0.0834,\ 0.1142,\ 0.1607)$ m/s $= \mathbf{v}_s + \boldsymbol{\omega}_s \times \mathbf{p}$.

Python

```python
import numpy as np
from robkit import exp_se3, fk_poe, homog, inv_T, rot_x, rot_y, rpy_to_R, skew, adjoint, unskew
np.set_printoptions(precision=5, suppress=True)

# 3-DoF bacak: kalça roll (x ekseni), kalça pitch (y ekseni), diz pitch (y ekseni). Uyluk = baldır = 0.2 m.
l1, l2 = 0.2, 0.2
# Ev (home) konumu: bacak dümdüz aşağıda. Vida eksenleri uzay (kalça) çerçevesinde: S = [ω; v], v = -ω x q
def screw(w, q_point): w = np.array(w, float); return np.concatenate([w, -np.cross(w, q_point)])
Slist = [screw([1, 0, 0], [0, 0, 0]),          # kalça roll ekseni, kalça merkezinden geçer
         screw([0, 1, 0], [0, 0, 0]),          # kalça pitch
         screw([0, 1, 0], [0, 0, -l1])]        # diz pitch, dizden geçer
M = homog(np.eye(3), [0, 0, -l1 - l2])         # ev konumunda ayak çerçevesi
print("vida eksenleri S_i = [ω; v]:"); [print("  ", S) for S in Slist]

def fk_chain(q):
    """Aynı bacak, 'dönme-öteleme' zinciri ile (DH benzeri elle kurulmuş)"""
    T = homog(rot_x(q[0]), np.zeros(3))@homog(rot_y(q[1]), np.zeros(3))
    T = T@homog(np.eye(3), [0, 0, -l1])@homog(rot_y(q[2]), np.zeros(3))@homog(np.eye(3), [0, 0, -l2])
    return T

q = np.array([0.2, -0.5, 0.9])
```

```python
T1, T2 = fk_poe(Slist, M, q), fk_chain(q)
print("ayak konumu (PoE)   :", T1[:3, 3]); print("ayak konumu (zincir):", T2[:3, 3], "  fark",
np.abs(T1 - T2).max())
print("ayak yönelimi (PoE) R =\n", T1[:3, :3])

# İşaret konvansiyonunu sayısal doğrula: pozitif kalça pitch ayağı hangi yöne götürür?
q0 = np.array([0.0, 0.3, 0.0]); p0 = fk_poe(Slist, M, q0)[:3, 3]
print(f"sadece kalça pitch =+0.3: ayak = {p0}  -> y ekseni etrafında + dönme, aşağı bakan bacağı -x
yönüne götürür (sağ el kuralı)")

# Gövde -> kalça -> ayak: dünya çerçevesinde ayak konumu
T_wb = homog(rpy_to_R(0.05, -0.1, 0.3), [1.0, 0.5, 0.45])      # gövde pozu (dünyada)
T_bh = homog(np.eye(3), [0.1, -0.08, -0.05])                   # sağ kalça, gövde çerçevesinde
T_wf = T_wb@T_bh@T1
print("dünyada ayak konumu:", T_wf[:3, 3])
print("ayak -> gövde dönüşümü (ters):", (inv_T(T_wb)@T_wf)[:3, 3], " = T_bh*T_hf:", (T_bh@T1)[:3, 3])

# Twist ve adjoint: dq/dt = qd iken ayak çerçevesinin hızı. Uzay twist'i V_s: [V_s] = Ṫ T^{-1}; gövde
twist'i V_b: [V_b] = T^{-1} Ṫ
qd = np.array([0.4, -1.0, 1.5]); h = 1e-6
Td = (fk_poe(Slist, M, q + h*qd) - fk_poe(Slist, M, q - h*qd))/(2*h)          # Ṫ sonlu farkla
Vs_m = Td@inv_T(T1); Vb_m = inv_T(T1)@Td
Vs = np.concatenate([unskew(Vs_m[:3, :3]), Vs_m[:3, 3]]); Vb = np.concatenate([unskew(Vb_m[:3, :3]),
Vb_m[:3, 3]])
print("uzay twist V_s =", Vs); print("gövde twist V_b =", Vb)
print("Ad_T V_b =", adjoint(T1)@Vb, " (V_s'ye eşit olmalı), fark", np.abs(adjoint(T1)@Vb - Vs).max())
pdot = Td[:3, 3]
print("ayak ucunun doğrusal hızı ṗ =", pdot, " = v_s + ω_s x p:", Vs[3:] + np.cross(Vs[:3], T1[:3, 3]))
```

Çıktı:

```
vida eksenleri S_i = [ω; v]:
   [ 1.  0.  0. -0. -0. -0.]
   [ 0.  1.  0. -0. -0. -0.]
   [ 0.   1.   0.   0.2 -0.  -0. ]
ayak konumu (PoE)   : [ 0.018    0.07147 -0.35256]
ayak konumu (zincir): [ 0.018    0.07147 -0.35256]   fark 2.7755575615628914e-17
ayak yönelimi (PoE) R =
 [[ 0.92106  0.       0.38942]
 [ 0.07737  0.98007 -0.18299]
 [-0.38166  0.19867  0.9027 ]]
sadece kalça pitch =+0.3: ayak = [-0.11821  0.      -0.38213]  -> y ekseni etrafında + dönme, aşağı bakan bacağı -x
yönüne götürür (sağ el kuralı)
dünyada ayak konumu: [1.14713 0.55765 0.06131]
ayak -> gövde dönüşümü (ters): [ 0.118   -0.00853 -0.40256]  = T_bh*T_hf: [ 0.118   -0.00853 -0.40256]
uzay twist V_s = [ 0.4      0.49003  0.09933  0.26327 -0.02857  0.14096]
gövde twist V_b = [0.36842 0.5     0.15577 0.02432 0.14389 0.15667]
Ad_T V_b = [ 0.4      0.49003  0.09933  0.26327 -0.02857  0.14096]  (V_s'ye eşit olmalı), fark 1.5590972957113536e-11
ayak ucunun doğrusal hızı ṗ = [0.08341 0.11424 0.16073]  = v_s + ω_s x p: [0.08341 0.11424 0.16073]
```

C++

```cpp
// c03_fk.cpp — PoE ile ileri kinematik, çerçeve zinciri ve twist/adjoint kontrolü
#include "robkit.hpp"
#include <cstdio>
using namespace rk;

Vec6 screw(const Vec3& w, const Vec3& q) { Vec6 S; S << w, -w.cross(q); return S; }

int main() {
```

```cpp
    const double l1 = 0.2, l2 = 0.2;
    std::vector<Vec6> S = {screw(Vec3::UnitX(), Vec3::Zero()), screw(Vec3::UnitY(), Vec3::Zero()),
                           screw(Vec3::UnitY(), Vec3(0, 0, -l1))};
    Mat4 M = homog(Mat3::Identity(), Vec3(0, 0, -l1 - l2));
    Eigen::Vector3d q(0.2, -0.5, 0.9);
    Mat4 T = fk_poe(S, M, q);
    // Zincir ile: Rx(q1) Ry(q2) Tz(-l1) Ry(q3) Tz(-l2)
    Mat4 Tc = homog(rot_x(q(0)), Vec3::Zero()) * homog(rot_y(q(1)), Vec3::Zero()) *
              homog(Mat3::Identity(), Vec3(0, 0, -l1)) * homog(rot_y(q(2)), Vec3::Zero()) *
              homog(Mat3::Identity(), Vec3(0, 0, -l2));
    Vec3 p = T.topRightCorner<3, 1>();
    std::printf("ayak konumu (PoE): [%.5f %.5f %.5f], zincirle fark %.1e\n", p(0), p(1), p(2), (T -
Tc).cwiseAbs().maxCoeff());

    Mat4 Twb = homog(rpy_to_R(0.05, -0.1, 0.3), Vec3(1.0, 0.5, 0.45));
    Mat4 Tbh = homog(Mat3::Identity(), Vec3(0.1, -0.08, -0.05));
    Vec3 pw = (Twb * Tbh * T).topRightCorner<3, 1>();
    std::printf("dünyada ayak konumu: [%.5f %.5f %.5f]\n", pw(0), pw(1), pw(2));

    // Twist: sonlu farkla Ṫ, sonra V_s = Ṫ T^-1, V_b = T^-1 Ṫ, kontrol: Ad_T V_b = V_s
    Eigen::Vector3d qd(0.4, -1.0, 1.5); double h = 1e-6;
    Mat4 Td = (fk_poe(S, M, q + h * qd) - fk_poe(S, M, q - h * qd)) / (2 * h);
    Mat4 Vs_m = Td * inv_T(T), Vb_m = inv_T(T) * Td;
    Vec6 Vs, Vb;
    Vs << unskew(Vs_m.topLeftCorner<3, 3>()), Vs_m.topRightCorner<3, 1>();
    Vb << unskew(Vb_m.topLeftCorner<3, 3>()), Vb_m.topRightCorner<3, 1>();
    std::printf("V_s = [%.5f %.5f %.5f %.5f %.5f %.5f]\n", Vs(0), Vs(1), Vs(2), Vs(3), Vs(4), Vs(5));
    std::printf("|Ad_T V_b - V_s| = %.1e\n", (adjoint(T) * Vb - Vs).cwiseAbs().maxCoeff());
    return 0;
}
```

Çıktı:

```
ayak konumu (PoE): [0.01800 0.07147 -0.35256], zincirle fark 2.8e-17
dünyada ayak konumu: [1.14713 0.55765 0.06131]
V_s = [0.40000 0.49003 0.09933 0.26327 -0.02857 0.14096]
|Ad_T V_b - V_s| = 1.4e-11
```

Python ve C++ notu: blok erişimi ve test alışkanlığı

- **Blok erişimi:** numpy `T[:3, :3]`, `T[:3, 3]` ↔ Eigen `T.topLeftCorner<3,3>()`, `T.topRightCorner<3,1>()` (veya `T.block<3,3>(0,0)`). Şablon argümanlı (`<3,3>`) sürümler boyutu derleme anında bilir, daha hızlıdır.
- **Virgülle başlatma:** `Vec6 S; S << w, -w.cross(q);` iki 3-vektörü birleştirir (numpy `np.concatenate`): bu satır, vida ekseni formülü $\mathbf{S} = (\hat{\boldsymbol{\omega}}, -\hat{\boldsymbol{\omega}} \times \mathbf{q}_a)$'nın birebir kodu.
- **İki bağımsız yolla hesapla:** PoE ile zinciri karşılaştırmak, Ad ile sonlu farkı karşılaştırmak: kinematik kodunun en ucuz ve en etkili testi. Cilt 2'de bunları otomatik birim testine çevirdik.
- **Merkezi sonlu fark:** $(f(x+h) - f(x-h))/2h$. Neden daha iyi? Taylor: $f(x \pm h) = f \pm hf' + \frac{h^2}{2}f'' \pm \frac{h^3}{6}f'''$; farkta çift terimler sadeleşir, hata $\frac{h^2}{6}f'''$ $(\mathcal{O}(h^2))$. İleri farkta hata $\frac{h}{2}f''$ $(\mathcal{O}(h))$. h çok küçükse yuvarlama hatası $(\approx \epsilon/h)$ büyür: $h \approx 10^{-6}$ iyi bir varsayılan.

Alıştırma

(Kâğıt) (a) $\mathbf{T} = \begin{bmatrix} \mathbf{R}_z(90^\circ) & (1,0,0) \\ \mathbf{0} & 1 \end{bmatrix}$ için $\mathbf{T}^{-1}$'i formülle yaz ve $\mathbf{T}\mathbf{T}^{-1} = \mathbf{I}$'yi kontrol et. **(b)** $\mathbf{q}_a = (0,0,-0.2)$'den geçen x ekseni etrafındaki dönel eklemin vida eksenini yaz. **(c)** Yukarıdaki 2 uzuvlu bacak için $q_1 = 0.3$, $q_2 = 0.6$'da ayağı PoE ile adım adım hesapla. **(d)** Bir cisim kendi orijini etrafında, dünya z ekseniyle paralel bir eksende 1 rad/s dönüyor; orijini sabit ve $(1,0,0)$'da. $\mathbf{v}_s$ ve $\mathbf{v}_b$ nedir? (İpucu: $\dot{\mathbf{p}} = \mathbf{0}$; dünya orijinindeki hayali noktanın hızı. Cevap: $\mathbf{v}_s = (0,-1,0)$, $\mathbf{v}_b = \mathbf{0}$.) **(e)** $\mathbf{G}(\theta)$'nın $\theta \to 0$'da $\mathbf{I}\theta$'ya gittiğini göster: küçük dönmede öteleme $\approx \mathbf{v}\theta$. **(Bilgisayar) (f)** Bacağa ayak bileği pitch eklemi ekle (ayak uzunluğu 0.05 m, $\mathbf{M}$ ve $\mathbf{S}_4$'ü yaz); FK'yı iki yolla doğrula. **(g)** MuJoCo Menagerie'deki `robotis_op3` modelinin MJCF dosyasından bir bacağın eklem eksenlerini ve ofsetlerini oku; PoE için $\mathbf{S}_i$ ve $\mathbf{M}$'yi çıkar, MuJoCo'nun hesapladığı ayak konumuyla karşılaştır (`data.xpos`).

Bölüm 4

Jacobian, Tekillikler ve Statik

Jacobian robotikte en çok kullanılan tek matristir: hızları (ayak hızı ↔ eklem hızı), kuvvetleri (temas kuvveti ↔ eklem torku) ve küçük yer değiştirmeleri (IK, Bölüm 5) birbirine bağlar. WBC'deki her görev bir Jacobian satırıdır. Bölüm 1'de düzlemsel bacağın Jacobian'ını doğrudan türev alarak bulduk; burada genel (3-B, her robot için geçerli) yolu PoE'den türetiyoruz. Modern Robotics karşılığı: Bölüm 5.

4.1 Uzay Jacobian'ı

4.1.1 PoE'den türetme

$\mathbf{V}_s = \mathbf{J}_s(\mathbf{q})\dot{\mathbf{q}}$ olacak şekilde $6\times n$ bir $\mathbf{J}_s$ arıyoruz. Kısaltma: $\mathbf{E}_i = e^{[\mathbf{S}_i]q_i}$, böylece $\mathbf{T} = \mathbf{E}_1\mathbf{E}_2\cdots\mathbf{E}_n\mathbf{M}$. $\frac{d}{dt}\mathbf{E}_i = [\mathbf{S}_i]\mathbf{E}_i\dot{q}_i$ (üstelin türevi). Çarpım kuralı:

$$\dot{\mathbf{T}} = [\mathbf{S}_1]\mathbf{E}_1\mathbf{E}_2\cdots\mathbf{M}\,\dot{q}_1 + \mathbf{E}_1[\mathbf{S}_2]\mathbf{E}_2\cdots\mathbf{M}\,\dot{q}_2 + \cdots + \mathbf{E}_1\cdots\mathbf{E}_{n-1}[\mathbf{S}_n]\mathbf{E}_n\mathbf{M}\,\dot{q}_n.$$

Sağdan $\mathbf{T}^{-1} = \mathbf{M}^{-1}\mathbf{E}_n^{-1}\cdots\mathbf{E}_1^{-1}$ ile çarp. İlk terimde her şey sadeleşir: $[\mathbf{S}_1]\dot{q}_1$. İkinci terimde $\mathbf{E}_2\cdots\mathbf{M}$ ile tersi sadeleşir, geriye $\mathbf{E}_1[\mathbf{S}_2]\mathbf{E}_1^{-1}\dot{q}_2$ kalır. Genel olarak i. terim:

$$\mathbf{E}_1\cdots\mathbf{E}_{i-1}\,[\mathbf{S}_i]\,(\mathbf{E}_1\cdots\mathbf{E}_{i-1})^{-1}\dot{q}_i = [\mathrm{Ad}_{\mathbf{E}_1\cdots\mathbf{E}_{i-1}}\mathbf{S}_i]\,\dot{q}_i$$

(Bölüm 3'te gösterdiğimiz $\mathbf{T}[\mathbf{V}]\mathbf{T}^{-1} = [\mathrm{Ad}_{\mathbf{T}}\mathbf{V}]$ özdeşliği). Toplayınca:

$$\boxed{\mathbf{J}_{s,i} = \mathrm{Ad}_{e^{[\mathbf{S}_1]q_1}\cdots e^{[\mathbf{S}_{i-1}]q_{i-1}}}\mathbf{S}_i}$$

Yorum: i. sütun, i. eklemin vida ekseninin *şu anki* konumu. Kendisinden önceki eklemler döndükçe eksen de yer değiştirir (Ad onu taşır); sonraki eklemler onu etkilemez. İlk sütun hep $\mathbf{S}_1$.

4.1.2 Ayak ucunun doğrusal hızı

Bölüm 3: ayak noktasının hızı $\dot{\mathbf{p}} = \mathbf{v}_s + \boldsymbol{\omega}_s \times \mathbf{p}$. $\mathbf{J}_s$'yi $[\mathbf{J}_\omega;\ \mathbf{J}_v]$ diye ikiye bölersek $\boldsymbol{\omega}_s = \mathbf{J}_\omega\dot{\mathbf{q}}$, $\mathbf{v}_s = \mathbf{J}_v\dot{\mathbf{q}}$ ve

$$\dot{\mathbf{p}} = \mathbf{J}_v\dot{\mathbf{q}} - \mathbf{p}\times\mathbf{J}_\omega\dot{\mathbf{q}} = (\mathbf{J}_v - [\mathbf{p}]\mathbf{J}_\omega)\dot{\mathbf{q}} \equiv \mathbf{J}_p\dot{\mathbf{q}}.$$

Geometrik biçim (dönel eklemler): i. eklemin şu anki ekseni yönü $\hat{\boldsymbol{\omega}}_i$, üzerindeki bir nokta $\mathbf{q}_i$ ise $\mathbf{J}_{s,i} = (\hat{\boldsymbol{\omega}}_i,\ -\hat{\boldsymbol{\omega}}_i\times\mathbf{q}_i)$ ve

$$\mathbf{J}_{p,i} = -\hat{\boldsymbol{\omega}}_i\times\mathbf{q}_i + \hat{\boldsymbol{\omega}}_i\times\mathbf{p} = \hat{\boldsymbol{\omega}}_i\times(\mathbf{p}-\mathbf{q}_i).$$

Bu, lise fiziğindeki "dönmede hız $= \boldsymbol{\omega}\times\mathbf{r}$" formülüdür: eklem i birim hızla dönünce ayak, eksenden ayağa giden vektöre dik hareket eder. Elle Jacobian kurmanın en hızlı yolu.

Gövde Jacobian'ı $\mathbf{J}_b = \mathrm{Ad}_{\mathbf{T}^{-1}}\mathbf{J}_s$ ise aynı twist'i ayak çerçevesinde ifade eder; kodda hangi çerçevede olduğunu *her zaman* yorum olarak yaz.

Elle: sagital 2 uzuvlu bacak, geometrik Jacobian

Bölüm 3'teki bacak, $q_1 = 0$ (kalça), $q_2 = 90^\circ$ (diz). Konumlar: kalça $\mathbf{0}$, diz $(0, 0, -0.2)$, ayak $\mathbf{p} = (-0.2,\ 0,\ -0.2)$. Eksenler $\hat{\boldsymbol{\omega}} = (0, 1, 0)$.
Sütun 1: $\hat{\boldsymbol{\omega}} \times (\mathbf{p} - \mathbf{0}) = (0, 1, 0) \times (-0.2, 0, -0.2)$. Çapraz çarpım: $(1 \cdot (-0.2) - 0 \cdot 0,\ 0 \cdot (-0.2) - 0 \cdot (-0.2),\ 0 \cdot 0 - 1 \cdot (-0.2)) = (-0.2,\ 0,\ 0.2)$.
Sütun 2: $(0, 1, 0) \times (\mathbf{p} - \text{diz}) = (0, 1, 0) \times (-0.2, 0, 0) = (0,\ 0,\ 0.2)$.
(x, z) satırları: $\mathbf{J} = \begin{bmatrix} -0.2 & 0 \\ 0.2 & 0.2 \end{bmatrix}$. Bölüm 1'dekiyle aynı (sadece x işareti, yön kuralı farkı yüzünden ters). $\det = -0.04$, $|\det| = \ell_1 \ell_2 \sin q_2$.

4.2 Tekillikler

$\mathbf{J}_p$ kare (3 eklem, 3 boyutlu ayak konumu) ve $\det \mathbf{J}_p = 0$ ise tekil. Geometrik biçimden neden olduğunu görmek kolay:

- **Diz düz (sınır tekilliği):** kalça pitch ve diz pitch sütunları $\hat{\boldsymbol{\omega}} \times (\mathbf{p} - \mathbf{q}_i)$; $\mathbf{p}$, kalça ve diz aynı doğru üzerindeyse ikisi *aynı yönde* vektörler olur (sadece boyları farklı). Rank düşer: bacak boyu yönünde ayak hareket edemez. Bölüm 1'deki $\ell_1 \ell_2 \sin q_2$ bunun düzlemsel hâli.
- **Eksen hizalanması (iç tekillik):** iki eklemin ekseni aynı doğruya düşerse sütunları aynı olur. Örnek: kalça roll ekseni ile (ayak bileği roll varsa) bilek roll ekseni bacak dümdüz ve dik iken hizalanabilir.

Tekillik yakınında ters hız patlar (Bölüm 1), ama *kuvvet* taşıma kapasitesi artar (aşağıda): düz bacak ağırlığı neredeyse torksuz taşır.

Manipülabilite ölçüsü $w = \sqrt{\det(\mathbf{J}\mathbf{J}^T)} = \prod \sigma_i$: hız elipsinin hacmiyle orantılı; sıfıra yaklaşması tekilliğe yaklaşmak demek. IK ve yörünge planlamada "tekillikten uzak dur" amacı için kullanılır.

4.3 Statik: $\boldsymbol{\tau} = \mathbf{J}^T \mathbf{F}$

4.3.1 Sanal iş ile türetme

Kütlesiz bir bacak, ayağa dışarıdan $\mathbf{F}_{ext}$ etki ediyor, eklemlerde $\boldsymbol{\tau}$ torkları var, sistem dengede. Dengedeki bir sistemde, kısıtlara uygun herhangi bir *sanal* (hayali, sonsuz küçük) yer değiştirmede tüm kuvvetlerin yaptığı toplam iş sıfırdır (aksi halde sistem o yöne kendiliğinden giderdi). Eklemler $\delta\mathbf{q}$ kadar oynasın; ayak $\delta\mathbf{p} = \mathbf{J}_p \delta\mathbf{q}$ kadar gider:

$$\boldsymbol{\tau}^T \delta\mathbf{q} + \mathbf{F}_{ext}^T \delta\mathbf{p} = \boldsymbol{\tau}^T \delta\mathbf{q} + \mathbf{F}_{ext}^T \mathbf{J}_p \delta\mathbf{q} = (\boldsymbol{\tau} + \mathbf{J}_p^T \mathbf{F}_{ext})^T \delta\mathbf{q} = 0 \quad \forall \delta\mathbf{q}.$$

Her $\delta\mathbf{q}$ için sıfırsa parantez sıfırdır:

$$\boxed{\boldsymbol{\tau} = -\mathbf{J}_p^T \mathbf{F}_{ext}} \qquad \text{(robotun yere uyguladığı } \mathbf{F} = -\mathbf{F}_{ext} \text{ ile } \boldsymbol{\tau} = \mathbf{J}_p^T \mathbf{F}\text{)}.$$

Aynı sonucu güç dengesinden de alırsın: eklemlerin verdiği güç $\boldsymbol{\tau}^T \dot{\mathbf{q}}$, ayağın dışarıya verdiği güç $\mathbf{F}^T \dot{\mathbf{p}} = \mathbf{F}^T \mathbf{J}_p \dot{\mathbf{q}}$; kütlesiz sistemde ikisi eşit.

Doğrudan moment ile sağlama: dönel eklem i'nin torku, ayaktaki kuvvetin o eklemin eksenine göre momentidir: $\tau_i = \hat{\boldsymbol{\omega}}_i \cdot ((\mathbf{p} - \mathbf{q}_i) \times \mathbf{F})$. Bunu üçlü çarpım özdeşliğiyle $(\hat{\boldsymbol{\omega}}_i \times (\mathbf{p} - \mathbf{q}_i)) \cdot \mathbf{F} = \mathbf{J}_{p,i}^T \mathbf{F}$ olarak yazınca $\mathbf{J}^T$ formülünün i. satırını elde edersin. Yani $\mathbf{J}^T \mathbf{F}$ "her eklem için moment kolu çarpı kuvvet"ten başka bir şey değil.

4.3.2 Kuvvet elipsi hız elipsinin tersidir

Birim büyüklükte torklar ($\|\boldsymbol{\tau}\| = 1$) hangi ayak kuvvetlerini üretir? $\mathbf{J}$ kare ve tersinirse $\mathbf{F} = \mathbf{J}^{-T}\boldsymbol{\tau}$. SVD ile $\mathbf{J}^T = \mathbf{V}\boldsymbol{\Sigma}\mathbf{U}^T$, $\mathbf{J}^{-T} = \mathbf{U}\boldsymbol{\Sigma}^{-1}\mathbf{V}^T$: elipsin yarı eksenleri aynı $\mathbf{u}_i$ yönlerinde ama boyları $1/\sigma_i$. **Hız kazancı küçük olan yönde kuvvet kazancı büyük.** Düz bacakta bacak boyu yönündeki $\sigma \to 0$: o yönde ayak hiç hızlanamaz ama aynı yöndeki kuvveti (ağırlığı) neredeyse sıfır torkla taşır. Bisiklet vitesi gibi: hız ile kuvvet arasındaki ödünleşim.

Elle: ayakta durma torku formülü

Ayak kalçanın tam altında (kalça pitch $= -q_d/2$, diz $= q_d$), yer ayağı F kuvvetiyle yukarı itiyor. Uyluk ve baldır, dikeyle $q_d/2$ açı yapan ikizkenar bir üçgenin iki kenarı. Diz, kalça–ayak dikey doğrusundan yatayda $\ell_2 \sin(q_d/2)$ uzakta. Kuvvet dikey olduğundan diz torku = $F\times$ yatay moment kolu:

$$|\tau_{\text{diz}}| = F\,\ell_2 \sin(q_d/2), \qquad \tau_{\text{kalça}} = 0$$

(kuvvet doğrusu kalçadan geçiyor). $q_d = 1$ rad, $F = 9.81$ N: $9.81 \cdot 0.2 \cdot \sin 0.5 = 0.941$ N·m. Aşağıdaki tabloyla aynı.

Sayılar: 3-DoF bacakta Jacobian ve ayakta durma torku

$\mathbf{q} = (0.2, -0.5, 0.9)$: $\mathbf{J}_p = \begin{bmatrix} 0 & -0.3597 & -0.1842 \\ 0.3526 & 0.0036 & -0.0155 \\ 0.0715 & -0.0176 & 0.0763 \end{bmatrix}$, sonlu farkla fark 4×10^{-11}.

İlk sütun sağlaması: kalça roll ekseni $(1,0,0)$, orijinden geçiyor; $\hat{\boldsymbol{\omega}} \times \mathbf{p} = (1,0,0) \times (0.0180,\ 0.0715,\ -0.3526) = (0,\ 0.3526,\ 0.0715)$. Diz açısı $1.2 \to 0.05 \to 0$ rad: $\sigma_{\min} = 0.100 \to 0.0045 \to 0$, $\kappa = 3.7 \to 100 \to \infty$. **Statik** (2 kg robot, iki ayakta, her ayağa 9.81 N; kalça pitch $= -$diz$/2$ ile ayak tam kalçanın altında):

Diz [rad]	Ayak z [m]	$\lvert\tau_{diz}\rvert$ [N·m]	[kg·cm]
0.2	−0.398	0.196	2.0
0.6	−0.382	0.580	5.9
1.0	−0.351	0.941	9.6
1.4	−0.306	1.264	12.9

Ayak $z = -2\ell\cos(q_d/2)$: $q_d = 1$'de $-0.4\cos 0.5 = -0.351$. Sanal iş: $\boldsymbol{\tau}^T\dot{\mathbf{q}} = -F_{ext}^T\dot{\mathbf{p}} = -1.155169$ W (iki taraf birebir). **Donanım dersi:** MG996R'ın datasheet'teki durma torku yaklaşık 9–11 kg·cm. 2 kg'lık bir biped, dizini 1 rad büktüğünde *sadece ayakta durmak için* bu sınıra ulaşıyor; tek ayakta (yürürken) tork iki katına çıkar. Bu servolarla diz bükük duran 2 kg'lık bir robot gerçekçi değil (Cilt 3 ve Cilt 6 Bölüm 4'te aynı sonuca daha ayrıntılı vardık).

Python

```python
import numpy as np
from robkit import fk_poe, space_jacobian, homog, skew
np.set_printoptions(precision=5, suppress=True)

l1, l2 = 0.2, 0.2
def screw(w, qp): w = np.array(w, float); return np.concatenate([w, -np.cross(w, qp)])
Slist = [screw([1, 0, 0], [0, 0, 0]), screw([0, 1, 0], [0, 0, 0]), screw([0, 1, 0], [0, 0, -l1])]
M = homog(np.eye(3), [0, 0, -l1 - l2])
foot = lambda q: fk_poe(Slist, M, q)[:3, 3]

def foot_jacobian(q):
```

```python
    """Ayak ucu doğrusal hız Jacobian'ı (3x3): ṗ = (J_v - [p] J_ω) q̇  (uzay Jacobian'ından)"""
    Js = space_jacobian(Slist, q); p = foot(q)
    return Js[3:] - skew(p)@Js[:3]

q = np.array([0.2, -0.5, 0.9])
Js = space_jacobian(Slist, q); Jp = foot_jacobian(q)
print("uzay Jacobian'ı J_s (6x3, satırlar [ω; v]):\n", Js)
print("ayak doğrusal Jacobian'ı J_p:\n", Jp)
h = 1e-6; Jfd = np.column_stack([(foot(q + h*e) - foot(q - h*e))/(2*h) for e in np.eye(3)])
print("sonlu fark ile karşılaştırma: max fark =", np.abs(Jfd - Jp).max())

print("\ntekillik: diz açısı -> det(J_p), koşul sayısı")
for qk in [1.2, 0.6, 0.2, 0.05, 0.0]:
    Jq = foot_jacobian(np.array([0.0, -qk/2, qk])); s = np.linalg.svd(Jq, compute_uv=False)
    print(f"  diz ={qk:4.2f}: det = {np.linalg.det(Jq):+.5f}, σ = {s.round(4)}, κ = {s[0]/max(s[-1],
    1e-300):.3g}")

# Statik: 2 kg robot iki ayak üzerinde, gövde yatay. Her ayağa yerden F = (0, 0, m g / 2) etki eder.
m, g = 2.0, 9.81; F_ext = np.array([0, 0, m*g/2])
print(f"\nstatik eklem torkları (τ = -J_pᵀ F_ext, F_ext = yerden ayağa {F_ext[2]:.3f} N yukarı), ayak
kalçanın altında tutulursa:")
for qk in [0.2, 0.6, 1.0, 1.4]:
    qq = np.array([0.0, -qk/2, qk])                    # kalça pitch = -diz/2 -> ayak tam kalçanın
    altında
    tau = -foot_jacobian(qq).T@F_ext; p = foot(qq)
    print(f"  diz ={qk:3.1f} rad: ayak {p.round(4)}, τ = {tau.round(4)} N·m  (|τ_diz| =
    {abs(tau[2]):.3f} N·m = {abs(tau[2])/9.81*100:.1f} kg·cm)")
# Sanal iş kontrolü: eklem gücü = -ayaktaki dış kuvvetin gücü
qd = np.array([0.3, -0.7, 1.1]); tau = -Jp.T@F_ext
print(f"sanal iş: τᵀq̇ = {tau@qd:.6f},  -F_extᵀ ṗ = {-F_ext@(Jp@qd):.6f}")
```

Çıktı:

```
uzay Jacobian'ı J_s (6x3, satırlar [ω; v]):
 [[ 1.       0.       0.     ]
 [ 0.       0.98007  0.98007]
 [ 0.       0.19867  0.19867]
 [ 0.       0.       0.17552]
 [ 0.       0.      -0.01905]
 [ 0.       0.       0.09397]]
ayak doğrusal Jacobian'ı J_p:
 [[ 0.      -0.35973 -0.18421]
 [ 0.35256  0.00358 -0.01547]
 [ 0.07147 -0.01764  0.07633]]
sonlu fark ile karşılaştırma: max fark = 4.155764274371698e-11

tekillik: diz açısı -> det(J_p), koşul sayısı
  diz =1.20: det = +0.01231, σ = [0.3728 0.3301 0.1   ], κ = 3.73
  diz =0.60: det = +0.00863, σ = [0.4281 0.3821 0.0528], κ = 8.11
  diz =0.20: det = +0.00316, σ = [0.4451 0.398  0.0179], κ = 24.9
  diz =0.05: det = +0.00080, σ = [0.4471 0.3999 0.0045], κ = 100
  diz =0.00: det = +0.00000, σ = [0.4472 0.4    0.    ], κ = 4.47e+299

statik eklem torkları (τ = -J_pᵀ F_ext, F_ext = yerden ayağa 9.810 N yukarı), ayak kalçanın altında tutulursa:
  diz =0.2 rad: ayak [ 0.     0.    -0.398], τ = [ 0.      0.     -0.1959] N·m  (|τ_diz| = 0.196 N·m = 2.0 kg·cm)
  diz =0.6 rad: ayak [ 0.      0.     -0.3821], τ = [ 0.      0.     -0.5798] N·m  (|τ_diz| = 0.580 N·m = 5.9 kg·cm)
  diz =1.0 rad: ayak [-0.     0.    -0.351], τ = [ 0.     -0.     -0.9406] N·m  (|τ_diz| = 0.941 N·m = 9.6 kg·cm)
  diz =1.4 rad: ayak [ 0.      0.     -0.3059], τ = [ 0.     0.    -1.264] N·m  (|τ_diz| = 1.264 N·m = 12.9 kg·cm)
sanal iş: τᵀq̇ = -1.155169,  -F_extᵀ ṗ = -1.155169
```

C++

```cpp
// c04_jac.cpp — ayak Jacobian'ı, sonlu fark kontrolü, statik eklem torkları
#include "robkit.hpp"
#include <cstdio>
using namespace rk;
using Eigen::Vector3d;

const double l1 = 0.2, l2 = 0.2;
Vec6 screw(const Vec3& w, const Vec3& q) { Vec6 S; S << w, -w.cross(q); return S; }
const std::vector<Vec6> S = {screw(Vec3::UnitX(), Vec3::Zero()), screw(Vec3::UnitY(), Vec3::Zero()),
                             screw(Vec3::UnitY(), Vec3(0, 0, -l1))};
const Mat4 M = homog(Mat3::Identity(), Vec3(0, 0, -l1 - l2));

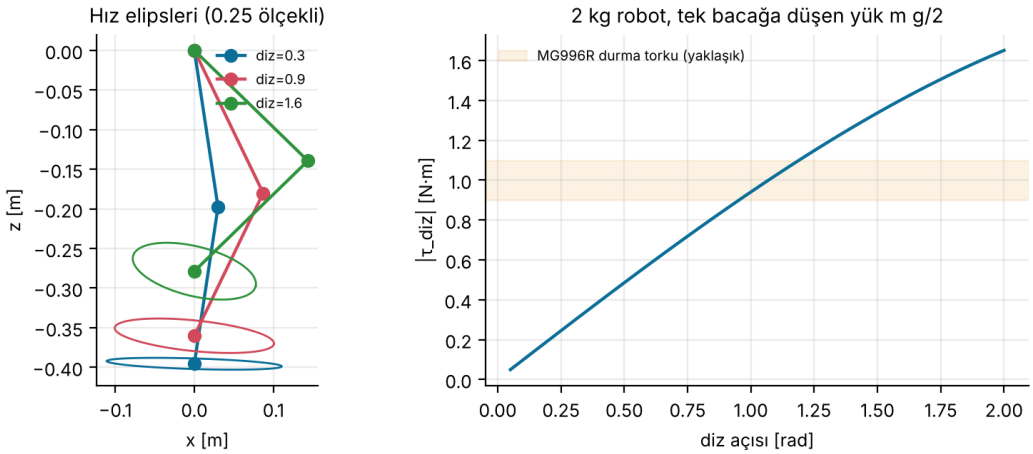
Vec3 foot(const Vector3d& q) { return fk_poe(S, M, q).topRightCorner<3, 1>(); }
Mat3 foot_jacobian(const Vector3d& q) {
    Jac Js = space_jacobian(S, q);
    return Js.bottomRows<3>() - skew(foot(q)) * Js.topRows<3>();     // J_v - [p] J_ω
}

int main() {
    Vector3d q(0.2, -0.5, 0.9);
    Mat3 Jp = foot_jacobian(q), Jfd;
    const double h = 1e-6;
    for (int i = 0; i < 3; ++i) {
        Vector3d e = Vector3d::Unit(i);
        Jfd.col(i) = (foot(q + h * e) - foot(q - h * e)) / (2 * h);
    }
    std::printf("J_p satır 1: [%.5f %.5f %.5f]; sonlu farkla max fark %.1e\n", Jp(0, 0), Jp(0, 1),
Jp(0, 2),
                (Jp - Jfd).cwiseAbs().maxCoeff());

    const double m = 2.0, g = 9.81; Vec3 F(0, 0, m * g / 2);
    for (double qk : {0.2, 0.6, 1.0, 1.4}) {
        Vector3d qq(0.0, -qk / 2, qk);
        Vec3 tau = -foot_jacobian(qq).transpose() * F;
        std::printf("diz =%.1f rad: τ = [%.4f %.4f %.4f] N·m, |τ_diz| = %.1f kg·cm\n", qk, tau(0),
tau(1), tau(2),
                    std::abs(tau(2)) / 9.81 * 100);
    }
    return 0;
}
```

Çıktı:

```
J_p satır 1: [0.00000 -0.35973 -0.18421]; sonlu farkla max fark 4.2e-11
diz =0.2 rad: τ = [-0.0000 0.0000 -0.1959] N·m, |τ_diz| = 2.0 kg·cm
diz =0.6 rad: τ = [-0.0000 -0.0000 -0.5798] N·m, |τ_diz| = 5.9 kg·cm
diz =1.0 rad: τ = [-0.0000 -0.0000 -0.9406] N·m, |τ_diz| = 9.6 kg·cm
diz =1.4 rad: τ = [-0.0000 0.0000 -1.2640] N·m, |τ_diz| = 12.9 kg·cm
```



Şekil 4.1: Sol: sagital düzlemde üç duruş ve ayak ucundaki hız elipsleri (kalça pitch + diz). Diz düzleştikçe elips dikey yönde incelir (tekillik). Sağ: diz açısına göre ayakta durmak için gereken diz torku; turuncu bant MG996R'ın yaklaşık durma torku.

Python ve C++ notu: satır/sütun seçimi

- numpy `Js[3:]` (son 3 satır), `Js[:3]` ↔ Eigen `Js.bottomRows<3>()`, `Js.topRows<3>()`. Sütun: `J[:, i]` ↔ `J.col(i)`. Koddaki `Js[3:] - skew(p)@Js[:3]` satırı tam olarak $\mathbf{J}_v - [\mathbf{p}]\mathbf{J}_\omega$.
- **Fantezi indeksleme:** `J[[0, 2]][:, 1:]` önce 0. ve 2. satırları, sonra 1. sütundan sonrasını alır (sagital 2×2 blok). Eigen'de `J({0,2}, Eigen::seq(1,2))` (Eigen 3.4+) veya elle kopyalama.
- **`const` global veri:** C++'ta `const std::vector<Vec6> S = {...};` gibi sabit robot parametreleri dosya kapsamında tanımlanabilir; Python'da modül seviyesindeki değişkenler aynı rolü oynar. Gerçek kodda bunlar bir `struct LegParams` içinde toplanır (Cilt 2).
- **Birim dönüşümü:** 1 kg·cm $= 9.81 \times 10^{-2}$ N·m. Hobi servo datasheet'leri kg·cm verir; hesapları SI'da yap, sadece raporlarken çevir.

Alıştırma

(Kâğıt) (a) Sagital bacakta $q_1 = 0.3$, $q_2 = 0.8$ için geometrik Jacobian'ı çapraz çarpımla kur; det'i $\ell_1 \ell_2 \sin q_2$ ile karşılaştır. **(b)** Aynı duruşta ayağa $(0,\ 10)$ N (yukarı) kuvvet etkiyor: $\boldsymbol{\tau} = -\mathbf{J}^T \mathbf{F}_{ext}$ ile torkları bul, sonra her eklem için "moment kolu × kuvvet" ile sağla. **(c)** Ayak kalçanın 5 cm önünde dururken kalça torku ne oluyor? (Kuvvet doğrusu artık kalçadan geçmiyor; moment kolu 5 cm.) **(d)** 1.2 kg ve uyluk/baldır 0.12 m olan bir robot için tabloyu elle yeniden hesapla: MG996R için hangi diz açısına kadar güvenli? **(e)** İki eklemli bir sistemde $\mathbf{J}_{s,2} = \mathrm{Ad}_{\mathbf{E}_1} \mathbf{S}_2$ formülünün, "ikinci eksenin birinci eklem döndükten sonraki konumu" olduğunu bir çizimle göster. **(Bilgisayar) (f)** $\mathbf{J}_b$'yi hesapla ve $\mathbf{J}_b = \mathrm{Ad}_{\mathbf{T}^{-1}} \mathbf{J}_s$ olduğunu sayısal doğrula. **(g)** Kuvvet elipsini ($\mathbf{J}^{-T}$'nin SVD'si) çiz ve hız elipsiyle karşılaştır.

Bölüm 5

Ters Kinematik

Ters kinematik (IK): istenen ayak konumu $\mathbf{p}_d$ için eklem açıları $\mathbf{q}$. Yürüyen bir robotta neredeyse her şey "ayak burada olsun, gövde şurada olsun" diye planlanır ve IK bunu motor komutlarına çevirir. İki yol var: **analitik** (kapalı form, hızlı, tüm çözümleri verir, ama sadece belirli geometrilerde mümkün) ve **sayısal** (her geometride çalışır, başlangıç tahminine bağlı). İkisini de adım adım türetiyoruz. Modern Robotics karşılığı: Bölüm 6.

5.1 Analitik IK: 3-DoF bacak

Kalça roll ve pitch eksenleri kalçada kesiştiği için problem iki kolay parçaya ayrılır: önce roll, bacağın içinde bulunduğu düzlemi seçer; sonra o düzlemde 2 uzuvlu bir problem kalır.

5.1.1 Adım 1: roll

Roll sıfırken bacak x–z (sagital) düzlemindedir ve ayak $(x,\ 0,\ z_s)$, $z_s < 0$. Roll $\mathbf{R}_x(q_1)$ bu noktayı x ekseni etrafında döndürür:

$$\mathbf{R}_x(q_1)\begin{bmatrix} x \\ 0 \\ z_s \end{bmatrix} = \begin{bmatrix} x \\ -\sin q_1\, z_s \\ \cos q_1\, z_s \end{bmatrix}.$$

Hedef (x, y, z) ile eşitle: $y = -z_s \sin q_1$, $z = z_s \cos q_1$. $z_s = -r$ $(r > 0)$ yazınca $y = r \sin q_1$, $-z = r \cos q_1$: bu, $(-z,\ y)$ noktasının kutupsal koordinatlarıdır:

$$q_1 = \operatorname{atan2}(y,\ -z), \qquad z_s = -\sqrt{y^2 + z^2}.$$

x roll'dan etkilenmez.

5.1.2 Adım 2: sagital 2 uzuv, kosinüs teoremi

Bölüm 3'te PoE ile bulduk: $x = -\ell_1 \sin q_2 - \ell_2 \sin(q_2 + q_3)$, $z_s = -\ell_1 \cos q_2 - \ell_2 \cos(q_2 + q_3)$.

Diz açısı. İki denklemin karelerini topla. $\sin^2 + \cos^2 = 1$ ve $\sin a \sin b + \cos a \cos b = \cos(a - b)$ ile:

$$x^2 + z_s^2 = \ell_1^2 + \ell_2^2 + 2\ell_1\ell_2 \cos q_3 \ \Rightarrow\ \cos q_3 = D \equiv \frac{x^2 + z_s^2 - \ell_1^2 - \ell_2^2}{2\ell_1\ell_2}.$$

Bu, kalça–diz–ayak üçgenine uygulanmış **kosinüs teoremidir** (dizdeki iç açı $\pi - q_3$). $q_3 = \pm \arccos D$: iki çözüm, *diz öne* ve *diz arkaya*. $|D| > 1$ ise hedef ulaşılamaz (bacak boyundan uzak veya $|\ell_1 - \ell_2|$'den yakın): kod hata vermeli, sessizce NaN üretmemeli.

Kalça açısı. $(-x,\ -z_s) = \ell_1(\sin q_2, \cos q_2) + \ell_2(\sin(q_2 + q_3), \cos(q_2 + q_3))$: iki vektörün toplamı. Birincisi dikeyden q_2 açısında, ikincisi $q_2 + q_3$. Toplam vektörün dikeyle yaptığı açı $\varphi = \operatorname{atan2}(-x,\ -z_s)$. Koordinat sistemini q_2 kadar döndürüp ilk uzvu "eksen" yaparsak, toplam vektör

bu yeni sistemde $(\ell_1 + \ell_2 \cos q_3,\ \ell_2 \sin q_3)$ olur; yani ilk uzuvdan $\beta = \operatorname{atan2}(\ell_2 \sin q_3,\ \ell_1 + \ell_2 \cos q_3)$ açısında. Dolayısıyla $\varphi = q_2 + \beta$:

$$q_2 = \operatorname{atan2}(-x,\ -z_s) - \operatorname{atan2}(\ell_2 \sin q_3,\ \ell_1 + \ell_2 \cos q_3).$$

Elle: ayak kalçanın 30 cm altında

$(x, y, z) = (0,\ 0,\ -0.3)$, $\ell_1 = \ell_2 = 0.2$. Roll: $q_1 = \operatorname{atan2}(0,\ 0.3) = 0$, $z_s = -0.3$. $D = (0+0.09-0.04-0.04)/(2\cdot 0.04) = 0.01/0.08 = 0.125$, $q_3 = \pm \arccos 0.125 = \pm 1.4455$ rad ($82.8°$). Diz öne ($q_3 > 0$): $\beta = \operatorname{atan2}(0.2 \sin 1.4455,\ 0.2 + 0.2 \cdot 0.125) = \operatorname{atan2}(0.1984,\ 0.225) = 0.7227$; $\varphi = \operatorname{atan2}(0,\ 0.3) = 0$; $q_2 = -0.7227$ rad ($-41.4°$). **Sağlama:** $x = -0.2 \sin(-0.7227) - 0.2 \sin(0.7227) = 0$ ve $z = -0.2 \cos(0.7227) \cdot 2 = -0.4 \cdot 0.75 = -0.3$. Uyluk dikeyden $41.4°$, $\cos = 0.75$: Cilt 6'daki diz torku hesabında kullandığımız açının aynısı.

5.2 Sayısal IK

5.2.1 Newton yöntemi: Taylor açılımından

Analitik çözüm her robot için yoktur (ör. ayak bileği ve kalça eksenleri kesişmiyorsa). Genel yol: hata $\mathbf{e}(\mathbf{q}) = \mathbf{p}_d - \mathrm{FK}(\mathbf{q})$'yu sıfırlamak. FK'yı şu anki $\mathbf{q}_k$ etrafında birinci dereceden açalım:

$$\mathrm{FK}(\mathbf{q}_k + \Delta\mathbf{q}) \approx \mathrm{FK}(\mathbf{q}_k) + \mathbf{J}(\mathbf{q}_k)\Delta\mathbf{q}.$$

Bu yaklaşık modelde hatanın sıfır olmasını iste: $\mathbf{p}_d = \mathrm{FK}(\mathbf{q}_k) + \mathbf{J}\Delta\mathbf{q}$, yani $\mathbf{J}\Delta\mathbf{q} = \mathbf{e}_k$. Bölüm 1'deki gibi en küçük normlu çözüm:

$$\Delta\mathbf{q} = \mathbf{J}^{+}\mathbf{e}_k, \qquad \mathbf{q}_{k+1} = \mathbf{q}_k + \Delta\mathbf{q}.$$

Neden karesel yakınsama? Doğrusal yaklaşımın hatası ikinci dereceden terim, $\mathcal{O}(\|\Delta\mathbf{q}\|^2)$. Bir adım sonra kalan hata, atılan bu terim kadardır: $\|\mathbf{e}_{k+1}\| \approx C\|\mathbf{e}_k\|^2$. Hata 10^{-3} ise sonraki $\sim 10^{-6}$, sonraki $\sim 10^{-12}$: her adımda doğru basamak sayısı iki katına çıkar (iyi bir başlangıçtan).

Elle: Newton'un bir adımı

Bölüm 1'deki bacak (pozitif açı $+x$), $\mathbf{q}_0 = (0,\ \pi/2)$, ayak $(0.2,\ -0.2)$; hedef $(0.22,\ -0.2)$. $\mathbf{e}_0 = (0.02,\ 0)$, $\mathbf{J} = \begin{bmatrix} 0.2 & 0 \\ 0.2 & 0.2 \end{bmatrix}$, $\mathbf{J}^{-1} = \frac{1}{0.04}\begin{bmatrix} 0.2 & 0 \\ -0.2 & 0.2 \end{bmatrix} = \begin{bmatrix} 5 & 0 \\ -5 & 5 \end{bmatrix}$ (2×2 tersi: köşegeni yer değiştir, köşegen dışını eksile, determinanta böl). $\Delta\mathbf{q} = (5 \cdot 0.02,\ -5 \cdot 0.02) = (0.1,\ -0.1)$. Yeni $\mathbf{q}_1 = (0.1,\ 1.4708)$: FK $= (0.21997,\ -0.19900)$, hata $\|\mathbf{e}_1\| = 1.0 \cdot 10^{-3}$ (20 kat azaldı). Bilgisayarla sonraki adımlar: $2.5 \cdot 10^{-6}$, $1.6 \cdot 10^{-11}$: karesel yakınsama gözle görülür.

5.2.2 Sönümlü en küçük kareler (Levenberg–Marquardt)

Newton, tekillik yakınında veya ulaşılamaz hedefte $\mathbf{J}^{+}$ patladığı için kaotikleşir. Bölüm 1'deki DLS'yi adım için kullan:

$$\Delta\mathbf{q} = \mathbf{J}^T(\mathbf{J}\mathbf{J}^T + \lambda^2\mathbf{I})^{-1}\mathbf{e},$$

yani "hatayı azalt ama adımı da küçük tut": $\min \|\mathbf{J}\Delta\mathbf{q} - \mathbf{e}\|^2 + \lambda^2\|\Delta\mathbf{q}\|^2$. λ küçükken Newton, büyükken *gradyan inişi* gibi davranır: $\lambda^2 \gg \sigma^2$ iken $\Delta\mathbf{q} \approx \mathbf{J}^T\mathbf{e}/\lambda^2$, ve $\mathbf{J}^T\mathbf{e}$ tam olarak $\frac{1}{2}\|\mathbf{e}\|^2$'nin eksi gradyanıdır ($\nabla_{\mathbf{q}} \frac{1}{2}\|\mathbf{p}_d - \mathrm{FK}\|^2 = -\mathbf{J}^T\mathbf{e}$). Bu yüzden ulaşılamaz bir hedefte DLS "en yakın noktada" durur: orada gradyan sıfırdır, yani $\mathbf{J}^T\mathbf{e} = \mathbf{0}$ (hata, ayağın hareket edebildiği yönlere dik).

Gerçek sistemlerde eklem limitleri de eklenir (QP ile: Cilt 6'daki WBC'nin IK versiyonu).

Sayılar: analitik ve sayısal IK

Analitik: 1000 rastgele duruşta FK→IK→FK hatası en fazla 1.4×10^{-16} m (C++: 1.5×10^{-16}). Hedef $\mathbf{p} = (0.05, 0.03, -0.33)$: diz öne $(0.0907,\ -0.7274,\ 1.1552)$, diz arkaya $(0.0907,\ 0.4278,\ -1.1552)$; ikisi de hedefe tam ulaşıyor. Roll sağlaması: $\text{atan2}(0.03,\ 0.33) = 0.0907$. İnsan ve çoğu biped "diz öne" çözümünü kullanır; kuş bacaklı robotlar (Cassie) farklı bir geometriye sahiptir. **Sayısal** (başlangıç $(0, -0.2, 0.4)$): Newton 5, DLS $\lambda = 0.01$ 7, DLS $\lambda = 0.05$ 14 iterasyonda 10^{-10}'a iniyor; hepsi aynı (diz öne) çözümü buluyor. **Ulaşılamaz hedef** ($|\mathbf{p}| = 0.45 > 0.4$ m): DLS 100 iterasyon sonra 0.0500 m hatayla (ulaşılabilecek en iyi değer: $0.45 - 0.40$) dümdüz bacakta duruyor, $\|\mathbf{q}\| = 0.22$. Newton ise kaotik: Python'da $\|\mathbf{q}\| = 252$ (diz -225 rad, yani defalarca tam tur), C++'ta $\|\mathbf{q}\| = 13.8$ ve hata 0.052. İki dilin *farklı* anlamsız sonuçlar vermesi şaşırtıcı değil: tekil $\mathbf{J}$'nin sözde tersi yuvarlama hatalarına aşırı duyarlı. **SB-1 çömelme:** ayak yerde sabitken kalça 5 cm alçalıp 3 cm öne gidiyor (kübik zaman ölçeklemesi $s = 3t^2 - 2t^3$: $s(0) = 0$, $s(1) = 1$, $\dot{s} = 6t - 6t^2$ başta ve sonda sıfır, böylece eklem hızları sıçramaz). Kalça çerçevesinde ayak $(0, 0, -0.38) \rightarrow (-0.03, 0, -0.33)$; eklemler kalça pitch $-0.318 \rightarrow -0.504$, diz $0.635 \rightarrow 1.189$ rad.

Python

```python
import numpy as np
from robkit import fk_poe, space_jacobian, homog, skew
np.set_printoptions(precision=5, suppress=True)

l1, l2 = 0.2, 0.2
def screw(w, qp): w = np.array(w, float); return np.concatenate([w, -np.cross(w, qp)])
Slist = [screw([1, 0, 0], [0, 0, 0]), screw([0, 1, 0], [0, 0, 0]), screw([0, 1, 0], [0, 0, -l1])]
M = homog(np.eye(3), [0, 0, -l1 - l2])
foot = lambda q: fk_poe(Slist, M, q)[:3, 3]
def Jp(q): Js = space_jacobian(Slist, q); return Js[3:] - skew(foot(q))@Js[:3]

def ik_analytic(p, knee_sign=+1):
    """Kapalı form IK. p = ayak konumu (kalça çerçevesi). knee_sign =+1: diz öne bükük (q3>0)."""
    x, y, z = p
    q1 = np.arctan2(y, -z)                                  # roll: y-z düzleminde bacağı hizala
    zs = -np.hypot(y, z)                                    # roll sonrası sagital düzlemde ayak yüksekliği
    D = (x*x + zs*zs - l1*l1 - l2*l2)/(2*l1*l2)             # kosinüs teoremi
    if abs(D) > 1:
        raise ValueError(f"ulaşılamaz hedef: |p| = {np.linalg.norm(p):.4f} > {l1 + l2}")
    q3 = knee_sign*np.arccos(D)
    q2 = np.arctan2(-x, -zs) - np.arctan2(l2*np.sin(q3), l1 + l2*np.cos(q3))
    return np.array([q1, q2, q3])

def ik_numeric(p, q0, lam=0.0, iters=100, tol=1e-10):
    """Newton (lam =0) veya sönümlü en küçük kareler (lam>0) ile sayısal IK"""
    q = np.array(q0, float)
    for k in range(iters):
        e = p - foot(q)
        if np.linalg.norm(e) < tol: return q, k, np.linalg.norm(e)
        J = Jp(q)
        dq = J.T@np.linalg.solve(J@J.T + lam**2*np.eye(3), e) if lam > 0 else np.linalg.lstsq(J, e,
        rcond=None)[0]
        q = q + dq
    return q, iters, np.linalg.norm(p - foot(q))

# (1) Analitik IK: rastgele (ama tekrar üretilebilir) 1000 hedef
```

```python
rng = np.random.default_rng(0); errs = []
for _ in range(1000):
    qt = np.array([rng.uniform(-0.4, 0.4), rng.uniform(-1.2, 0.6), rng.uniform(0.05, 2.2)])
    errs.append(np.linalg.norm(foot(ik_analytic(foot(qt))) - foot(qt)))
print(f"analitik IK, 1000 rastgele hedef: max konum hatası {max(errs):.2e} m")
p = np.array([0.05, 0.03, -0.33])
qa, qb = ik_analytic(p, +1), ik_analytic(p, -1)
print("hedef", p, "-> diz öne:", qa, " diz arkaya:", qb, " (iki çözüm)")
print("  FK kontrol:", foot(qa), foot(qb))

# (2) Sayısal IK: Newton vs DLS
for name, lam in [("Newton (pinv)", 0.0), ("DLS λ =0.01", 0.01), ("DLS λ =0.05", 0.05)]:
    q, k, e = ik_numeric(p, [0, -0.2, 0.4], lam=lam)
    print(f"{name:>14}: {k:3d} iterasyon, hata {e:.1e}, q = {q}")
# Ulaşılamaz hedef (bacak boyu 0.4 m, hedef 0.45 m uzakta)
pf = np.array([0.1, 0.0, -0.4387])
print(f"\nulaşılamaz hedef |p| = {np.linalg.norm(pf):.4f} m:")
for name, lam in [("Newton (pinv)", 0.0), ("DLS λ =0.05", 0.05)]:
    q, k, e = ik_numeric(pf, [0, -0.2, 0.4], lam=lam, iters=100)
    print(f"{name:>14}: 100 iterasyon sonra hata {e:.4f} m, |q| = {np.linalg.norm(q):.2f}, diz =
    {q[2]:.4f} rad")
try: ik_analytic(pf)
except ValueError as ex: print("analitik IK:", ex)

# (3) SB-1 için: ayak yerde sabit, kalça 5 cm alçalıp 3 cm öne gidiyor (çömelme + öne kayma)
t = np.linspace(0, 1, 6); p0 = np.array([0.0, 0.0, -0.38])
print("\nçömelme yörüngesi (ayak kalça çerçevesinde; kalça aşağı/öne gidince ayak göreli olarak
yukarı/geri gider):")
for ti in t:
    s = 3*ti**2 - 2*ti**3                                  # yumuşak başlangıç/bitiş (kübik)
    pt = p0 + np.array([-0.03*s, 0, 0.05*s]); qt = ik_analytic(pt)
    print(f"  t ={ti:.1f}: ayak {pt.round(4)} -> q = {qt.round(4)}")
```

Çıktı:

```
analitik IK, 1000 rastgele hedef: max konum hatası 1.39e-16 m
hedef [ 0.05  0.03 -0.33] -> diz öne: [ 0.09066 -0.72736  1.15518]  diz arkaya: [ 0.09066  0.42783 -1.15518]  (iki
çözüm)
  FK kontrol: [ 0.05  0.03 -0.33] [ 0.05  0.03 -0.33]
 Newton (pinv):   5 iterasyon, hata 8.7e-12, q = [ 0.09066 -0.72736  1.15518]
    DLS λ =0.01:   7 iterasyon, hata 7.8e-12, q = [ 0.09066 -0.72736  1.15518]
    DLS λ =0.05:  14 iterasyon, hata 4.6e-11, q = [ 0.09066 -0.72736  1.15518]

ulaşılamaz hedef |p| = 0.4500 m:
 Newton (pinv): 100 iterasyon sonra hata 0.0724 m, |q| = 252.04, diz = -225.5207 rad
    DLS λ =0.05: 100 iterasyon sonra hata 0.0500 m, |q| = 0.22, diz = -0.0000 rad
analitik IK: ulaşılamaz hedef: |p| = 0.4500 > 0.4

çömelme yörüngesi (ayak kalça çerçevesinde; kalça aşağı/öne gidince ayak göreli olarak yukarı/geri gider):
  t =0.0: ayak [ 0.    0.   -0.38] -> q = [ 0.     -0.3176  0.6351]
  t =0.2: ayak [-0.0031  0.     -0.3748] -> q = [ 0.     -0.3484  0.7135]
  t =0.4: ayak [-0.0106  0.     -0.3624] -> q = [ 0.     -0.407   0.8723]
  t =0.6: ayak [-0.0194  0.     -0.3476] -> q = [ 0.     -0.459   1.0297]
  t =0.8: ayak [-0.0269  0.     -0.3352] -> q = [ 0.     -0.4922  1.1445]
  t =1.0: ayak [-0.03  0.   -0.33] -> q = [ 0.     -0.5039  1.1891]
```

C++

```cpp
// c05_ik.cpp — analitik ve sayısal (Newton / DLS) ters kinematik
#include "robkit.hpp"
#include <cstdio>
```

```cpp
#include <random>
#include <stdexcept>
using namespace rk;
using Eigen::Vector3d;

const double l1 = 0.2, l2 = 0.2;
Vec6 screw(const Vec3& w, const Vec3& q) { Vec6 S; S << w, -w.cross(q); return S; }
const std::vector<Vec6> S = {screw(Vec3::UnitX(), Vec3::Zero()), screw(Vec3::UnitY(), Vec3::Zero()),
                             screw(Vec3::UnitY(), Vec3(0, 0, -l1))};
const Mat4 M = homog(Mat3::Identity(), Vec3(0, 0, -l1 - l2));
Vec3 foot(const Vector3d& q) { return fk_poe(S, M, q).topRightCorner<3, 1>(); }
Mat3 Jp(const Vector3d& q) { Jac Js = space_jacobian(S, q); return Js.bottomRows<3>() - skew(foot(q)) *
Js.topRows<3>(); }

Vector3d ik_analytic(const Vec3& p, int knee_sign = +1) {
    double x = p(0), y = p(1), z = p(2);
    double q1 = std::atan2(y, -z), zs = -std::hypot(y, z);
    double D = (x * x + zs * zs - l1 * l1 - l2 * l2) / (2 * l1 * l2);
    if (std::abs(D) > 1) throw std::domain_error("ulaşılamaz hedef");
    double q3 = knee_sign * std::acos(D);
    double q2 = std::atan2(-x, -zs) - std::atan2(l2 * std::sin(q3), l1 + l2 * std::cos(q3));
    return Vector3d(q1, q2, q3);
}

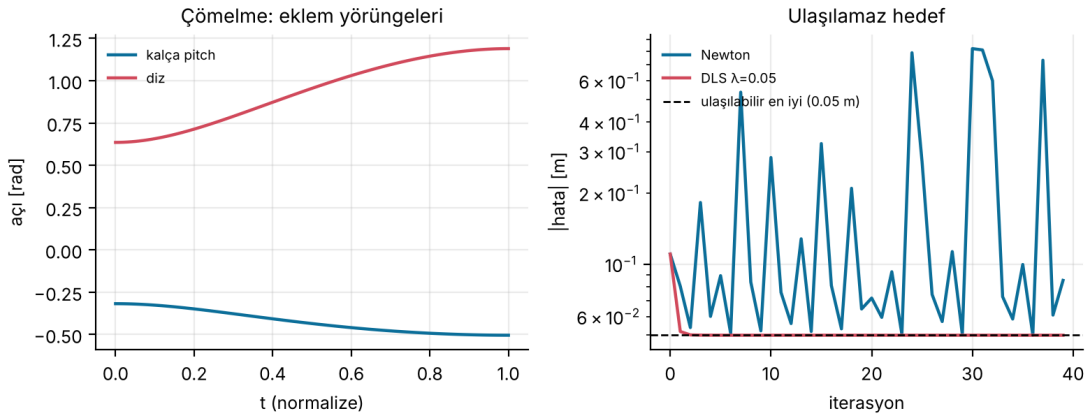
struct IkResult { Vector3d q; int iters; double err; };
IkResult ik_numeric(const Vec3& p, Vector3d q, double lam, int iters = 100, double tol = 1e-10) {
    for (int k = 0; k < iters; ++k) {
        Vec3 e = p - foot(q);
        if (e.norm() < tol) return {q, k, e.norm()};
        Mat3 J = Jp(q);
        Vec3 dq = (lam > 0) ? Vec3(J.transpose() * (J * J.transpose() + lam * lam *
Mat3::Identity()).ldlt().solve(e))
                            : Vec3(J.completeOrthogonalDecomposition().solve(e));
        q += dq;
    }
    return {q, iters, (p - foot(q)).norm()};
}

int main() {
    std::mt19937 gen(0);
    std::uniform_real_distribution<double> u1(-0.4, 0.4), u2(-1.2, 0.6), u3(0.05, 2.2);
    double maxerr = 0;
    for (int i = 0; i < 1000; ++i) {
        Vector3d qt(u1(gen), u2(gen), u3(gen));
        maxerr = std::max(maxerr, (foot(ik_analytic(foot(qt))) - foot(qt)).norm());
    }
    std::printf("analitik IK, 1000 rastgele hedef: max hata %.2e m\n", maxerr);
    Vec3 p(0.05, 0.03, -0.33);
    Vector3d qa = ik_analytic(p, +1), qb = ik_analytic(p, -1);
    std::printf("diz öne: [%.5f %.5f %.5f], diz arkaya: [%.5f %.5f %.5f]\n", qa(0), qa(1), qa(2),
qb(0), qb(1), qb(2));
    for (double lam : {0.0, 0.01, 0.05}) {
        auto r = ik_numeric(p, Vector3d(0, -0.2, 0.4), lam);
        std::printf("λ =%.2f: %3d iterasyon, hata %.1e\n", lam, r.iters, r.err);
    }
    Vec3 pf(0.1, 0.0, -0.4387);
    for (double lam : {0.0, 0.05}) {
        auto r = ik_numeric(pf, Vector3d(0, -0.2, 0.4), lam);
        std::printf("ulaşılamaz hedef, λ =%.2f: hata %.4f m, |q| = %.2f\n", lam, r.err, r.q.norm());
    }
```

```
    try { ik_analytic(pf); } catch (const std::domain_error& ex) { std::printf("analitik IK istisna: %s\n", ex.what()); }
    return 0;
}
```

Çıktı:

```
analitik IK, 1000 rastgele hedef: max hata 1.50e-16 m
diz öne: [0.09066 -0.72736 1.15518], diz arkaya: [0.09066 0.42783 -1.15518]
λ =0.00:   5 iterasyon, hata 8.7e-12
λ =0.01:   7 iterasyon, hata 7.8e-12
λ =0.05:  14 iterasyon, hata 4.6e-11
ulaşılamaz hedef, λ =0.00: hata 0.0516 m, |q| = 13.75
ulaşılamaz hedef, λ =0.05: hata 0.0500 m, |q| = 0.22
analitik IK istisna: ulaşılamaz hedef
```



Şekil 5.1: Sol: çömelme hareketinde analitik IK'dan gelen eklem yörüngeleri. Sağ: ulaşılamaz bir hedefte hata normu; DLS ulaşılabilir en iyi değerde (kesikli) duruyor, Newton salınıyor.

Python ve C++ notu: hata yönetimi ve rastgele sayılar

- **İstisnalar:** Python `raise ValueError(...)` / `try: ... except ValueError as ex:` ↔ C++ `throw std::domain_error(...)` / `try {...} catch (const std::domain_error& ex)`. Gerçek zamanlı kontrol döngüsünde (1 kHz) istisna fırlatmak yerine bir durum kodu döndürmek tercih edilir (`std::optional`; Cilt 2'deki `sb1::inverse` böyle).
- **Birden fazla değer döndürmek:** Python tuple (`return q, k, e`) ↔ C++ `struct IkResult {...}` ve `auto r = ...; r.iters`.
- **Rastgele sayılar:** numpy `default_rng(0)` ve C++ `std::mt19937 gen(0)` *farklı* diziler üretir; iki dilde aynı testi yapmak için aynı sayıları bekleme, aynı *istatistiği* (ör. max hata) karşılaştır.
- **`atan2(y, x)`:** argüman sırası y önce; dört çeyrekte doğru açı verir. `atan(y/x)` kullanma: $x < 0$ iken açıyı π kadar yanlış verir ve $x = 0$'da sıfıra böler.

Alıştırma

(Kâğıt) (a) Hedef $(0.1,\ 0,\ -0.3)$ için analitik IK'yı (diz öne ve diz arkaya) elle çöz ve FK ile sağla. **(b)** Hedef $(0,\ 0.05,\ -0.35)$: önce roll açısını, sonra sagital problemi çöz. **(c)** $|D| \le 1$ koşulunun $|\ell_1 - \ell_2| \le \sqrt{x^2 + z_s^2} \le \ell_1 + \ell_2$'ye eşit olduğunu göster. **(d)** Elle Newton örneğinde

hedefi $(0.25,\ -0.2)$ yap; bir adım at, hatanın ne kadar azaldığını hesapla. **(e)** $\lambda = 0.1$ ile aynı adımı DLS ile at (2×2 matris tersi); adımın kısaldığını gör. **(Bilgisayar) (f)** Ayak bileği pitch eklemi ekle ve "ayak tabanı yere paralel kalsın" koşuluyla analitik IK yaz ($q_4 = -(q_2 + q_3)$ yeterli mi? roll ile birlikte düşün). **(g)** Sayısal IK'ya eklem limitleri ekle ($q_3 \in [0.05, 2.4]$): her adımdan sonra kırpma (clip) ile null uzayı kullanma yöntemini karşılaştır. **(h)** Elindeki 2 servoyla (P2): sagital 2-uzuvlu bacak için bu IK'yı Arduino'da (C++) yaz; ayak ucunu 4 cm'lik bir kare üzerinde gezdir ve cetvelle ölç.

Bölüm 6

Rijit Cisim Dinamiği: Lagrange ve RNEA

Kinematik "nereye"yi söyler, dinamik "hangi kuvvetle"yi. Bir robotun hareket denklemi (manipülatör denklemi)

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau} \qquad (\mathbf{h} = \mathbf{C}\dot{\mathbf{q}} + \mathbf{g}: \text{ "bias" kuvvetleri})$$

ve iki temel problem: **ters dinamik** ($\mathbf{q},\dot{\mathbf{q}},\ddot{\mathbf{q}} \to \boldsymbol{\tau}$; kontrol için: "bu ivme için hangi tork?") ve **ileri dinamik** ($\boldsymbol{\tau} \to \ddot{\mathbf{q}} = \mathbf{M}^{-1}(\boldsymbol{\tau} - \mathbf{h})$; simülasyon için). Bu bölümde denklemi iki yoldan, Lagrange ve Newton–Euler ile, 2 uzuvlu bacak için tüm adımlarıyla türetiyoruz. Modern Robotics karşılığı: Bölüm 8; derinleşme: Featherstone, *Rigid Body Dynamics Algorithms*.

6.1 Atalet

6.1.1 Rijit cismin kinetik enerjisi

Cismi küçük kütleler m_i'den oluşmuş say. Kütle merkezi $\mathbf{c} = \frac{1}{m}\sum m_i\mathbf{r}_i$, her parçacık $\mathbf{r}_i = \mathbf{c} + \boldsymbol{\rho}_i$, tanım gereği $\sum m_i\boldsymbol{\rho}_i = \mathbf{0}$. Rijit cisimde $\dot{\boldsymbol{\rho}}_i = \boldsymbol{\omega}\times\boldsymbol{\rho}_i$:

$$T = \tfrac{1}{2}\sum m_i\|\dot{\mathbf{c}} + \boldsymbol{\omega}\times\boldsymbol{\rho}_i\|^2 = \tfrac{1}{2}m\|\dot{\mathbf{c}}\|^2 + \dot{\mathbf{c}}\cdot\Big(\boldsymbol{\omega}\times\underbrace{\sum m_i\boldsymbol{\rho}_i}_{\mathbf{0}}\Big) + \tfrac{1}{2}\sum m_i\|\boldsymbol{\omega}\times\boldsymbol{\rho}_i\|^2.$$

Çapraz terim kayboldu (kütle merkezini referans seçmenin faydası). Son terim için $\|\boldsymbol{\omega}\times\boldsymbol{\rho}\|^2 = \boldsymbol{\omega}^T(\|\boldsymbol{\rho}\|^2\mathbf{I} - \boldsymbol{\rho}\boldsymbol{\rho}^T)\boldsymbol{\omega}$ (Lagrange özdeşliği):

$$\boxed{T = \tfrac{1}{2}m\|\dot{\mathbf{c}}\|^2 + \tfrac{1}{2}\boldsymbol{\omega}^T\mathbf{I}_c\boldsymbol{\omega}}, \qquad \mathbf{I}_c = \sum m_i(\|\boldsymbol{\rho}_i\|^2\mathbf{I} - \boldsymbol{\rho}_i\boldsymbol{\rho}_i^T).$$

Öteleme enerjisi + kütle merkezi etrafında dönme enerjisi (König teoremi). $\mathbf{I}_c$ **atalet tensörü**: simetrik, pozitif tanımlı 3×3.

Kutu ($a\times b\times c$, düzgün yoğunluk): $I_{xx} = \int(y^2+z^2)\,dm$. y, $[-b/2, b/2]$'de düzgün dağılmış: $\frac{1}{b}\int_{-b/2}^{b/2} y^2dy = \frac{b^2}{12}$, aynısı z için. Böylece $I_{xx} = \frac{m}{12}(b^2+c^2)$ ve

$$\mathbf{I}_c = \tfrac{m}{12}\mathrm{diag}(b^2+c^2,\ a^2+c^2,\ a^2+b^2).$$

İnce çubuk ($a,b\to 0$, boy ℓ): eksenine dik $I = m\ell^2/12$.

Paralel eksen (Steiner): başka bir $\mathbf{o}$ noktasına göre, $\mathbf{r} = \mathbf{o}$'dan kütle merkezine vektör, $\mathbf{r}_i - \mathbf{o} = \mathbf{r} + \boldsymbol{\rho}_i$. Tanımı açınca $\sum m_i\boldsymbol{\rho}_i = \mathbf{0}$ yüzünden çapraz terimler yine kaybolur:

$$\mathbf{I}_o = \mathbf{I}_c + m(\|\mathbf{r}\|^2\mathbf{I} - \mathbf{r}\mathbf{r}^T).$$

Skaler hâli $I_o = I_c + md^2$. İnce çubuğun ucuna göre $\frac{m\ell^2}{12} + m\frac{\ell^2}{4} = \frac{m\ell^2}{3}$. SB-1'in MJCF dosyasını yazarken (Cilt 2) her uzvun `inertial` değerlerini tam bu formüllerle hesapladık. CAD programları bunu otomatik verir; ama bir değeri "mantıklı mı?" diye kontrol edebilmen gerekir.

6.2 Lagrange yöntemi

6.2.1 Euler–Lagrange denklemi

$\mathcal{L} = T - V$ (kinetik eksi potansiyel enerji). Genelleştirilmiş koordinatlar $\mathbf{q}$ ve her koordinata etki eden **genelleştirilmiş kuvvet** τ_i (dönel eklemde motor torku) için:

$$\frac{d}{dt}\frac{\partial \mathcal{L}}{\partial \dot{q}_i} - \frac{\partial \mathcal{L}}{\partial q_i} = \tau_i.$$

Neden doğru? En basit durumda kontrol et: tek parçacık, $q = x$, $T = \frac{1}{2}m\dot{x}^2$, $V(x)$. $\frac{d}{dt}(m\dot{x}) + V'(x) = \tau \Rightarrow m\ddot{x} = \tau - V'$: Newton'un ikinci yasası (potansiyel kuvvet $-V'$ ve dış kuvvet τ). Genel ispat, Newton yasalarını her parçacık için yazıp d'Alembert/sanal iş ilkesiyle (Bölüm 4) genelleştirilmiş koordinatlara izdüşürmekle yapılır. Avantajı: kısıt kuvvetlerini (eklemdeki tepki kuvvetleri) hiç hesaplamadan doğrudan hareket denklemini verir.

Elle: tek sarkaç

Kütle m, boy ℓ, açı θ dikey-aşağıdan. Konum $(\ell\sin\theta,\ -\ell\cos\theta)$, hız büyüklüğü $\ell\dot{\theta}$: $T = \frac{1}{2}m\ell^2\dot{\theta}^2$, $V = -mg\ell\cos\theta$. $\frac{\partial \mathcal{L}}{\partial \dot{\theta}} = m\ell^2\dot{\theta}$, $\frac{\partial \mathcal{L}}{\partial \theta} = -mg\ell\sin\theta$:

$$m\ell^2\ddot{\theta} + mg\ell\sin\theta = \tau.$$

Newton ile sağlama: eklem etrafında moment, $I\ddot{\theta} = \tau - mg\ell\sin\theta$ (yerçekiminin moment kolu $\ell\sin\theta$), $I = m\ell^2$. Aynı.

6.2.2 Genel manipülatör denklemine geçiş

Robotlarda $T = \frac{1}{2}\dot{\mathbf{q}}^T\mathbf{M}(\mathbf{q})\dot{\mathbf{q}}$ (her uzvun $\frac{1}{2}m\|\dot{\mathbf{c}}\|^2 + \frac{1}{2}I\omega^2$'si, $\dot{\mathbf{c}}$ ve ω eklem hızlarında doğrusal olduğu için kuadratik). Euler–Lagrange'ı vektör hâlinde uygula:

$$\begin{aligned} &\frac{d}{dt}(\mathbf{M}\dot{\mathbf{q}}) - \frac{1}{2}\frac{\partial}{\partial \mathbf{q}}(\dot{\mathbf{q}}^T\mathbf{M}\dot{\mathbf{q}}) + \frac{\partial V}{\partial \mathbf{q}} = \boldsymbol{\tau} \\ \Rightarrow\ &\mathbf{M}\ddot{\mathbf{q}} + \dot{\mathbf{M}}\dot{\mathbf{q}} - \underbrace{\frac{1}{2}\frac{\partial}{\partial \mathbf{q}}(\dot{\mathbf{q}}^T\mathbf{M}\dot{\mathbf{q}})}_{\mathbf{C}\dot{\mathbf{q}}} + \underbrace{\frac{\partial V}{\partial \mathbf{q}}}_{\mathbf{g}} = \boldsymbol{\tau}. \end{aligned}$$

$\mathbf{C}\dot{\mathbf{q}}$'nun bileşenleri **Christoffel sembolleri** ile yazılır:

$$C_{ij} = \sum_k c_{ijk}\dot{q}_k, \qquad c_{ijk} = \frac{1}{2}\Big(\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{jk}}{\partial q_i}\Big).$$

(İndisleri açarak $\sum_j C_{ij}\dot{q}_j$'nin yukarıdaki iki terimin i. bileşenine eşit olduğunu doğrulayabilirsin.) Bu terimler merkezkaç ($\dot{q}_j^2$) ve Coriolis ($\dot{q}_j\dot{q}_k$) kuvvetleridir.

6.2.3 2 uzuvlu bacak: tam türetme

Kalça sabit, açılar dikey-aşağıdan (Bölüm 1 kuralı), düzgün çubuklar: kütle merkezleri ortada $\ell_{ci} = \ell_i/2$, $I_i = m_i\ell_i^2/12$. $c_1 = \cos q_1$, $s_{12} = \sin(q_1 + q_2)$ kısaltmaları.

Konumlar ve hızlar.

$$\mathbf{c}_1 = \ell_{c1}(s_1,\ -c_1), \qquad \mathbf{c}_2 = \ell_1(s_1,\ -c_1) + \ell_{c2}(s_{12},\ -c_{12}),$$

$$\dot{\mathbf{c}}_1 = \ell_{c1}\dot{q}_1(c_1,\ s_1), \qquad \dot{\mathbf{c}}_2 = \ell_1\dot{q}_1(c_1,\ s_1) + \ell_{c2}(\dot{q}_1+\dot{q}_2)(c_{12},\ s_{12}).$$

Kinetik enerji. $\|\dot{\mathbf{c}}_2\|^2 = \ell_1^2\dot{q}_1^2 + \ell_{c2}^2(\dot{q}_1+\dot{q}_2)^2 + 2\ell_1\ell_{c2}\dot{q}_1(\dot{q}_1+\dot{q}_2)\,\underbrace{(c_1c_{12}+s_1s_{12})}_{\cos q_2}$. Açısal hızlar $\omega_1 = \dot{q}_1$, $\omega_2 = \dot{q}_1+\dot{q}_2$:

$$T = \tfrac{1}{2}(m_1\ell_{c1}^2 + I_1)\dot{q}_1^2 + \tfrac{1}{2}m_2\|\dot{\mathbf{c}}_2\|^2 + \tfrac{1}{2}I_2(\dot{q}_1+\dot{q}_2)^2.$$

$T = \tfrac{1}{2}(M_{11}\dot{q}_1^2 + 2M_{12}\dot{q}_1\dot{q}_2 + M_{22}\dot{q}_2^2)$ ile katsayıları eşleştirince:

$$M_{11} = I_1 + I_2 + m_1\ell_{c1}^2 + m_2(\ell_1^2 + \ell_{c2}^2 + 2\ell_1\ell_{c2}\cos q_2),$$
$$M_{12} = I_2 + m_2(\ell_{c2}^2 + \ell_1\ell_{c2}\cos q_2), \qquad M_{22} = I_2 + m_2\ell_{c2}^2.$$

Potansiyel enerji (z yukarı, $V = \sum m_i g z_i$): $V = -m_1 g\ell_{c1}c_1 - m_2 g(\ell_1 c_1 + \ell_{c2}c_{12})$. Türev:

$$\mathbf{g} = \frac{\partial V}{\partial \mathbf{q}} = \begin{bmatrix} m_1 g\ell_{c1}s_1 + m_2 g(\ell_1 s_1 + \ell_{c2}s_{12}) \\ m_2 g\ell_{c2}s_{12} \end{bmatrix}.$$

Coriolis/merkezkaç. M sadece q_2'ye bağlı. $h \equiv -m_2\ell_1\ell_{c2}\sin q_2$ ile $\partial M_{11}/\partial q_2 = 2h$, $\partial M_{12}/\partial q_2 = h$, $\partial M_{22}/\partial q_2 = 0$. Christoffel sembollerinden sıfır olmayanlar: $c_{112} = c_{121} = c_{122} = h$, $c_{211} = -h$. Örnek: $c_{211} = \tfrac{1}{2}(\frac{\partial M_{21}}{\partial q_1} + \frac{\partial M_{21}}{\partial q_1} - \frac{\partial M_{11}}{\partial q_2}) = \tfrac{1}{2}(0+0-2h) = -h$. Toplarsak:

$$\mathbf{C} = \begin{bmatrix} h\dot{q}_2 & h(\dot{q}_1+\dot{q}_2) \\ -h\dot{q}_1 & 0 \end{bmatrix}.$$

Fiziksel yorum: $C_{21}\dot{q}_1 = -h\dot{q}_1^2 = m_2\ell_1\ell_{c2}\sin q_2\,\dot{q}_1^2$; uyluk hızla dönerken baldıra etki eden merkezkaç kuvvetinin diz etrafındaki momenti.

6.2.4 Yapısal özellik: $\dot{\mathbf{M}} - 2\mathbf{C}$ ters simetrik

Christoffel tanımından $c_{ijk} + c_{jik} = \partial M_{ij}/\partial q_k$ (ilk iki terim toplanır, üçüncüler birbirini götürür). Dolayısıyla $C_{ij} + C_{ji} = \sum_k \frac{\partial M_{ij}}{\partial q_k}\dot{q}_k = \dot{M}_{ij}$, yani $\dot{\mathbf{M}} = \mathbf{C} + \mathbf{C}^T$ ve

$$\dot{\mathbf{M}} - 2\mathbf{C} = \mathbf{C}^T - \mathbf{C} \quad \text{(ters simetrik)}.$$

Sonucu: $\dot{\mathbf{q}}^T(\dot{\mathbf{M}} - 2\mathbf{C})\dot{\mathbf{q}} = 0$. Toplam enerjinin türevi $\dot{E} = \dot{\mathbf{q}}^T\boldsymbol{\tau}$ çıkar (motorların verdiği güç); enerji tabanlı kontrolcülerin ve pasiflik ispatlarının kullandığı özellik budur. Lagrange 2–3 eklem için öğreticidir; 12 eklemli bir bipedde elle türetmek pratik değildir: RNEA'ya geçiyoruz.

Elle: $\mathbf{q} = (0.4,\ 0.7)$'de $\mathbf{M}$ ve $\mathbf{g}$

$m_1 = 0.3$, $m_2 = 0.2$, $\ell = 0.2$, $\ell_c = 0.1$. $I_1 = 0.3\cdot 0.04/12 = 0.001$, $I_2 = 0.000667$, $\cos 0.7 = 0.7648$. $M_{11} = 0.001 + 0.000667 + 0.3\cdot 0.01 + 0.2(0.04 + 0.01 + 2\cdot 0.2\cdot 0.1\cdot 0.7648) = 0.001 + 0.000667 + 0.003 + 0.01612 = 0.02079$. $M_{12} = 0.000667 + 0.2(0.01 + 0.02\cdot 0.7648) = 0.00573$; $M_{22} = 0.000667 + 0.002 = 0.00267$. $g_1 = 0.3\cdot 9.81\cdot 0.1\sin 0.4 + 0.2\cdot 9.81(0.2\sin 0.4 + 0.1\sin 1.1) = 0.1146 + 0.3277 = 0.4423$; $g_2 = 0.2\cdot 9.81\cdot 0.1\sin 1.1 = 0.1749$ N·m. Program çıktısıyla (aşağıda) birebir.

6.3 RNEA: özyinelemeli Newton–Euler

6.3.1 Tek bir uzvun denklemleri

Uzuv i'ye iki kuvvet ve moment etki eder: ebeveyn eklemden $(\mathbf{f}_i, n_i)$ ve çocuk uzuvdan tepki olarak $(-\mathbf{f}_{i+1}, -n_{i+1})$ (Newton'un üçüncü yasası). Kütle merkezi ivmesi $\mathbf{a}_{c,i}$, açısal ivme α_i:

$$\begin{aligned} \text{Newton:}\ \ & \mathbf{f}_i - \mathbf{f}_{i+1} = m_i \mathbf{a}_{c,i} \ \Rightarrow\ \mathbf{f}_i = m_i \mathbf{a}_{c,i} + \mathbf{f}_{i+1}, \\ \text{Euler (eklem } i \text{ etrafında):}\ \ & n_i = I_i \alpha_i + \mathbf{r}_{c,i} \times m_i \mathbf{a}_{c,i} + n_{i+1} + \mathbf{r}_{i,i+1} \times \mathbf{f}_{i+1}. \end{aligned}$$

(İkincisi: eklem i etrafındaki moment, kütle merkezi etrafındaki dönme $(I\alpha)$ + kütle merkezinin ivmelenmesi için gereken kuvvetin momenti + çocuğa iletilen moment ve kuvvetin momenti.) Dönel eklemde motor torku $\tau_i = n_i$ (düzlemde; 3-B'de eksen bileşeni).

İvmeler. Dönen bir uzuv üzerinde, uzvun başlangıç noktasından $\mathbf{r}$ ötedeki noktanın ivmesi (düzlemde $\mathbf{r}^{\perp}$, $\mathbf{r}$'nin 90° döndürülmüşü):

$$\mathbf{a} = \mathbf{a}_p + \alpha\, \mathbf{r}^{\perp} - \omega^2 \mathbf{r} \quad (\text{teğetsel} + \text{merkezcil}).$$

Bu, $\mathbf{r}$'yi $\mathbf{r} = r(\cos\phi, \sin\phi)$ yazıp iki kez türev almakla çıkar.

6.3.2 İki geçiş

1. **İleri geçiş** (tabandan uca): $\omega_i = \sum_{k \le i} \dot{q}_k$, $\alpha_i = \sum_{k \le i} \ddot{q}_k$, ve yukarıdaki formülle her uzvun kütle merkezi ve bir sonraki eklem ivmesi. İvme bilgisi tabandan uca akar.
2. **Geri geçiş** (uçtan tabana): son uzuvda $\mathbf{f}_{n+1} = \mathbf{0}$; Newton ve Euler denklemleriyle $\mathbf{f}_i$, n_i, τ_i. Kuvvet bilgisi uçtan tabana akar.

Yerçekimi hilesi: yerçekimini her uzva ayrı eklemek yerine tabanı $+g$ ile yukarı ivmelendir. Neden eşdeğer? Newton denklemine yerçekimi $\mathbf{f}_i - \mathbf{f}_{i+1} + m_i \mathbf{g}_{vec} = m_i \mathbf{a}_{c,i}$ diye girer; $\mathbf{g}_{vec} = (0, -g)$'yi sola atınca $m_i(\mathbf{a}_{c,i} - \mathbf{g}_{vec})$: her ivmeye $(0, +g)$ eklemek. İleri geçiş bu eklemeyi tabandan tüm uzuvlara kendiliğinden taşır.

Maliyet $\mathcal{O}(n)$. RNEA ile her şey hesaplanır: $\mathbf{g}(\mathbf{q}) = \text{RNEA}(\mathbf{q}, 0, 0)$; $\mathbf{h} = \text{RNEA}(\mathbf{q}, \dot{\mathbf{q}}, 0)$; $\mathbf{M}$'nin i. sütunu $= \text{RNEA}(\mathbf{q}, 0, \mathbf{e}_i)$ yerçekimsiz (çünkü o durumda denklem $\boldsymbol{\tau} = \mathbf{M}\mathbf{e}_i$). CRBA bunu daha verimli yapar. Pinocchio, MuJoCo, Drake'in içindeki algoritmalar bunların 3-B "uzaysal vektör" (6-B) halleridir.

6.4 İleri dinamik ve sayısal integrasyon

$\ddot{\mathbf{q}} = \mathbf{M}^{-1}(\boldsymbol{\tau} - \mathbf{h})$ ile simülasyon. Integratör seçimi enerji korunumunu doğrudan etkiler; enerji, simülasyonun doğruluğunu test etmenin en ucuz yoludur (sürtünmesiz sistemde sabit kalmalı).

6.4.1 Üç integratör ve harmonik osilatörde ne yaptıkları

Durum (x, v), $\dot{x} = v$, $\dot{v} = a(x, v)$:

- **Açık Euler:** $x_{k+1} = x_k + \Delta t\, v_k$, $v_{k+1} = v_k + \Delta t\, a_k$.

- **Yarı-örtük (symplectic) Euler:** önce $v_{k+1} = v_k + \Delta t\, a_k$, sonra $x_{k+1} = x_k + \Delta t\, v_{k+1}$ (*yeni* hızla).
- **RK4:** adım içinde dört eğim (k_1 başta, k_2, k_3 ortada, k_4 sonda) ve ağırlıklı ortalama $\frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$; hata $\mathcal{O}(\Delta t^4)$.

Neden açık Euler enerji pompalar? $\ddot{x} = -\omega^2 x$ için açık Euler adımı

$$\begin{bmatrix} x \\ v/\omega \end{bmatrix}_{k+1} = \begin{bmatrix} 1 & \omega\Delta t \\ -\omega\Delta t & 1 \end{bmatrix} \begin{bmatrix} x \\ v/\omega \end{bmatrix}_{k}.$$

Enerji $\propto x^2 + (v/\omega)^2$, yani bu vektörün boyunun karesi. Matris bir dönme çarpı $\sqrt{1+\omega^2\Delta t^2}$ (determinant $1+\omega^2\Delta t^2$): **her adımda enerji** $(1+\omega^2\Delta t^2)$ **katına çıkar**. $\omega = 10$, $\Delta t = 0.01$: adım başına %1, 1000 adımda $1.01^{1000} \approx 2 \cdot 10^4$ kat! Yarı-örtük Euler'in matrisi $\begin{bmatrix} 1-\omega^2\Delta t^2 & \omega\Delta t \\ -\omega\Delta t & 1 \end{bmatrix}$, determinantı tam 1: alanı korur, enerji sınırlı kalır (hafifçe salınır ama sürüklenmez).

Sayılar: atalet, RNEA ve integratörler

Atalet: $0.04 \times 0.04 \times 0.20$ m, 0.30 kg uyluk: $\mathbf{I}_c = \mathrm{diag}(1.04,\ 1.04,\ 0.08) \times 10^{-3}$ kg·m^2 (elle: $\frac{0.3}{12}(0.0016+0.04) = 1.04\cdot10^{-3}$); kalçaya taşınınca $I_y = 1.04\cdot10^{-3} + 0.3\cdot0.1^2 = 4.04\times10^{-3}$ (ince çubuk yaklaşımı $m\ell^2/3 = 4.00\times10^{-3}$, %1 fark). **RNEA = Lagrange:** 100 rastgele $(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$'da en büyük tork farkı 2×10^{-16} N·m (C++: 3×10^{-16}). $\mathbf{q} = (0.4, 0.7)$'de $\mathbf{M} = \begin{bmatrix} 0.02079 & 0.00573 \\ 0.00573 & 0.00267 \end{bmatrix}$ hem RNEA sütunlarından hem kapalı formdan (elle örnekle aynı); $\mathbf{g} = (0.4423,\ 0.1749)$ N·m. $\dot{\mathbf{M}} - 2\mathbf{C}$ ters simetri hatası 4×10^{-13} (sonlu fark). **Enerji** (serbest salınan bacak, $\mathbf{q}_0 = (1.2, 0.5)$, 10 s, başlangıç enerjisi -0.2236 J):

Δt	açık Euler	yarı-örtük Euler	RK4
10^{-3}	$+0.291$ J (%130!)	$+0.101$ J	-6.2×10^{-10} J
10^{-4}	$+0.024$ J	$+0.0086$ J	$\sim 10^{-14}$ J

Açık Euler 1 ms adımla sisteme başlangıç enerjisinin büyüklüğünden fazla enerji pompalıyor; Δt 10 kat küçülünce hata da yaklaşık 10 kat küçülüyor (birinci derece). Yarı-örtük Euler burada daha iyi ama sıfır değil ($\mathbf{M}(\mathbf{q})$ kuplajı nedeniyle harmonik osilatördeki garanti tam geçerli değil). RK4 aynı adımda 10^9 kat daha doğru. **Hız:** Python RNEA çağrı başına ~ 50 µs, C++ ~ 0.05 µs: yaklaşık **1000 kat**. 1 kHz'lik bir kontrol döngüsünde 12 eklemli bir robot için onlarca dinamik çağrı yapılır; gerçek zamanlı kontrol kodunun C++ olmasının sebebi budur.

Python

```python
import numpy as np, time
np.set_printoptions(precision=5, suppress=True)
g = 9.81

# (1) Atalet: kutu şeklinde uyluk (0.04 x 0.04 x 0.20 m, 0.30 kg), kütle merkezi etrafında ve kalça eksenine göre
m, a, b, c = 0.30, 0.04, 0.04, 0.20                    # kütle, x, y, z boyutları
Ic = m/12*np.diag([b*b + c*c, a*a + c*c, a*a + b*b])   # kutu: I_xx = m(b²+c²)/12, ...
r = np.array([0, 0, -0.10])                            # kalça -> kütle merkezi
Ihip = Ic + m*(r@r*np.eye(3) - np.outer(r, r))         # paralel eksen (Steiner) teoremi, tensör hali
print("uyluk kütle merkezinde I_c =", np.diag(Ic), " kg·m²;  kalçada I =", np.diag(Ihip))
print(f"kontrol: ince çubuk yaklaşımı I_y(kalça) = m l²/3 = {m*c*c/3:.6f} (kutu: {Ihip[1,1]:.6f})")

```

```python
# (2) Düzlemsel 2-uzuvlu bacak (kalça sabit): Lagrange ile kapalı form M, C, g
L = np.array([0.2, 0.2]); ms = np.array([0.30, 0.20]); lc = L/2; Is = ms*L**2/12      # düzgün çubuklar
def M_closed(q):
    c2 = np.cos(q[1]); m11 = Is[0] + Is[1] + ms[0]*lc[0]**2 + ms[1]*(L[0]**2 + lc[1]**2 +
    2*L[0]*lc[1]*c2)
    m12 = Is[1] + ms[1]*(lc[1]**2 + L[0]*lc[1]*c2); m22 = Is[1] + ms[1]*lc[1]**2
    return np.array([[m11, m12], [m12, m22]])
def C_closed(q, qd):
    h = -ms[1]*L[0]*lc[1]*np.sin(q[1])
    return np.array([[h*qd[1], h*(qd[0] + qd[1])], [-h*qd[0], 0.0]])
def g_closed(q):
    return np.array([ms[0]*g*lc[0]*np.sin(q[0]) + ms[1]*g*(L[0]*np.sin(q[0]) + lc[1]*np.sin(q[0] +
    q[1])),
                     ms[1]*g*lc[1]*np.sin(q[0] + q[1])])

# (3) RNEA: düzlemsel, n uzuvlu seri zincir (açılar dikey-aşağıdan; x = Σ l sin a, z = -Σ l cos a)
def rnea(q, qd, qdd, grav=g):
    n = len(q); a = np.cumsum(q); w = np.cumsum(qd); al = np.cumsum(qdd)
    u = np.column_stack([np.sin(a), -np.cos(a)])                # uzuv yön vektörleri
    perp = lambda v: np.array([-v[1], v[0]])                    # z-ekseni etrafında 90°: ω x r
    (düzlemde)
    acc_p = np.array([0.0, grav])                               # yerçekimi hilesi: tabanı +g ile
    yukarı ivmelendir
    ac = []; ap = [acc_p]
    for i in range(n):                                          # ileri geçiş: hız/ivmeler
        rc = lc[i]*u[i]; rl = L[i]*u[i]
        ac.append(ap[i] + al[i]*perp(rc) - w[i]**2*rc)           # kütle merkezi ivmesi
        ap.append(ap[i] + al[i]*perp(rl) - w[i]**2*rl)           # bir sonraki eklemin ivmesi
    tau = np.zeros(n); f_next = np.zeros(2); n_next = 0.0
    cross = lambda r_, f_: r_[0]*f_[1] - r_[1]*f_[0]
    for i in reversed(range(n)):                                # geri geçiş: kuvvet/momentler
        f = ms[i]*ac[i] + f_next
        nm = Is[i]*al[i] + cross(lc[i]*u[i], ms[i]*ac[i]) + n_next + cross(L[i]*u[i], f_next)
        tau[i] = nm; f_next, n_next = f, nm
    return tau

rng = np.random.default_rng(1); mx = 0
for _ in range(100):
    q, qd, qdd = rng.normal(size=2), rng.normal(size=2), rng.normal(size=2)
    tau_l = M_closed(q)@qdd + C_closed(q, qd)@qd + g_closed(q)
    mx = max(mx, np.abs(rnea(q, qd, qdd) - tau_l).max())
print(f"\nRNEA vs Lagrange (100 rastgele durum): max tork farkı {mx:.2e} N·m")
q = np.array([0.4, 0.7]); qd = np.array([1.0, -2.0])
Mr = np.column_stack([rnea(q, np.zeros(2), e, grav=0) for e in np.eye(2)])       # CRBA'nın en basit
hali: M'nin sütunları
print("M (RNEA sütunlarıyla) =\n", Mr, "\nM (kapalı form) =\n", M_closed(q))
print("g(q) = RNEA(q,0,0) =", rnea(q, np.zeros(2), np.zeros(2)), " kapalı form:", g_closed(q))
hq = 1e-6; Mdot = (M_closed(q + hq*qd) - M_closed(q - hq*qd))/(2*hq); N = Mdot - 2*C_closed(q, qd)
print("Ṁ - 2C ters simetrik mi? (N + Nᵀ) max:", np.abs(N + N.T).max())

# (4) İleri dinamik ve integratörler: serbest salınan bacak (τ = 0), enerji korunumu
def fwd(q, qd, tau): return np.linalg.solve(M_closed(q), tau - rnea(q, qd, np.zeros(2)))     # q¨ =
M⁻¹(τ - h)
def energy(q, qd):
    a = np.cumsum(q); z1 = -lc[0]*np.cos(a[0]); z2 = -L[0]*np.cos(a[0]) - lc[1]*np.cos(a[1])
    return 0.5*qd@M_closed(q)@qd + g*(ms[0]*z1 + ms[1]*z2)
def simulate(method, dt, T=10.0):
    q, qd = np.array([1.2, 0.5]), np.zeros(2); E0 = energy(q, qd)
    for _ in range(int(round(T/dt))):
        if method == "euler":
```

```
            qdd = fwd(q, qd, 0); q, qd = q + dt*qd, qd + dt*qdd
        elif method == "semi":
            qd = qd + dt*fwd(q, qd, 0); q = q + dt*qd
        else:
            f = lambda s: np.concatenate([s[2:], fwd(s[:2], s[2:], 0)]); s = np.concatenate([q, qd])
            k1 = f(s); k2 = f(s + dt/2*k1); k3 = f(s + dt/2*k2); k4 = f(s + dt*k3); s = s + dt/6*(k1 +
            2*k2 + 2*k3 + k4); q, qd = s[:2], s[2:]
    return energy(q, qd) - E0
print("\nserbest bacak, q0 =(1.2, 0.5), 10 s, enerji hatası [J] (başlangıç enerjisi",
round(energy(np.array([1.2, .5]), np.zeros(2)), 5), "J):")
for dt in [1e-3, 1e-4]:
    print(f"  dt ={dt:.0e}: açık Euler {simulate('euler', dt):+.3e}, yarı-örtük Euler {simulate('semi',
    dt):+.3e}, RK4 {simulate('rk4', dt):+.3e}")

# (5) Hız: RNEA çağrısı başına süre
q, qd, qdd = np.array([0.3, 0.4]), np.array([0.1, 0.2]), np.array([0.5, 0.6]); n_call = 20000
t0 = time.perf_counter()
for _ in range(n_call): rnea(q, qd, qdd)
print(f"\nPython RNEA (2 uzuv): {1e6*(time.perf_counter() - t0)/n_call:.1f} µs/çağrı")
```

Çıktı:

```
uyluk kütle merkezinde I_c = [0.00104 0.00104 0.00008]  kg·m²;  kalçada I = [0.00404 0.00404 0.00008]
kontrol: ince çubuk yaklaşımı I_y(kalça) = m l²/3 = 0.004000 (kutu: 0.004040)

RNEA vs Lagrange (100 rastgele durum): max tork farkı 2.22e-16 N·m
M (RNEA sütunlarıyla) =
 [[0.02079 0.00573]
 [0.00573 0.00267]]
M (kapalı form) =
 [[0.02079 0.00573]
 [0.00573 0.00267]]
g(q) = RNEA(q,0,0) = [0.44227 0.17485]  kapalı form: [0.44227 0.17485]
Ṁ - 2C ters simetrik mi? (N + Nᵀ) max: 4.0029438097555214e-13

serbest bacak, q0 =(1.2, 0.5), 10 s, enerji hatası [J] (başlangıç enerjisi -0.22355 J):
  dt =1e-03: açık Euler +2.907e-01, yarı-örtük Euler +1.012e-01, RK4 -6.173e-10
  dt =1e-04: açık Euler +2.438e-02, yarı-örtük Euler +8.628e-03, RK4 -1.693e-14

Python RNEA (2 uzuv): 47.9 µs/çağrı
```

C++

```
// c06_dyn.cpp — düzlemsel RNEA, Lagrange ile karşılaştırma, RK4 enerji korunumu, hız ölçümü
#include <Eigen/Dense>
#include <chrono>
#include <cmath>
#include <cstdio>
#include <random>
using Eigen::Vector2d; using Eigen::Matrix2d;

constexpr double g = 9.81;
constexpr double L[2] = {0.2, 0.2}, ms[2] = {0.30, 0.20};
constexpr double lc[2] = {0.1, 0.1}, Is[2] = {0.30 * 0.04 / 12, 0.20 * 0.04 / 12};

inline Vector2d perp(const Vector2d& v) { return Vector2d(-v.y(), v.x()); }
inline double cross2(const Vector2d& r, const Vector2d& f) { return r.x() * f.y() - r.y() * f.x(); }

Vector2d rnea(const Vector2d& q, const Vector2d& qd, const Vector2d& qdd, double grav = g) {
    double a[2] = {q(0), q(0) + q(1)}, w[2] = {qd(0), qd(0) + qd(1)}, al[2] = {qdd(0), qdd(0) + qdd(1)};
```

```
    Vector2d u[2], ac[2], ap(0.0, grav);
    for (int i = 0; i < 2; ++i) {                                   // ileri geçiş
        u[i] = Vector2d(std::sin(a[i]), -std::cos(a[i]));
        Vector2d rc = lc[i] * u[i], rl = L[i] * u[i];
        ac[i] = ap + al[i] * perp(rc) - w[i] * w[i] * rc;
        ap = ap + al[i] * perp(rl) - w[i] * w[i] * rl;
    }
    Vector2d tau, f_next = Vector2d::Zero(); double n_next = 0;
    for (int i = 1; i >= 0; --i) {                                  // geri geçiş
        Vector2d f = ms[i] * ac[i] + f_next;
        double nm = Is[i] * al[i] + cross2(lc[i] * u[i], ms[i] * ac[i]) + n_next + cross2(L[i] * u[i], f_next);
        tau(i) = nm; f_next = f; n_next = nm;
    }
    return tau;
}
Matrix2d M_closed(const Vector2d& q) {
    double c2 = std::cos(q(1)), m11 = Is[0] + Is[1] + ms[0] * lc[0] * lc[0] + ms[1] * (L[0] * L[0] + lc[1] * lc[1] + 2 * L[0] * lc[1] * c2);
    double m12 = Is[1] + ms[1] * (lc[1] * lc[1] + L[0] * lc[1] * c2), m22 = Is[1] + ms[1] * lc[1] * lc[1];
    Matrix2d M; M << m11, m12, m12, m22; return M;
}
Vector2d lagrange_tau(const Vector2d& q, const Vector2d& qd, const Vector2d& qdd) {
    double h = -ms[1] * L[0] * lc[1] * std::sin(q(1));
    Matrix2d C; C << h * qd(1), h * (qd(0) + qd(1)), -h * qd(0), 0;
    Vector2d gv(ms[0] * g * lc[0] * std::sin(q(0)) + ms[1] * g * (L[0] * std::sin(q(0)) + lc[1] * std::sin(q(0) + q(1))),
                ms[1] * g * lc[1] * std::sin(q(0) + q(1)));
    return M_closed(q) * qdd + C * qd + gv;
}
double energy(const Vector2d& q, const Vector2d& qd) {
    double a1 = q(0), a2 = q(0) + q(1);
    return 0.5 * qd.dot(M_closed(q) * qd) + g * (ms[0] * (-lc[0] * std::cos(a1)) + ms[1] * (-L[0] * std::cos(a1) - lc[1] * std::cos(a2)));
}
using State = Eigen::Vector4d;
State f(const State& s) {
    Vector2d q = s.head<2>(), qd = s.tail<2>();
    Vector2d qdd = M_closed(q).ldlt().solve(-rnea(q, qd, Vector2d::Zero()));   // τ = 0
    State ds; ds << qd, qdd; return ds;
}

int main() {
    std::mt19937 gen(1); std::normal_distribution<double> N01;
    double mx = 0;
    for (int k = 0; k < 100; ++k) {
        Vector2d q(N01(gen), N01(gen)), qd(N01(gen), N01(gen)), qdd(N01(gen), N01(gen));
        mx = std::max(mx, (rnea(q, qd, qdd) - lagrange_tau(q, qd, qdd)).cwiseAbs().maxCoeff());
    }
    std::printf("RNEA vs Lagrange (100 rastgele durum): max fark %.2e N·m\n", mx);

    for (double dt : {1e-3, 1e-4}) {
        State s; s << 1.2, 0.5, 0, 0; double E0 = energy(s.head<2>(), s.tail<2>());
        int n = (int)std::lround(10.0 / dt);
        for (int k = 0; k < n; ++k) {
            State k1 = f(s), k2 = f(s + dt / 2 * k1), k3 = f(s + dt / 2 * k2), k4 = f(s + dt * k3);
            s += dt / 6 * (k1 + 2 * k2 + 2 * k3 + k4);
        }
       std::printf("RK4 dt =%.0e: enerji hatası %+.3e J\n", dt, energy(s.head<2>(), s.tail<2>()) - E0);
```

```cpp
    }

    Vector2d q(0.3, 0.4), qd(0.1, 0.2), qdd(0.5, 0.6), acc = Vector2d::Zero();
    const int n_call = 2000000;
    auto t0 = std::chrono::steady_clock::now();
    for (int k = 0; k < n_call; ++k) { qdd(0) += 1e-12; acc += rnea(q, qd, qdd); }   // derleyici döngüyü silmesin diye
    double us = std::chrono::duration<double, std::micro>(std::chrono::steady_clock::now() - t0).count() / n_call;
    std::printf("C++ RNEA (2 uzuv): %.3f µs/çağrı (kontrol toplamı %.3f)\n", us, acc(0));
    return 0;
}
```

Çıktı:

```
RNEA vs Lagrange (100 rastgele durum): max fark 3.33e-16 N·m
RK4 dt =1e-03: enerji hatası -6.173e-10 J
RK4 dt =1e-04: enerji hatası +3.580e-15 J
C++ RNEA (2 uzuv): 0.052 µs/çağrı (kontrol toplamı 688065.536)
```

Python ve C++ notu: performans ölçmek

- **Zaman ölçümü:** Python `time.perf_counter()`; C++ `std::chrono::steady_clock::now()` ve `std::chrono::duration<double, std::micro>(t1 - t0).count()`. `system_clock` kullanma (saat ayarlanınca geri gidebilir).
- **Ölü kod eleme tuzağı:** C++ derleyicisi (`-O2`) sonucu kullanılmayan bir hesabı tamamen silebilir ve ölçüm "0 µs" çıkar. Bu yüzden sonuçları bir toplamda biriktirip yazdırdım ve girdiyi her turda biraz değiştirdim.
- **Koddaki RNEA ve türetme:** `ac.append(ap[i] + al[i]*perp(rc) - w[i]**2*rc)` satırı $\mathbf{a} = \mathbf{a}_p + \alpha\mathbf{r}^{\perp} - \omega^2\mathbf{r}$; geri geçişteki `f = ms[i]*ac[i] + f_next` Newton denklemi; `acc_p = [0, grav]` yerçekimi hilesi.
- **`ldlt()`:** $\mathbf{M}$ simetrik pozitif tanımlı olduğundan (kinetik enerji her hızda pozitif) Cholesky türü çözücü en uygunu; `inverse()` hesaplama.

Alıştırma

(Kâğıt) (a) İnce bir çubuğun $(m,\ \ell)$ ucuna göre ataletini integralle ($\int_0^{\ell} x^2 \frac{m}{\ell} dx$) bul ve Steiner ile karşılaştır. **(b)** Tek sarkaçta θ küçükken $\sin\theta \approx \theta$ ile salınım frekansını bul ($\omega = \sqrt{g/\ell}$); $\ell = 0.4$ m için periyot? **(c)** 2 uzuvlu bacakta $q_1 = 0$, $q_2 = \pi/2$ için $\mathbf{M}$'yi elle hesapla. **(d)** $\mathbf{q} = (0, 0)$'da (bacak dik aşağı) $\mathbf{g}$ neden sıfır? $\mathbf{q} = (\pi/2, 0)$'da (bacak yatay) g_1'i hesapla ve "ağırlık × moment kolu" ile sağla. **(e)** $c_{122} = h$'yi Christoffel formülünden adım adım hesapla. **(f)** Harmonik osilatörde ($\omega = 5$, $\Delta t = 0.02$) açık Euler ile 100 adım sonra enerji kaç katına çıkar? **(Bilgisayar) (g)** RNEA'yı 3 uzuva genişlet; $\mathbf{M}$'nin simetrik ve pozitif tanımlı olduğunu 1000 rastgele $\mathbf{q}$'da doğrula. **(h)** Bacağa PD + yerçekimi kompanzasyonu uygula: $\boldsymbol{\tau} = \mathbf{g}(\mathbf{q}) + \mathbf{K}_p(\mathbf{q}_d - \mathbf{q}) - \mathbf{K}_d\dot{\mathbf{q}}$; RK4 ile simüle et, $\mathbf{q}_d$'ye yakınsadığını göster (Cilt 4A Bölüm 5). **(i)** Enerji hatasını Δt'ye karşı log-log çiz; eğimlerden Euler'in 1., RK4'ün 4. dereceden olduğunu doğrula.

Bölüm 7

Yüzen Taban, Kütle Merkezi ve Temas Kısıtları

Şimdiye kadar bacağın kalçası sabitti (sabit tabanlı kol gibi). Gerçek bir yürüyen robotun gövdesi hiçbir yere bağlı değildir: **yüzen taban** (floating base). Bu bölüm, bacaklı robotları underactuated yapan yapıyı *tek bir denklemde* türetiyor ve Cilt 4–6'daki her şeyin (ZMP, WBC, MPC) temelini kuruyor. Kaynak: ETH Robot Dynamics notlarının floating base bölümü; Featherstone Bölüm 9.

7.1 Genelleştirilmiş koordinatlar ve seçim matrisi

7.1.1 Koordinatlar

Düzlemsel tek bacaklı model (SB-1'in yan görünüşteki yarısı): gövde (1.2 kg, 0.3 m), uyluk (0.3 kg), baldır (0.2 kg), ayak (0.1 kg, topuk ayak bileğinin 3 cm arkasında, burun 7 cm önünde). Robotun konfigürasyonunu tam olarak tarif etmek için önce gövdenin dünyadaki yerini ve açısını, sonra eklem açılarını vermek gerekir:

$$\mathbf{q} = [\underbrace{x,\ z,\ \theta_b}_{\text{yüzen taban: motor yok}} \mid \underbrace{q_{\text{kalça}},\ q_{\text{diz}},\ q_{\text{bilek}}}_{\text{motorlu eklemler}}] \in \mathbb{R}^6.$$

3-B'de taban 6 serbestliktir (konum + yönelim, genelde kuaterniyonla; MuJoCo'da `qpos[0:7]`, 4 sayıyla 3 serbestlik).

7.1.2 Hareket denklemi ve seçim matrisi

Bölüm 6'nın Lagrange türetmesi aynen geçerli: $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{h} = \boldsymbol{\tau}_{\text{gen}}$; sadece genelleştirilmiş kuvvetleri doğru yazmak gerekir. İki kaynak var:

- **Motorlar** yalnızca eklem koordinatlarına kuvvet verir; gövdenin x, z, θ_b'sini doğrudan iten bir motor yok. Bunu **seçim matrisi** $\mathbf{S} = [\mathbf{0}_{3\times 3} \quad \mathbf{I}_3]$ ile yazarız: motor torkları $\boldsymbol{\tau} \in \mathbb{R}^3$, genelleştirilmiş kuvvet $\mathbf{S}^T\boldsymbol{\tau} = (0, 0, 0, \tau_1, \tau_2, \tau_3)$.
- **Temas kuvvetleri** $\boldsymbol{\lambda}$ (yerin ayağa uyguladığı). Bir noktaya uygulanan kuvvetin genelleştirilmiş kuvvete katkısı, Bölüm 4'teki sanal iş argümanıyla, o noktanın Jacobian'ının transpozudur: $\mathbf{J}_c^T\boldsymbol{\lambda}$ ($\mathbf{J}_c$: temas noktalarının konumunun $\mathbf{q}$'ya göre türevi).

$$\boxed{\mathbf{M}\ddot{\mathbf{q}} + \mathbf{h} = \mathbf{S}^T\boldsymbol{\tau} + \mathbf{J}_c^T\boldsymbol{\lambda}}$$

Fiziksel anlam: denklemin ilk üç satırı (taban) *hiç motor torku içermez*. Gövdeyi hareket ettirebilecek tek dış etki temas kuvvetleridir: **gövdenin ve kütle merkezinin hareketi sadece yerden gelen kuvvetlerle değişebilir.** Havadaki bir robot ne yaparsa yapsın kütle merkezi parabol çizer. "Underactuated" kelimesinin bacaklı robotlardaki karşılığı tam olarak budur: 6 serbestlik, 3 motor.

7.2 Kütle matrisini Jacobian'larla kurmak

7.2.1 Türetme

Her cisim k için kütle merkezi hızı $\dot{\mathbf{c}}_k = \mathbf{J}_{v,k}\dot{\mathbf{q}}$ ve açısal hızı $\omega_k = \mathbf{J}_{\omega,k}\dot{\mathbf{q}}$ (Jacobian'lar, Bölüm 1'deki gibi konumun $\mathbf{q}$'ya göre türevi). Bölüm 6'daki kinetik enerji formülünü (öteleme + dönme) her cisim için yazıp topla:

$$\begin{aligned} T &= \frac{1}{2}\sum_k \left(m_k \|\mathbf{J}_{v,k}\dot{\mathbf{q}}\|^2 + I_k(\mathbf{J}_{\omega,k}\dot{\mathbf{q}})^2\right) \\ &= \frac{1}{2}\dot{\mathbf{q}}^T\Big[\sum_k \left(m_k \mathbf{J}_{v,k}^T\mathbf{J}_{v,k} + I_k\mathbf{J}_{\omega,k}^T\mathbf{J}_{\omega,k}\right)\Big]\dot{\mathbf{q}}. \end{aligned}$$

$T = \frac{1}{2}\dot{\mathbf{q}}^T\mathbf{M}\dot{\mathbf{q}}$ ile karşılaştırınca:

$$\mathbf{M}(\mathbf{q}) = \sum_k \left(m_k \mathbf{J}_{v,k}^T\mathbf{J}_{v,k} + I_k\mathbf{J}_{\omega,k}^T\mathbf{J}_{\omega,k}\right),$$

$$\mathbf{g}(\mathbf{q}) = \frac{\partial V}{\partial \mathbf{q}}, \qquad \mathbf{h} = \dot{\mathbf{M}}\dot{\mathbf{q}} - \tfrac{1}{2}\frac{\partial}{\partial \mathbf{q}}(\dot{\mathbf{q}}^T\mathbf{M}\dot{\mathbf{q}}) + \mathbf{g}.$$

Bu formül her robota aynen uygulanır ve $\mathbf{M}$'nin neden simetrik ve pozitif (yarı) tanımlı olduğunu açıkça gösterir: $\mathbf{J}^T\mathbf{J}$ biçimindeki terimlerin toplamı.

$M_{xx} = m_{\textbf{toplam}}$ **neden?** Gövdeyi x'te kaydırmak ($\dot{x} = 1$, diğerleri 0) her cismi aynı miktarda kaydırır: her $\mathbf{J}_{v,k}$'nın x sütunu $(1, 0)$, $\mathbf{J}_{\omega,k}$'nınki 0. Formülde $(1,1)$ elemanı $\sum_k m_k \cdot 1 = m_{\text{toplam}}$. Aynı şekilde g_z (taban z satırı) $= \partial V/\partial z = \sum m_k g$: toplam ağırlık.

Bu bölümdeki kod Jacobian'ları *sayısal türevle* alıyor: yavaş ama her modele aynen uygulanır ve Bölüm 6'daki analitik formüllerle kolayca test edilir. Gerçek uygulamada (Pinocchio, MuJoCo) RNEA/CRBA'nın 3-B halleri kullanılır.

7.3 Temas kısıtları ve KKT sistemi

7.3.1 Kısıttan ivme denklemine

Yerdeki ayak noktasının konumu $\boldsymbol{\phi}(\mathbf{q})$ sabit kalmalı: $\boldsymbol{\phi}(\mathbf{q}) =$ sabit. Zamana göre türev (zincir kuralı):

$$\dot{\boldsymbol{\phi}} = \mathbf{J}_c\dot{\mathbf{q}} = \mathbf{0}, \qquad \ddot{\boldsymbol{\phi}} = \mathbf{J}_c\ddot{\mathbf{q}} + \dot{\mathbf{J}}_c\dot{\mathbf{q}} = \mathbf{0}.$$

Hareket denklemiyle birlikte $\ddot{\mathbf{q}}$ ve $\boldsymbol{\lambda}$ bilinmeyen; ikisini tek doğrusal sistemde yaz:

$$\begin{bmatrix} \mathbf{M} & -\mathbf{J}_c^T \\ \mathbf{J}_c & \mathbf{0} \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{q}} \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} \mathbf{S}^T\boldsymbol{\tau} - \mathbf{h} \\ -\dot{\mathbf{J}}_c\dot{\mathbf{q}} \end{bmatrix}.$$

Bu bir **KKT sistemi**: kısıtlı optimizasyondaki Karush–Kuhn–Tucker koşullarıyla aynı yapı. Bağlantı: Gauss'un en az kısıt ilkesine göre doğa, kısıtı sağlayan ivmeler arasından $\frac{1}{2}(\ddot{\mathbf{q}} - \ddot{\mathbf{q}}_{\text{serbest}})^T\mathbf{M}(\ddot{\mathbf{q}} - \ddot{\mathbf{q}}_{\text{serbest}})$'yi en aza indireni seçer; $\boldsymbol{\lambda}$ bu problemin Lagrange çarpanıdır (Bölüm 1'deki pseudo-inverse türetmesinin aynı kalıbı).

Dikkat: kısıtlar bağımsız olmalı. Rijit bir ayağın topuk ve burnunun (x, z)'si 4 denklem verir ama topuk-burun mesafesi sabit olduğundan biri gereksizdir; $\mathbf{J}_c$'nin satırları doğrusal bağımlı, matris tekil olur (ilk denememde `LinAlgError: Singular matrix` aldım). Topuk (x, z) + burun $(z) = 3$ bağımsız kısıt.

7.3.2 Zemin sadece iter

En önemli fiziksel koşul: $\lambda_z \geq 0$ her temas noktasında (ve sürtünme konisi $|\lambda_x| \leq \mu\lambda_z$). KKT çözümü bu eşitsizliği bilmez; $\lambda_z < 0$ çıkarsa "ayak yere yapışık kalsaydı böyle olurdu, ama gerçekte ayak kalkar" demektir.

7.3.3 Statik denge: basınç merkezi = kütle merkezi

Statik durumda ($\dot{\mathbf{q}} = \ddot{\mathbf{q}} = \mathbf{0}$) hareket denklemi $\mathbf{g} = \mathbf{S}^T\boldsymbol{\tau} + \mathbf{J}_c^T\boldsymbol{\lambda}$ olur. İlk üç satır (taban) torksuzdur: $\mathbf{g}_{\text{taban}} = \mathbf{J}_{c,\text{taban}}^T\boldsymbol{\lambda}$. Bu satırları açık yazalım. Taban x'i kaydırınca her temas noktası x'te aynı miktarda kayar, taban z için aynısı; taban θ_b'yi döndürünce temas noktası i, $(-(z_i - z_b),\ x_i - x_b)$ yönünde (dönme yarıçapına dik) hareket eder:

$$\begin{aligned} x \text{ satırı:} &\quad 0 = \textstyle\sum_i \lambda_{x,i} \qquad \text{(yatayda yerçekimi yok)}, \\ z \text{ satırı:} &\quad mg = \textstyle\sum_i \lambda_{z,i}, \\ \theta_b \text{ satırı:} &\quad mg\,(x_c - x_b) = \textstyle\sum_i \left[(x_i - x_b)\lambda_{z,i} - (z_i - z_b)\lambda_{x,i}\right]. \end{aligned}$$

Üçüncüsü gövde noktası etrafında moment dengesi. Bizim kısıt seçimimizde sadece topukta yatay kuvvet var ve x satırı onu sıfır yapıyor; temas noktaları yerde ($z_i = 0$). İkinci ve üçüncü satırdan:

$$x_{\text{CoP}} \equiv \frac{\sum_i x_i \lambda_{z,i}}{\sum_i \lambda_{z,i}} = x_c.$$

Statik denge = basınç merkezi destek alanının içinde

Tüm $\lambda_z \geq 0 \Leftrightarrow$ CoP topuk ile burun arasında (ağırlıklı ortalama, ağırlıklar pozitifse uçların arasında kalır) $\Leftrightarrow$ kütle merkezi destek alanının üstünde. Eklem torkları bu üç satırda hiç görünmüyor: eklemler ne kadar güçlü olursa olsun bu koşulu değiştiremez. Bu, underactuation'ın en somut hâlidir ve Cilt 6'daki ZMP kriterinin statik özel durumudur.

Elle: iki temas noktasında yük paylaşımı

Topuk $x_h = -0.03$, burun $x_t = 0.07$ m, kütle merkezi $x_c = -0.0071$, $mg = 1.8 \cdot 9.81 = 17.658$ N. Topuk etrafında moment: $\lambda_t(x_t - x_h) = mg(x_c - x_h)$, yani

$$\lambda_t = 17.658 \cdot \frac{-0.0071 + 0.03}{0.10} = 4.044 \text{ N},$$
$$\lambda_h = 17.658 - 4.044 = 13.614 \text{ N}.$$

Kütle merkezi topuğa daha yakın, yükün çoğu topukta. $x_c < x_h$ olsaydı formül $\lambda_t < 0$ verirdi: devrilme. Aşağıdaki tabloyla birebir aynı: tüm bu karmaşık 6×6 makine, statik durumda bir kaldıraç hesabına indirgeniyor.

Sayılar: yüzen tabanlı tek bacak

Toplam kütle 1.8 kg: $M_{xx} = M_{zz} = 1.8000$; $g_z = 17.658$ N $= 1.8 \cdot 9.81$ (türetmedeki gibi). Kalça -0.3, diz 0.6 rad, ayak tabanı yere paralel:

Gövde	KM x [m]	λ_z topuk [N]	λ_z burun [N]	sonuç
dik ($\theta_b = 0$)	-0.0071	13.614	4.044	dengede; $\boldsymbol{\tau} = (0,\ -0.783,\ 0.145)$ N·m
geriye eğik ($+0.35$)	-0.1483	38.541	-20.883	devrilir (KM topuğun arkasında)
öne eğik (-0.35)	$+0.1351$	-11.489	29.147	devrilir (KM burnun önünde)

Destek alanı $x \in [-0.03,\ 0.07]$ m; her durumda CoP = KM x ve $\sum \lambda_z = 17.658$ N. Geriye eğik satırı elle: $\lambda_t = 17.658(-0.1483+0.03)/0.1 = -20.89$. Python ve C++ birebir aynı. Dik duruşta kalça torku sıfır: gövdenin ağırlık merkezi kalçanın tam üstünde. **KKT simülasyonu** (0.3 s, $\Delta t = 2$ ms): dik duruşun torklarıyla robot kıpırdamıyor (gövde açısı 0.0000), temas kuvvetleri sabit. Kalçaya sadece $+0.4$ N·m fazla tork verince gövde geriye dönüyor ve $t = 0.084$ s'de bir λ_z negatif oluyor: gerçek robotta ayak bu anda kalkardı ve modelimiz (ayak yere yapışık varsayımı) geçersizleşirdi. Bu yüzden bacaklı robot kontrolcüleri temas kuvvetlerini *açıkça* hesaplar ve $\lambda_z \geq 0$, sürtünme konisi gibi kısıtları bir QP'de zorlar (Cilt 6 Bölüm 4: WBC).

Python

```python
import numpy as np
np.set_printoptions(precision=4, suppress=True)
g = 9.81
# Düzlemsel tek bacaklı "SB-1 yarısı": gövde + uyluk + baldır + ayak. Genelleştirilmiş koordinatlar:
# q = [x, z, θ_b | q_kalça, q_diz, q_bilek]  (ilk 3: yüzen taban, aktüatörsüz; son 3: motorlu eklemler)
m_t, m_th, m_sh, m_ft = 1.2, 0.3, 0.2, 0.1                    # kütleler [kg]
L_t, L_th, L_sh = 0.30, 0.20, 0.20; HEEL, TOE, FCOM = -0.03, 0.07, 0.02
I_t, I_th, I_sh, I_ft = m_t*L_t**2/12, m_th*L_th**2/12, m_sh*L_sh**2/12, m_ft*0.10**2/12
down = lambda p: np.array([np.sin(p), -np.cos(p)])          # bacak uzvu yönü (φ =0: aşağı)
fwd = lambda p: np.array([np.cos(p), np.sin(p)])            # ayak yönü (φ =0: yatay ileri)

def bodies(q):
    """Her cisim için (kütle, atalet, kütle merkezi konumu, mutlak açı) ve temas noktaları"""
    x, z, th = q[:3]; hip = np.array([x, z])
    ph1 = th + q[3]; ph2 = ph1 + q[4]; ph3 = ph2 + q[5]
    knee = hip + L_th*down(ph1); ankle = knee + L_sh*down(ph2)
    B = [(m_t, I_t, hip + 0.5*L_t*np.array([-np.sin(th), np.cos(th)]), th),
         (m_th, I_th, hip + 0.5*L_th*down(ph1), ph1),
         (m_sh, I_sh, knee + 0.5*L_sh*down(ph2), ph2),
         (m_ft, I_ft, ankle + FCOM*fwd(ph3), ph3)]
    contacts = [ankle + HEEL*fwd(ph3), ankle + TOE*fwd(ph3)]
    return B, contacts

def jac(fun, q, h=1e-7):
    """f(q)'nin q'ya göre Jacobian'ı (merkezi sonlu fark). f vektör döndürür."""
    return np.column_stack([(fun(q + h*e) - fun(q - h*e))/(2*h) for e in np.eye(len(q))])

def mass_matrix(q):
    """M = Σ m J_vᵀJ_v + I J_ωᵀJ_ω  (her cismin kütle merkezi doğrusal ve açısal hız Jacobian'ları)"""
    M = np.zeros((6, 6))
    for k in range(4):
        Jv = jac(lambda s: bodies(s)[0][k][2], q); Jw = jac(lambda s: np.array([bodies(s)[0][k][3]]), q)
        m, I = bodies(q)[0][k][:2]; M += m*Jv.T@Jv + I*Jw.T@Jw
    return M
def potential(q): return sum(b[0]*g*b[2][1] for b in bodies(q)[0])
def gravity(q): return jac(lambda s: np.array([potential(s)]), q)[0]                # g(q) = ∂V/∂q
def Jc(q): return jac(lambda s: np.concatenate([bodies(s)[1][0], bodies(s)[1][1][1:]]), q)   # 3x6: topuk (x,z), burun (z)
```

```python
# Not: burun x'i eklemek 4. bir kısıt olurdu ama rijit ayakta topuk-burun mesafesi sabit -> bağımlı (rank 3), KKT tekil olur
def com(q): B = bodies(q)[0]; return sum(b[0]*b[2] for b in B)/sum(b[0] for b in B)
def bias(q, qd, h=1e-6):
    """h(q,q̇) = C q̇ + g = Ṁq̇ - ½ ∂(q̇ᵀMq̇)/∂q + g  (sonlu farklarla)"""
    Md = (mass_matrix(q + h*qd) - mass_matrix(q - h*qd))/(2*h)
    dT = np.array([(qd@mass_matrix(q + h*e)@qd - qd@mass_matrix(q - h*e)@qd)/(2*h) for e in np.eye(6)])
    return Md@qd - 0.5*dT + gravity(q)

def standing_q(th_b, q_hip, q_knee):
    """Ayak tabanı yere paralel ve zeminde (z =0), ayak bileği x =0 olacak şekilde taban konumunu hesapla"""
    q_ank = -(th_b + q_hip + q_knee); q = np.array([0, 0, th_b, q_hip, q_knee, q_ank])
    ankle = bodies(q)[1][0] - HEEL*fwd(0.0); q[0] -= ankle[0]; q[1] -= ankle[1]; return q

S = np.hstack([np.zeros((3, 3)), np.eye(3)])                 # seçim matrisi: τ sadece son 3 koordinata etki eder
mtot = m_t + m_th + m_sh + m_ft
for label, (thb, qh, qk) in [("dik duruş", (0.0, -0.3, 0.6)), ("gövde geriye eğik θ_b =+0.35", (0.35, -0.3, 0.6)), ("gövde öne eğik θ_b =-0.35", (-0.35, -0.3, 0.6))]:
    q = standing_q(thb, qh, qk); M = mass_matrix(q); G = gravity(q); J = Jc(q); c = com(q)
    if label == "dik duruş":
        print("M (6x6) =\n", M); print(f"M[0,0] = M[1,1] = toplam kütle? {M[0,0]:.4f}, {M[1,1]:.4f} (gerçek {mtot})")
        print("g(q) =", G, f"  g_z = {G[1]:.4f} = m_toplam·g = {mtot*g:.4f}")
    # Statik denge: g(q) = Sᵀτ + Jcᵀλ.  İlk 3 satır (taban) τ içermez: g_base = Jc_baseᵀ λ (underactuation!)
    Jb = J[:, :3].T                                          # 3x3
    lam = np.linalg.solve(Jb, G[:3])                         # λ = [topuk_x, topuk_z, burun_z]: 3 denklem, 3 bilinmeyen
    tau = G[3:] - J[:, 3:].T@lam
    heel, toe = bodies(q)[1]; Fz = lam[1] + lam[2]; cop = (heel[0]*lam[1] + toe[0]*lam[2])/Fz
    ok = lam[1] >= 0 and lam[2] >= 0
    print(f"\n{label}: KM x = {c[0]:+.4f} m, destek [{heel[0]:+.3f}, {toe[0]:+.3f}] m;  λ_z topuk = {lam[1]:+.3f} N, burun = {lam[2]:+.3f} N, Σ = {Fz:.3f} N, λ_x = {lam[0]:+.3f} N")
    print(f"   CoP (basınç merkezi) x = {cop:+.4f} m  (statikte = KM x)  ->  {'DENGEDE (iki kuvvet de iter)' if ok else 'İMKANSIZ: bir temas yeri ÇEKMEK zorunda -> robot devrilir'}")
    print(f"   eklem torkları τ = {tau} N·m")

# Dinamik kontrol: dik duruş torklarını uygula, temas kısıtlı ileri dinamik (KKT) ile 0.3 s simüle et
q = standing_q(0.0, -0.3, 0.6); G = gravity(q); J = Jc(q); lam0 = np.linalg.solve(J[:, :3].T, G[:3]); tau0 = G[3:] - J[:, 3:].T@lam0
def kkt(q, qd, tau):
    M = mass_matrix(q); J = Jc(q); h = bias(q, qd); hh = 1e-6
    Jdqd = (Jc(q + hh*qd) - Jc(q - hh*qd))/(2*hh)@qd          # J̇ q̇
    K = np.block([[M, -J.T], [J, np.zeros((3, 3))]]); rhs = np.concatenate([S.T@tau - h, -Jdqd])
    sol = np.linalg.solve(K, rhs); return sol[:6], sol[6:]
for name, dtau in [("denge torkları", np.zeros(3)), ("kalçaya +0.4 N·m fazla", np.array([0.4, 0, 0]))]:
    qq, qd = q.copy(), np.zeros(6); dt = 0.002; first_neg = None
    for k in range(150):
        qdd, lam = kkt(qq, qd, tau0 + dtau)
        if first_neg is None and min(lam[1], lam[2]) < 0: first_neg = k*dt
        qd = qd + dt*qdd; qq = qq + dt*qd
    print(f"\nKKT simülasyonu ({name}), 0.3 s: gövde açısı {qq[2]:+.4f} rad, λ_z (topuk, burun) = ({lam[1]:+.3f}, {lam[2]:+.3f}) N; "
          + ("temas kuvvetleri hep ≥ 0" if first_neg is None else f"t = {first_neg:.3f} s'de bir λ_z < 0: gerçekte ayak burada kalkardı"))
```

Çıktı:

```
M (6x6) =
 [[ 1.8     0.     -0.0558  0.1242  0.0382  0.    ]
 [ 0.      1.8    -0.0128 -0.0128  0.0138  0.002 ]
 [-0.0558 -0.0128  0.072   0.036   0.0135  0.0001]
 [ 0.1242 -0.0128  0.036   0.036   0.0135  0.0001]
 [ 0.0382  0.0138  0.0135  0.0135  0.007   0.0002]
 [ 0.      0.002   0.0001  0.0001  0.0002  0.0001]]
M[0,0] = M[1,1] = toplam kütle? 1.8000, 1.8000 (gerçek 1.8)
g(q) = [ 0.     17.658  -0.1253 -0.1253  0.1356  0.0196]   g_z = 17.6580 = m_toplam·g = 17.6580

dik duruş: KM x = -0.0071 m, destek [-0.030, +0.070] m;  λ_z topuk = +13.614 N, burun = +4.044 N, Σ = 17.658 N, λ_x
= +0.000 N
   CoP (basınç merkezi) x = -0.0071 m  (statikte = KM x)  ->  DENGEDE (iki kuvvet de iter)
   eklem torkları τ = [ 0.     -0.7827  0.145 ] N·m

gövde geriye eğik θ_b =+0.35: KM x = -0.1483 m, destek [-0.030, +0.070] m;  λ_z topuk = +38.541 N, burun = -20.883
N, Σ = 17.658 N, λ_x = +0.000 N
   CoP (basınç merkezi) x = -0.1483 m  (statikte = KM x)  ->  İMKANSIZ: bir temas yeri ÇEKMEK zorunda -> robot
   devrilir
   eklem torkları τ = [0.6055 0.7379 2.6377] N·m

gövde öne eğik θ_b =-0.35: KM x = +0.1351 m, destek [-0.030, +0.070] m;  λ_z topuk = -11.489 N, burun = +29.147 N, Σ
= 17.658 N, λ_x = +0.000 N
   CoP (basınç merkezi) x = +0.1351 m  (statikte = KM x)  ->  İMKANSIZ: bir temas yeri ÇEKMEK zorunda -> robot
   devrilir
   eklem torkları τ = [-0.6055 -2.2084 -2.3653] N·m

KKT simülasyonu (denge torkları), 0.3 s: gövde açısı -0.0000 rad, λ_z (topuk, burun) = (+13.614, +4.044) N; temas
kuvvetleri hep ≥ 0

KKT simülasyonu (kalçaya +0.4 N·m fazla), 0.3 s: gövde açısı +0.7912 rad, λ_z (topuk, burun) = (-0.178, -1.867) N;
t = 0.084 s'de bir λ_z < 0: gerçekte ayak burada kalkardı
```

C++

```cpp
// c07_float.cpp — yüzen tabanlı düzlemsel bacak: M(q), g(q), statik temas kuvvetleri (Python ile aynı model)
#include <Eigen/Dense>
#include <array>
#include <cmath>
#include <cstdio>
#include <functional>
using Eigen::Vector2d; using Eigen::VectorXd; using Eigen::MatrixXd;
using Vec6 = Eigen::Matrix<double, 6, 1>;

constexpr double g = 9.81, m_t = 1.2, m_th = 0.3, m_sh = 0.2, m_ft = 0.1;
constexpr double L_t = 0.30, L_th = 0.20, L_sh = 0.20, HEEL = -0.03, TOE = 0.07, FCOM = 0.02;
const double I_[4] = {m_t * L_t * L_t / 12, m_th * L_th * L_th / 12, m_sh * L_sh * L_sh / 12, m_ft * 0.01 / 12};
const double m_[4] = {m_t, m_th, m_sh, m_ft};

Vector2d down(double p) { return Vector2d(std::sin(p), -std::cos(p)); }
Vector2d fwd(double p) { return Vector2d(std::cos(p), std::sin(p)); }

struct Kin { std::array<Vector2d, 4> com; std::array<double, 4> ang; Vector2d heel, toe; };
Kin kin(const Vec6& q) {
    Kin k; Vector2d hip(q(0), q(1)); double th = q(2), p1 = th + q(3), p2 = p1 + q(4), p3 = p2 + q(5);
    Vector2d knee = hip + L_th * down(p1), ankle = knee + L_sh * down(p2);
    k.com = {hip + 0.5 * L_t * Vector2d(-std::sin(th), std::cos(th)), hip + 0.5 * L_th * down(p1),
             knee + 0.5 * L_sh * down(p2), ankle + FCOM * fwd(p3)};
    k.ang = {th, p1, p2, p3};
    k.heel = ankle + HEEL * fwd(p3); k.toe = ankle + TOE * fwd(p3);
    return k;
```

```cpp
}
// Genel sayısal Jacobian: f: R^6 -> R^m
MatrixXd jac(const std::function<VectorXd(const Vec6&)>& f, const Vec6& q, double h = 1e-7) {
    VectorXd f0 = f(q); MatrixXd J(f0.size(), 6);
    for (int i = 0; i < 6; ++i) { Vec6 e = Vec6::Unit(i) * h; J.col(i) = (f(q + e) - f(q - e)) / (2 * h); }
    return J;
}
MatrixXd mass_matrix(const Vec6& q) {
    MatrixXd M = MatrixXd::Zero(6, 6);
    for (int b = 0; b < 4; ++b) {
        MatrixXd Jv = jac([b](const Vec6& s) { return VectorXd(kin(s).com[b]); }, q);
        MatrixXd Jw = jac([b](const Vec6& s) { VectorXd v(1); v(0) = kin(s).ang[b]; return v; }, q);
        M += m_[b] * Jv.transpose() * Jv + I_[b] * Jw.transpose() * Jw;
    }
    return M;
}
VectorXd gravity(const Vec6& q) {
    auto V = [](const Vec6& s) { Kin k = kin(s); VectorXd v(1); v(0) = 0; for (int b = 0; b < 4; ++b) v(0) += m_[b] * g * k.com[b].y(); return v; };
    return jac(V, q).row(0).transpose();
}
MatrixXd Jc(const Vec6& q) {
    return jac([](const Vec6& s) { Kin k = kin(s); VectorXd v(3); v << k.heel, k.toe.y(); return v; }, q);
}
Vec6 standing_q(double thb, double qh, double qk) {
    Vec6 q; q << 0, 0, thb, qh, qk, -(thb + qh + qk);
    Vector2d ankle = kin(q).heel - HEEL * fwd(0.0); q(0) -= ankle.x(); q(1) -= ankle.y(); return q;
}

int main() {
    struct Case { const char* name; double thb, qh, qk; };
    for (const Case& c : {Case{"dik duruş", 0.0, -0.3, 0.6}, Case{"geriye eğik +0.35", 0.35, -0.3, 0.6}, Case{"öne eğik -0.35", -0.35, -0.3, 0.6}}) {
        Vec6 q = standing_q(c.thb, c.qh, c.qk);
        MatrixXd M = mass_matrix(q), J = Jc(q); VectorXd G = gravity(q);
        VectorXd lam = J.leftCols(3).transpose().partialPivLu().solve(G.head(3));      // taban satırları: g_b = J_bᵀ λ
        VectorXd tau = G.tail(3) - J.rightCols(3).transpose() * lam;
        bool ok = lam(1) >= 0 && lam(2) >= 0;
        std::printf("%-18s M00 =%.4f  λz topuk =%+.3f burun =%+.3f N  τ =[%+.4f %+.4f %+.4f]  %s\n", c.name, M(0, 0),
                    lam(1), lam(2), tau(0), tau(1), tau(2), ok ? "DENGEDE" : "DEVRİLİR");
    }
    return 0;
}
```

Çıktı:

```
dik duruş          M00 =1.8000  λz topuk =+13.614 burun =+4.044 N  τ =[-0.0000 -0.7827 +0.1450]  DENGEDE
geriye eğik +0.35 M00 =1.8000  λz topuk =+38.541 burun =-20.883 N  τ =[+0.6055 +0.7379 +2.6377]  DEVRİLİR
öne eğik -0.35    M00 =1.8000  λz topuk =-11.489 burun =+29.147 N  τ =[-0.6055 -2.2084 -2.3653]  DEVRİLİR
```

Python ve C++ notu: fonksiyonları argüman olarak geçmek

- Python'da `jac(lambda s: bodies(s)[0][k][2], q)`: herhangi bir fonksiyonun Jacobian'ını alan genel bir araç. C++'ta aynısı `std::function<VectorXd(const Vec6&)>` parametresi ve

lambda: `[b](const Vec6& s) { return VectorXd(kin(s).com[b]); }`. Köşeli parantez `[b]` lambda'nın dışarıdan `b`'yi *kopyalayarak* yakaladığını söyler.

- **Python lambda tuzağı:** döngü içinde `lambda s: ...[k]...` yazarken `k` *çağrıldığı andaki* değeri kullanılır (geç bağlama). Burada lambda hemen çağrıldığı için sorun yok; saklayıp sonra çağırsaydın hepsi son `k`'yı görürdü. Çözüm: `lambda s, k =k: ....`
- **`std::array` ve `struct`:** sabit sayıda elemanı olan gruplar için (`std::array<Vector2d,4> com`); Python'daki tuple listesinin tip güvenli hali.
- **Blok matris:** numpy `np.block([[M, -J.T], [J, Z]])`; Eigen'de `K << M, -J.transpose(), J, Z;` (önce boyutu ayarla: `MatrixXd K(9, 9)`). Bu satır, yukarıdaki KKT matrisinin birebir kodu.

Alıştırma

(Kâğıt) (a) Topuk -0.03, burun 0.07; kütle merkezi 0.05'te iken λ_h ve λ_t'yi elle bul. Hangi x_c aralığında ikisi de pozitif? **(b)** Havada (temas yok) $\boldsymbol{\lambda} = \mathbf{0}$ ile taban satırlarının kütle merkezinin $\ddot{\mathbf{c}} = (0, -g)$ olduğunu söylediğini göster (ipucu: $M_{xx} = m$ ve momentum). **(c)** 3 temas kısıtının neden bağımsız, 4'ünün neden bağımlı olduğunu $\mathbf{J}_c$ satırlarını yazarak göster (topuk–burun mesafesi sabit). **(d)** Sürtünme katsayısı $\mu = 0.6$ ve $\lambda_z = 17.658$ N: ayak kaymadan en fazla ne kadar yatay kuvvet alabilir? Bu, kütle merkezinin en büyük yatay ivmesine ($a = \mu g$) nasıl karşılık gelir? **(Bilgisayar) (e)** Statik durumda motor ısısını ($\propto \sum \tau^2$) en aza indiren duruşu ($q_{\text{kalça}}, q_{\text{diz}}$ ızgara araması) bul. **(f)** Kütle merkezinin hızı ile toplam momentumu karşılaştır: $m\dot{\mathbf{c}} = \mathbf{M}_{[0:2],:}\dot{\mathbf{q}}$ olduğunu sayısal göster (centroidal momentum, Cilt 6'nın temeli). **(g)** KKT simülasyonunda kısıt sapmasını (ϕ, topuk z'si) ölç; Δt büyüdükçe neden artıyor? Baumgarte stabilizasyonu ($-\dot{\mathbf{J}}_c\dot{\mathbf{q}} - 2\alpha\dot{\phi} - \alpha^2\phi$) ekle.

Bölüm 8

Bitirme Projesi P1: SB-1 Bacak Kütüphanesi

Bu ciltte yazdığın her fonksiyon, SB-1'in (seri boyunca kullanacağın kendi biped modelin) bacak yazılımının çekirdeği. P1'in hedefi bunları **test edilmiş, iki dilde, tek komutla derlenen** bir depoya dönüştürmek. Bu, CV'ne koyabileceğin ilk somut yazılım işi ve Cilt 2'nin (yazılım mühendisliği) giriş pratiği olacak.

8.1 Depo yapısı

```
sb1_kinematik/
├── CMakeLists.txt          # C++ derleme + test tanımı
├── include/robkit.hpp      # C++ kütüphanesi (sadece başlık)
├── tests/test_robkit.cpp   # C++ birim testleri
├── python/robkit.py        # Python kütüphanesi (aynı API)
├── python/test_robkit.py   # Python testleri (pytest ile çalışır)
└── README.md               # ne, nasıl çalıştırılır, ne doğrulandı
```

Bu klasörün çalışan bir kopyası kod arşivinde `proje/sb1_kinematik` olarak var.

8.2 Neden test?

Robot kodunda hata genelde "çökme" olarak değil, *yanlış ama makul görünen sayı* olarak ortaya çıkar: işareti ters bir eksen, $[x, y, z, w]$ yerine $[w, x, y, z]$, unutulmuş bir transpoz. Bunları yakalamanın en ucuz yolu, bu ciltte sürekli yaptığımız şeyi otomatikleştirmek: **iki bağımsız yoldan hesapla ve karşılaştır**.

- **Özdeşlik testleri:** $\mathbf{R}^T\mathbf{R} = \mathbf{I}$, $\det \mathbf{R} = 1$, $\log(\exp \boldsymbol{\omega}) = \boldsymbol{\omega}$, kuaterniyon ve RPY gidiş-dönüşleri.
- **İki algoritma testi:** PoE = elle kurulan zincir; Jacobian = sonlu fark; RNEA = Lagrange (Bölüm 6); IK→FK = hedef (Bölüm 5).
- **Fiziksel testler:** enerji korunumu (sürtünmesiz simülasyon), statik dengede $\sum \lambda_z = mg$ (Bölüm 7).

Testlerin *geçerli bölgesini* düşün: log ancak $|\boldsymbol{\omega}| < \pi$ için tek değerlidir, RPY ancak $|\theta| < \pi/2$ için; rastgele girdileri bu bölgelerden seçmek gerekir (aksi halde testin kendisi yanlış alarm verir).

Python testleri

```python
"""robkit.py için birim testler (pytest ile de çalışır: `pytest c08_tests.py`)"""
import numpy as np
```

```python
from robkit import exp_so3, log_so3, R_to_quat, quat_to_R, rpy_to_R, R_to_rpy, fk_poe, space_jacobian, homog, rot_x, rot_y, inv_T, unskew

rng = np.random.default_rng(42)
S = [np.array([1., 0, 0, 0, 0, 0]), np.array([0, 1., 0, 0, 0, 0]), np.array([0, 1., 0, 0.2, 0, 0])]
M = homog(np.eye(3), [0, 0, -0.4])

def test_so3():
    for _ in range(200):
        w = rng.uniform(-3, 3, 3); w *= min(1, 3.1/np.linalg.norm(w)); R = exp_so3(w)
        assert np.allclose(R.T@R, np.eye(3), atol=1e-12) and abs(np.linalg.det(R) - 1) < 1e-12
        assert np.allclose(log_so3(R), w, atol=1e-9)
        assert np.allclose(quat_to_R(R_to_quat(R)), R, atol=1e-12)
        rpy = rng.uniform(-3, 3, 3)/np.array([2, 2.2, 1]); assert np.allclose(R_to_rpy(rpy_to_R(*rpy)), rpy, atol=1e-9)

def test_leg():
    for _ in range(100):
        q = rng.uniform(-3, 3, 3)/np.array([4, 2, 2]); T = fk_poe(S, M, q)
        Tc = homog(rot_x(q[0]), np.zeros(3))@homog(rot_y(q[1]), np.zeros(3))@homog(np.eye(3), [0, 0, -.2])@homog(rot_y(q[2]), np.zeros(3))@homog(np.eye(3), [0, 0, -.2])
        assert np.allclose(T, Tc, atol=1e-12)
        Js = space_jacobian(S, q); h = 1e-6
        for i in range(3):
            e = np.eye(3)[i]*h; V = (fk_poe(S, M, q + e) - fk_poe(S, M, q - e))/(2*h)@inv_T(T)
            assert np.allclose(np.concatenate([unskew(V[:3, :3]), V[:3, 3]]), Js[:, i], atol=1e-8)

if __name__ == "__main__":
    n = 0
    for f in [test_so3, test_leg]:
        f(); n += 1; print(f"{f.__name__}: geçti")
    print(f"{n} test fonksiyonu geçti")
```

Çıktı:

```
test_so3: geçti
test_leg: geçti
2 test fonksiyonu geçti
```

C++ testleri

```cpp
// c08_tests.cpp — robkit için küçük bir birim test seti (çerçevesiz; Cilt 2'de GoogleTest'e taşınacak)
#include "robkit.hpp"
#include <cstdio>
#include <random>
using namespace rk;

static int n_pass = 0, n_fail = 0;
#define CHECK_NEAR(a, b, tol, name)                                                  \
    do { double _e = std::abs((a) - (b));                                            \
         if (_e <= (tol)) ++n_pass;                                                  \
         else { ++n_fail; std::printf("BAŞARISIZ: %s (hata %.2e > %.0e)\n", name, _e, tol); } } while (0)

int main() {
    std::mt19937 gen(42); std::uniform_real_distribution<double> U(-3.0, 3.0);
    for (int t = 0; t < 200; ++t) {
        Vec3 w(U(gen), U(gen), U(gen));
```

```
        if (w.norm() > 3.1) w *= 3.1 / w.norm();                        // log'un tek değerli olduğu bölge: |w| < π
        Mat3 R = exp_so3(w);
        CHECK_NEAR((R.transpose() * R - Mat3::Identity()).norm(), 0.0, 1e-12, "R ortogonal");
        CHECK_NEAR(R.determinant(), 1.0, 1e-12, "det R = 1");
        CHECK_NEAR((log_so3(R) - w).norm(), 0.0, 1e-9, "log(exp(w)) = w");
        CHECK_NEAR((quat_to_R(R_to_quat(R)) - R).norm(), 0.0, 1e-12, "quat gidiş-dönüş");
        Vec3 rpy(U(gen) / 2, U(gen) / 2.2, U(gen));                     // pitch |.| < π/2 (gimbal lock dışı)
        CHECK_NEAR((R_to_rpy(rpy_to_R(rpy(0), rpy(1), rpy(2))) - rpy).norm(), 0.0, 1e-9, "rpy gidiş-dönüş");
    }
    // Bacak: PoE = zincir, Jacobian = sonlu fark
    std::vector<Vec6> S(3);
    S[0] << 1, 0, 0, 0, 0, 0; S[1] << 0, 1, 0, 0, 0, 0; S[2] << 0, 1, 0, 0.2, 0, 0;
    Mat4 M = homog(Mat3::Identity(), Vec3(0, 0, -0.4));
    for (int t = 0; t < 100; ++t) {
        Eigen::Vector3d q(U(gen) / 4, U(gen) / 2, U(gen) / 2);
        Mat4 T = fk_poe(S, M, q);
        Mat4 Tc = homog(rot_x(q(0)), Vec3::Zero()) * homog(rot_y(q(1)), Vec3::Zero()) * homog(Mat3::Identity(), Vec3(0, 0, -0.2)) *
                  homog(rot_y(q(2)), Vec3::Zero()) * homog(Mat3::Identity(), Vec3(0, 0, -0.2));
        CHECK_NEAR((T - Tc).norm(), 0.0, 1e-12, "PoE = zincir");
        Jac Js = space_jacobian(S, q); const double h = 1e-6;
        for (int i = 0; i < 3; ++i) {
            Eigen::Vector3d e = Eigen::Vector3d::Unit(i) * h;
            Mat4 Td = (fk_poe(S, M, q + e) - fk_poe(S, M, q - e)) / (2 * h);
            Mat4 V = Td * inv_T(T); Vec6 Vs; Vs << unskew(V.topLeftCorner<3, 3>()), V.topRightCorner<3, 1>();
            CHECK_NEAR((Vs - Js.col(i)).norm(), 0.0, 1e-8, "J_s sütunu = sonlu fark");
        }
    }
    std::printf("%d test geçti, %d test başarısız\n", n_pass, n_fail);
    return n_fail == 0 ? 0 : 1;
}
```

Çıktı:

```
1400 test geçti, 0 test başarısız
```

8.3 CMake ile derleme

```
cmake_minimum_required(VERSION 3.16)
project(sb1_kinematik CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
if(NOT CMAKE_BUILD_TYPE)
  set(CMAKE_BUILD_TYPE Release)
endif()

find_package(Eigen3 3.3 QUIET NO_MODULE)          # Ubuntu: sudo apt install libeigen3-dev
add_library(robkit INTERFACE)                      # sadece başlık dosyasından oluşan kütüphane
target_include_directories(robkit INTERFACE include)
if(Eigen3_FOUND)
  target_link_libraries(robkit INTERFACE Eigen3::Eigen)
else()
```

```
  target_include_directories(robkit INTERFACE ${EIGEN_DIR})   # cmake -DEIGEN_DIR =/yol/eigen
endif()

enable_testing()
add_executable(test_robkit tests/test_robkit.cpp)
target_link_libraries(test_robkit PRIVATE robkit)
add_test(NAME robkit_birim_testleri COMMAND test_robkit)
```

Kullanım (kendi bilgisayarında, Eigen kuruluysa):

```
cmake -S . -B build          # yapılandır
cmake --build build -j4      # derle
cd build && ctest            # testleri çalıştır
```

Bu ortamda aldığım çıktı (Eigen sistemde kurulu olmadığı için yolu `-DEIGEN_DIR` ile verdim): **Çıktı:**

```
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done (0.3s)
-- Generating done (0.0s)
-- Build files have been written to: <ust_klasor>/seri/cilt1/proje/sb1_kinematik/build
[ 50%] Building CXX object CMakeFiles/test_robkit.dir/tests/test_robkit.cpp.o
[100%] Linking CXX executable test_robkit
[100%] Built target test_robkit
1/1 Test #1: robkit_birim_testleri .............   Passed    0.01 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =   0.01 sec
```

8.4 P1 kontrol listesi

1. Kodu bu PDF'ten *kopyalamadan*, kendi elinle yeniden yaz (bakarak olur, yapıştırarak olmaz).
2. Ayak bileği eklemini ekle (Bölüm 3 ve 5 alıştırmaları) ve testlerini yaz.
3. RNEA'nın (Bölüm 6) 3 uzuvlu sürümünü kütüphaneye ekle; Python ve C++ çıktılarını karşılaştıran bir test yaz (aynı girdiler bir dosyadan okunur).
4. README: bir şekil (Bölüm 4'teki elipsler veya Bölüm 5'teki çömelme), testlerin listesi, "Python ve C++ 10^{-15} içinde aynı" gibi somut cümleler.
5. GitHub'a koy; her değişiklikte testleri çalıştıran bir GitHub Actions dosyası ekle (Cilt 2'de adım adım).

Python ve C++ notu: makro, `do {} while(0)` ve dönüş kodu

- `CHECK_NEAR` bir C önişlemci makrosu; gövdesinin `do { ... } while (0)` içinde olması, makronun `if (x) CHECK_NEAR(...); else ...` gibi yerlerde tek bir ifade gibi davranmasını sağlar (C'den bildiğin bir hile).
- `return n_fail == 0 ? 0 : 1;` programın çıkış kodu; `ctest` ve CI sistemleri testin geçip geçmediğini buna bakarak anlar.

- Python'da `assert np.allclose(a, b, atol =...)`; `pytest test_` ile başlayan fonksiyonları otomatik bulur ve çalıştırır.